

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

NGUYỄN ĐỨC TRUNG
PHẠM QUỐC TIẾN

KHÓA LUẬN TỐT NGHIỆP
PHÁT TRIỂN ROBOT DI ĐỘNG ĐA HƯỚNG THU THẬP
VẬT THỂ THÔNG MINH

**Developing an omnidirectional mobile robot for intelligent object
gathering**

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

NGUYỄN ĐỨC TRUNG - 22521564

PHẠM QUỐC TIẾN - 22521472

KHÓA LUẬN TỐT NGHIỆP
PHÁT TRIỂN ROBOT DI ĐỘNG ĐA HƯỚNG THU THẬP
VẬT THỂ THÔNG MINH

**Developing an omnidirectional mobile robot for intelligent object
gathering**

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

GIẢNG VIÊN HƯỚNG DẪN
THS. PHẠM MINH QUÂN

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số
ngày của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Lời đầu tiên, nhóm xin được bày tỏ lòng biết ơn chân thành và sâu sắc nhất đến toàn thể quý thầy/cô tại Trường Đại học Công nghệ Thông tin – ĐHQG-HCM. Trong suốt những năm tháng học tập dưới mái trường này, nhóm không chỉ được tiếp cận với những kiến thức chuyên môn cốt lõi, nền tảng vững chắc cho con đường sự nghiệp sau này, mà còn nhận được sự dạy dỗ tận tâm về đạo đức và tư duy. Những bài giảng trên lớp của quý thầy/cô chính là bệ phóng giúp nhóm tích lũy được những kỹ năng mềm quan trọng, sự tự tin và tác phong làm việc chuyên nghiệp để sẵn sàng thích nghi với môi trường làm việc thực tế tại các doanh nghiệp.

Đặc biệt, nhóm xin gửi lời cảm ơn đặc biệt tới quý thầy/cô thuộc Khoa Kỹ thuật Máy tính. Chính sự nhiệt huyết và những chia sẻ tâm huyết của quý thầy/cô đã khơi dậy và thắp sáng ngọn lửa đam mê trong chúng em đối với lĩnh vực Hệ thống nhúng nói riêng và ngành Công nghệ thông tin nói chung. Nhóm vô cùng trân trọng việc Khoa đã luôn tạo điều kiện thuận lợi nhất, hỗ trợ nhóm trong việc tiếp cận, trải nghiệm và triển khai khóa luận trên các bộ Kit nhúng hiện đại, có giá trị cao. Đây là những cơ hội quý báu giúp nhóm rút ngắn khoảng cách giữa lý thuyết và thực tiễn, gặt hái được những kinh nghiệm thực tế mà không một trang sách nào có thể thay thế được.

Sau cùng, với tất cả lòng kính trọng, nhóm xin gửi lời cảm ơn sâu sắc nhất đến thầy Phạm Minh Quân. Trong suốt quá trình thực hiện khóa luận, thầy đã luôn dành thời gian quý báu để sát cánh, đưa ra những lời khuyên kịp thời, những chỉ dẫn chuyên môn tận tình và những định hướng đúng đắn. Sự kiên nhẫn và giúp đỡ nhiệt tâm của thầy chính là động lực lớn nhất để nhóm vượt qua những khó khăn, thử thách nhằm hoàn thành khóa luận này một cách trọn vẹn nhất.

MỤC LỤC

Chương 1.	TỔNG QUAN ĐỀ TÀI.....	3
1.1.	Giới thiệu đề tài.....	3
1.2.	Tình hình nghiên cứu	4
1.2.1.	Ngoài nước.....	4
1.2.2.	Trong nước.....	6
Chương 2.	CƠ SỞ LÝ THUYẾT.....	9
2.1.	Thuật toán điều khiển động cơ DC	9
2.1.1.	Mô hình toán học động cơ DC.....	9
2.1.2.	Mô hình điều khiển PI kết hợp Feedforward	10
2.1.3.	Phương pháp gán cực – Pole placement	11
2.2.	Mô hình toán học robot sử dụng bánh mecanum.....	12
2.2.1.	Giới thiệu về mô hình toán học của bánh xe mecanum.....	12
2.2.2.	Phương trình động học robot sử dụng bánh mecanum	14
2.3.	Bộ lọc Kalman mở rộng	17
2.3.1.	Tổng quan về bộ lọc Kalman mở rộng	17
2.3.2.	Thuật toán của bộ lọc Kalman mở rộng.....	17
2.3.3.	Nhận xét	19
2.4.	Hệ thống thị giác ngữ nghĩa trên thiết bị biên.....	20
2.4.1.	Phát hiện vật thể với YOLOv8	20
2.4.2.	Mô hình ngôn ngữ - thị giác kiến trúc Transformer	21
2.4.3.	Vision Transformer (ViT).....	21
2.4.4.	Kiến trúc mô hình VLM Qwen2.5-VL-3B	22
2.4.5.	Lượng tử hóa mô hình.....	22

2.4.6.	Định dạng GGUF và biến thể Q4_K_M.....	23
2.4.7.	Engine suy luận llama.cpp	24
2.4.8.	Tối ưu CPU và GPU	25
2.4.9.	KV cache và độ dài ngữ cảnh	25
2.4.10.	API tương thích OpenAI và hỗ trợ đa phương thức.....	25
2.5.	Nền tảng ROS2.....	26
2.5.1.	Giới thiệu tổng quan	26
2.5.2.	Cấu trúc và cơ chế hoạt động.....	27
2.6.	Hệ thống điều hướng Navigation2 – Nav2	30
2.7.	Thuật toán tự động khám phá - Frontier-base exploration.....	33
2.7.1.	Giới thiệu tổng quan	33
2.7.2.	Thuật toán Wavefront Frontier Detector – WFD.....	34
Chương 3.	PHÂN TÍCH THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG.....	36
3.1.	Thiết kế hệ thống phần cứng	36
3.1.1.	Tổng quan về các linh kiện	36
3.1.2.	Sơ đồ kết nối và phân bổ tài nguyên phần cứng	40
3.2.	Camera chiều sâu Orbbec Astra	44
3.2.1.	Nguyên lý cảm biến chiều sâu	44
3.2.2.	Mô hình camera pinhole và phép chiếu ngược	45
3.2.3.	Căn chỉnh depth-RGB và quy ước hệ tọa độ	45
3.3.	Kiến trúc phần cứng NVIDIA Jetson AGX Xavier	46
3.4.	Thiết kế hệ thống phần mềm	47
3.4.1.	Tổng quan kiến trúc hệ thống	47
3.4.2.	Kiến trúc tổng thể ba lớp.....	48

3.4.3.	Lớp Vision: pipeline phát hiện và cơ chế kích hoạt	49
3.4.4.	Lớp Brain: tích hợp VLM qua llama.cpp	50
3.4.5.	Lựa chọn thiết kế cho thiết bị nhúng Jetson Xavier.....	52
3.4.6.	Tích hợp ở cấp service	53
3.5.	Tích hợp camera chiều sâu Astra	53
3.5.1.	Vấn đề và động cơ thiết kế.....	53
3.5.2.	Lớp trừu tượng camera (Camera Abstraction Layer)	54
3.5.3.	Tương thích ngược và truyền metadata chiều sâu	54
3.6.	Phân hệ định vị và điều hướng tự hành ROS2	55
3.6.1.	ROS2 và ranh giới phân hệ thị giác	55
3.6.2.	Cây biến đổi tọa độ và mô tả robot.....	55
3.6.3.	Pipeline cảm biến - detph image tới LaserScan.....	56
3.6.4.	Cấu hình SLAM và NAV2	57
3.6.5.	Giao tiếp với đế di động Mecanum qua TCP	58
3.7.	Hiện thực hệ thống	59
3.7.1.	Môi trường phát triển và triển khai.....	59
3.7.2.	Hiện thực Lớp 1 — Vision Node.....	59
3.7.3.	Hiện thực Lớp 2 — Brain Server.....	60
3.7.4.	Hiện thực phân hệ ROS2	60
3.7.5.	Hiển thị, ghi nhật ký và giám sát	60
3.8.	Tích hợp end-to-end	60
3.8.1.	Nguyên lý điều phối.....	60
3.8.2.	Máy trạng thái của node điều phối.....	61
Chương 4.	ĐÁNH GIÁ KẾT QUẢ THỰC NGHIỆM	63

4.1.	Thiết lập thực nghiệm.....	63
4.2.	Đánh giá YOLOv8n	63
4.3.	Đánh giá VLM Qwen2.5-VL-3B (Q4_K_M)	64
4.4.	So sánh với baseline	66
4.5.	Độ trễ end-to-end.....	67
4.6.	Phân tích nhiệt độ và công suất trên Jetson.....	68
4.7.	Tích hợp camera chiều sâu Astra	69
4.8.	Cơ chế kích hoạt ổn định StableTrigger.....	70
4.9.	Hệ thống điều hướng tự hành trên ROS2	71
4.9.1.	Pipeline cảm biến: DepthImageToLaser.....	72
4.9.2.	Ràng buộc USB 2.0 và cơ chế phân pha camera	72
4.10.	Trực quan hóa trên Foxglove Studio	73
4.11.	Kịch bản demo thu gom vật thể end-to-end	74
Chương 5.	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	75
5.1.	Kết luận	75
5.2.	Hạn chế.....	76
5.3.	Hướng phát triển.....	76
TÀI LIỆU THAM KHẢO.....		78
PHỤ LỤC.....		84
Phụ lục A. Prompt tiếng Việt cho Qwen2.5-VL		84

DANH MỤC HÌNH

Hình 1.1: Máy in 3D vẫn thực thi ngay cả khi đã in lỗi [1]	3
Hình 1.2: Robot Atlas của Boston Dynamics [2]	5
Hình 1.3: Robot ANYmal của ANYbotics [3]	5
Hình 1.4: Dự án Mobile ALOHA của đại học Stanford [4]	6
Hình 1.5: Công nghệ Touch2Park của VinAI [5]	7
Hình 1.6: VIBOT của Học viện Kỹ thuật Quân sự [6]	7
Hình 1.7: Hệ thống robot tự hành quan trắc mức phóng xạ trong các môi trường có bức xạ hạt nhân của nhóm các nhà nghiên cứu Viện Vật lý - Viện Hàn lâm Khoa học và Công nghệ Việt Nam [7]	8
Hình 2.1: Mô hình toán học của bánh xe mecanum	12
Hình 2.2: Mô hình robot bốn bánh xe mecanum [9]	15
Hình 2.3: Kiến trúc mô hình Qwen2.5-VL-3B [10]	22
Hình 2.4: Lượng tử hóa affine bất đối xứng từ FP32 sang INT8 [11]	23
Hình 2.5: Logo ROS [12]	26
Hình 2.6: Sơ đồ giao tiếp qua topics [12]	28
Hình 2.7: Sơ đồ giao tiếp qua services [12]	29
Hình 2.8: Sơ đồ giao tiếp qua actions [13]	29
Hình 2.9: Các khung tọa độ trên robot [13]	30
Hình 2.10: Sơ đồ cấu trúc của Nav2 [14]	32
Hình 3.1: Cảm biến BNO055 [16]	36
Hình 3.2: Cảm biến PMW3901 [17]	37
Hình 3.3: Động cơ JGB37-545B [18]	38
Hình 3.4: Module BTS7960 [19]	38
Hình 3.5: Vi điều khiển ESP32 S3 [20]	39
Hình 3.6: Pin Lipo được nhóm sử dụng [21]	40
Hình 3.7: Sơ đồ kết nối của nguồn 12V	41
Hình 3.8: Sơ đồ kết nối của mạch mở rộng 12V	41
Hình 3.9: Hình ảnh hoàn thiện của mạch mở rộng 12V	41

Hình 3.10: Sơ đồ đi dây của mạch mở rộng 5V.....	42
Hình 3.11: Hình ảnh hoàn thiện của mạch mở rộng 5V	42
Hình 3.12: Sơ đồ kết nối của nguồn 5V.....	43
Hình 3.13: Sơ đồ kết nối của đế mecanum	44
Hình 3.14: Camera Orbbec Astra [24].....	44
Hình 3.15: Jetson AGX Xavier x211 [25]	46
Hình 3.16: Sơ đồ tổng quan kiến trúc hệ thống	47
Hình 3.17: Sơ đồ kiến trúc ba lớp của hệ thống robot thu gom.	49
Hình 3.18: Lưu đồ giải thuật của thuật toán StableTrigger	49
Hình 3.19: Pipeline xử lý lớp Brain	51
Hình 3.20: Cấu trúc lớp trừu tượng camera	54
Hình 3.21: Cấu trúc dữ liệu truyền đi sau khi detect vật thể.....	54
Hình 3.22: Cây TF thiết kế theo chuẩn REP-105 [13].....	55
Hình 3.23: Cây TF dựng lại của hệ thống dựa theo chuẩn REP-105.....	55
Hình 3.24: Pipeline cảm biến – Depth Image to Laser Scan	56
Hình 3.25: Pipeline điều hướng runtime.....	57
Hình 3.26: Sơ đồ kết nối TCP và khung dữ liệu đế Mecanum	58
Hình 3.27: Máy trạng thái của node điều phối.....	61
Hình 4.1: FPS trung bình của YOLOv8n theo cấu hình phần cứng	64
Hình 4.2: Tỷ lệ kích hoạt theo từng cấp của cơ chế JSON fallback ba cấp.....	65
Hình 4.3: Độ trễ suy luận VLM theo số lớp offload lên GPU	66
Hình 4.4: Độ chính xác collectibility theo từng lớp vật thể.....	67
Hình 4.5: Phân rã độ trễ end-to-end theo giai đoạn	67
Hình 4.6: Nhiệt độ GPU và CPU theo thời gian trong phiên đo.....	68
Hình 4.7: Công suất tiêu thụ theo thời gian ở load tải bền vững	69
Hình 4.8: Độ nhiễu Depth theo cự ly	69
Hình 4.9: Tỷ lệ pixel depth hợp lệ theo cự ly	70
Hình 4.10: Timeline StableTrigger đo trực tiếp trên Astra.....	70
Hình 4.11: Phân bố khoảng cách của 110/640 tia hữu hạn trên /scan.	72

Hình 4.12: Độ trễ chuyển chế độ camera (depth/color/both).....	73
Hình 4.13: Trực quan hóa tích hợp trên Foxglove.....	73

DANH MỤC BẢNG

Bảng 2.1: Thông số của các bánh xe mecanum	15
Bảng 2.2: Trade-off của các mức lượng tử hóa GGUF trên Qwen2.5-VL-3B.....	24
Bảng 3.1: Thông số phần cứng chính của Jetson AGX Xavier 32 GB. [25].....	46
Bảng 3.2: Ngân sách bộ nhớ trên Jetson AGX Xavier 16 GB.....	52
Bảng 3.3: So sánh vấn đề sử dụng camera.....	53
Bảng 4.1: Hiệu năng YOLOv8n trên PC và Jetson Xavier.....	63
Bảng 4.2: Độ trễ và tài nguyên Qwen2.5-VL-3B Q4_K_M theo cấu hình.	65
Bảng 4.3: So sánh hệ đề xuất với hai baseline trên cùng tập 50 ảnh.	66
Bảng 4.4: Chỉ số edge (tegrastats, 9700 mẫu @2s, phiên ~5,5h)	68
Bảng 4.5: Đặc trưng camera Astra đo đạc thực tế	69
Bảng 4.6: Bẫy gói ROS2.....	71

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Nguyên nghĩa
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	ARM	Advanced RISC Machine
4	CNN	Convolutional Neural Network
5	COCO	Common Objects in Context (dataset)
6	CPU	Central Processing Unit
7	CUDA	Compute Unified Device Architecture
8	DVFS	Dynamic Voltage Frequency Scaling
9	eMMC	embedded Multi-Media Card
10	FastAPI	Fast Application Programming Interface (Python framework)
11	FP16/FP32	Floating Point 16-bit / 32-bit
12	FPS	Frames Per Second
13	GGUF	GPT-Generated Unified Format
14	GPU	Graphics Processing Unit
15	HTTP	HyperText Transfer Protocol
16	IK	Inverse Kinematics
17	JSON	JavaScript Object Notation
18	KLTN	Khóa luận tốt nghiệp
19	KV cache	Key-Value cache
20	L4T	Linux for Tegra
21	LLM	Large Language Model
22	mAP	mean Average Precision
23	MAXN	Maximum Performance Power Mode (Jetson)
24	NMS	Non-Maximum Suppression
25	ONNX	Open Neural Network Exchange

26	REST	Representational State Transfer
27	RGB	Red Green Blue
28	ROS	Robot Operating System
29	SDK	Software Development Kit
30	TDP	Thermal Design Power
31	ViT	Vision Transformer
32	VLM	Vision-Language Model
33	VRAM	Video Random Access Memory
34	YOLO	You Only Look Once

TÓM TẮT KHÓA LUẬN

Trong bối cảnh công nghiệp 5.0 và sự phát triển vượt bậc của trí tuệ nhân tạo, robot ngày càng được kỳ vọng chuyển từ mô hình lập trình cố định, cứng nhắc sang khả năng tự thích nghi với môi trường chưa được cấu trúc hóa. Khóa luận "Phát triển robot di động đa hướng thu thập vật thể thông minh" hướng tới việc xây dựng nền tảng cho một robot di động đa hướng có khả năng vừa di chuyển linh hoạt, vừa hiểu chỉ dẫn bằng ngôn ngữ tự nhiên và nhận diện vật thể mục tiêu một cách thông minh, làm tiền đề cho các ứng dụng kho vận, y tế và sản xuất linh hoạt. Trong phạm vi khóa luận, nhóm tập trung hiện thực và kiểm chứng hai trụ cột song song, bổ trợ cho nhau: nền tảng cơ điện tử – điều khiển chuyển động cho đế di động bốn bánh mecanum, và hệ thống thị giác ngữ nghĩa chạy trên thiết bị biên.

Điểm đầu tiên, nền tảng cơ điện tử và điều khiển chuyển động. Khóa luận xây dựng mô hình toán học động cơ DC và thiết kế bộ điều khiển PI kết hợp Feedforward, sử dụng phương pháp gán cực (pole placement) để đảm bảo đáp ứng nhanh, không vọt lố và triệt tiêu sai số tĩnh. Trên cơ sở cấu tạo bánh xe mecanum, nhóm thiết lập mô hình động học thuận và nghịch cho robot bốn bánh bố trí đối xứng (thông qua ma trận Jacobian), cho phép robot di chuyển đa hướng theo quỹ đạo thẳng và cong mà không cần đổi góc hướng. Bộ lọc Kalman mở rộng (EKF) được áp dụng để ước lượng trạng thái và định vị robot ổn định trong môi trường có nhiễu quá trình và nhiễu cảm biến. Phần thực nghiệm của trụ cột này tập trung kiểm chứng năng lực di chuyển đa hướng của đế robot — bám quỹ đạo, ổn định vận tốc và định hướng — làm nền tảng cho việc tích hợp cánh tay máy ở giai đoạn sau.

Điểm thứ hai, hệ thống thị giác ngữ nghĩa trên thiết bị biên (edge). Hệ thống được tổ chức theo kiến trúc ba lớp giao tiếp qua REST API: Lớp *Vision* (phát hiện vật thể bằng YOLOv8n và cơ chế kích hoạt ổn định), Lớp *Brain* (suy luận ngữ nghĩa bằng mô hình thị giác-ngôn ngữ Qwen2.5-VL-3B), và Lớp *Control* (chuyển lệnh điều khiển tới robot). Việc tách lớp mang lại khả năng kiểm thử độc lập qua *mock mode* và triển khai phân tán. Để giải quyết xung đột giữa luồng camera 30 FPS và độ trễ 4–8

giây của mô hình thị giác-ngôn ngữ, nhóm đề xuất cơ chế StableTrigger dạng máy trạng thái hữu hạn ($T_{\text{stable}} = 2 \text{ s}$, $T_{\text{cool}} = 5 \text{ s}$), loại bỏ hoàn toàn kích hoạt sai (false-positive) trong thực nghiệm sơ bộ. Điểm nhấn kỹ thuật là việc nhúng thành công Qwen2.5-VL-3B lên NVIDIA Jetson AGX Xavier 16 GB trong giới hạn tài nguyên khắc nghiệt: mô hình được lượng tử hóa về định dạng GGUF mức Q4_K_M ($\approx 2,1 \text{ GB}$) và chạy trên engine suy luận llama.cpp với backend CUDA cho kiến trúc ARM64. Khóa luận tải liệu hóa đầy đủ quy trình biên dịch, danh sách lỗi và cách khắc phục, chiến lược GPU offloading, phân bổ bộ nhớ thống nhất 16 GB, prompt tiếng Việt với cơ chế phân tích JSON ba cấp, cùng ba dịch vụ *systemd* tự phục hồi giúp hệ thống vận hành liên tục, ngoại tuyến, không phụ thuộc kết nối đám mây.

Kết quả và tích hợp. Toàn bộ hệ thống đã được tích hợp theo kiến trúc ba lớp Vision–Brain–Control và kiểm chứng theo luồng đầu cuối (end-to-end): robot di chuyển trong không gian làm việc, phát hiện vật bằng YOLOv8n, xác nhận ngữ nghĩa bằng VLM, sau đó căn cứ tọa độ mục tiêu để điều hướng để di động tiếp cận vật. Thực nghiệm cho thấy YOLOv8n đạt $\sim 106 \text{ FPS}$ trên PC (vượt mục tiêu $\geq 15 \text{ FPS}$), cơ chế fallback JSON đạt tỉ lệ thành công 100%, và thiết kế *hybrid* (YOLO + VLM có trigger) giữ được khả năng suy luận ngữ nghĩa mà không phải trả giá bằng tải tính toán liên tục như phương án chỉ dùng VLM; phần điều khiển động học được kiểm chứng trên để robot mecanum thực tế. Khóa luận thẳng thắn nêu các hạn chế — độ trễ VLM còn ở mức vài giây, tập dữ liệu kiểm thử còn nhỏ, hệ mới xử lý một vật mỗi chu kỳ, và chưa tích hợp thao tác gấp vật bằng cánh tay máy — đồng thời đề xuất các hướng phát triển tiếp theo: bổ sung và kiểm thử cánh tay máy cùng vòng điều khiển phản hồi thị giác (visual servoing) để hoàn tất thao tác gấp – thu gom, tích hợp camera 3D, tăng tốc VLM bằng speculative decoding/TensorRT-LLM, xuất YOLO sang TensorRT engine và hỗ trợ đa vật thể.

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Trong kỷ nguyên công nghiệp 5.0 và sự phát triển vượt bậc của trí tuệ nhân tạo trong vài năm gần đây, robot không chỉ còn là những cỗ máy hoạt động theo một kịch bản cứng nhắc (Hình 1.1) mà chúng có thể tự học hỏi thông qua các cảm biến môi trường, camera để tự tối ưu hóa hoạt động của bản thân để hoàn thành mục đích.



Hình 1.1: Máy in 3D vẫn thực thi ngay cả khi đã in lỗi [1]

Nhờ đó, các công việc có tính linh hoạt cao cũng dần được đáp ứng. Một minh chứng rõ ràng là robot tự hành tích hợp cánh tay vừa có sự linh hoạt trong di chuyển cũng như có thể tác động tới môi trường xung quanh thì việc tích hợp thêm phương pháp cho bản thân nó có khả năng thích nghi đối với môi trường chưa được lập cấu trúc sẽ giúp tăng khả năng tự động hóa, giảm chi phí và thời gian sản xuất so với robot được lập trình cố định cùng loại.

Dù vậy, tiềm năng đó không đến mà không có thách thức. Việc chuyển từ lập trình cố định sang hệ thống tự thích nghi thường đi kèm với sự đánh đổi về độ chính xác lặp lại và tính ổn định. Trong môi trường công nghiệp, nơi sai số yêu cầu thường ở mức milimet, một sai số nhỏ trong khâu nhận diện môi trường của trí tuệ nhân tạo có thể dẫn đến những va chạm ngoài ý muốn hoặc hư hỏng hàng hóa, điều mà các hệ thống lập trình cứng vốn kiểm soát rất tốt trong môi trường xác định.

Xuất phát từ bài toán đó, đề tài “PHÁT TRIỂN ROBOT DI ĐỘNG ĐA HƯỚNG THU THẬP VẬT THỂ THÔNG MINH” mà nhóm đề ra nhằm hướng tới việc xây dựng một nền tảng robot hỗ trợ đắc lực trong các kho hàng logistic, bệnh viện hoặc môi trường sản xuất linh hoạt nơi mà sự tương tác giữa người và máy cần sự trực quan và hiệu quả cao.

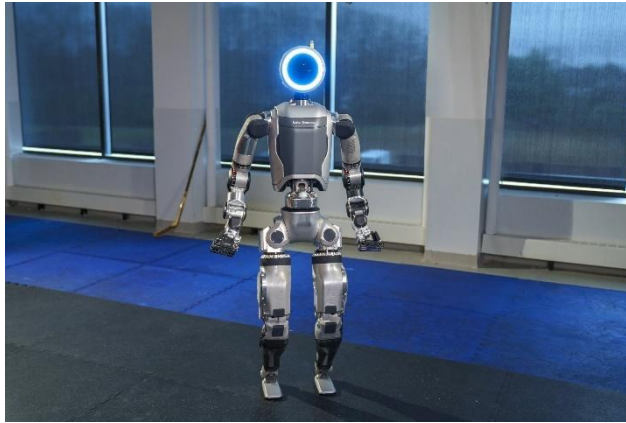
Đề tài này tập trung nghiên cứu và phát triển hệ thống mobile manipulator tích mô hình thị giác (Vision-Language Model - VLM). Sự kết hợp này cho phép robot không chỉ di chuyển đa hướng và thao tác linh hoạt với cánh tay máy, mà có khả năng hiểu các chỉ dẫn bằng ngôn ngữ tự nhiên và nhận diện vật thể mục tiêu một cách thông minh, giúp tối ưu hóa quy trình thu thập vật thể tự động.

1.2. Tình hình nghiên cứu

1.2.1. Ngoài nước

Trên thế giới, nhiều loại robot tự học đã được phát triển và ứng dụng trong nhiều lĩnh vực đa dạng từ công nghiệp, logistic cho đến dịch vụ tương tác với con người. Chúng không chỉ là những sản phẩm thể hiện khả năng vận hành hiệu quả mà còn đóng góp trong công trình nghiên cứu về giải pháp robot tự học.

Robot Atlas của Boston Dynamics là loại robot hình người tiên tiến, được thiết kế cho các ứng dụng công nghiệp, hậu cần và kho bãi. Nó có khả năng hoạt động độc lập không cần tác động của con người, linh hoạt và bền bỉ. Nó được trang bị các cảm biến nhận diện môi trường như camera chiều sâu, LiDAR, cảm biến IMU, cảm biến xúc giác, ... kèm với đó là hệ thống AI để vận động linh hoạt và tương tác chính xác với môi trường.



Hình 1.2: Robot Atlas của Boston Dynamics [2]

Robot ANYmal của ANYbotics là một robot bốn chân được thiết kế để thay thế con người thực hiện các nhiệm vụ giám sát định kỳ tại các nhà máy dầu khí, năng lượng, hóa chất và khai thác mỏ. Robot này được tích hợp cảm biến LiDAR, 6 camera chiều sâu, camera nhiệt, micro siêu âm và cảm biến khí gas để có thể điều hướng và tránh vật cản, đồng thời phát hiện các tính trạng bất thường trong môi trường làm việc. Các dữ liệu cảm biến trên sẽ được xử lý bởi nhiều hệ thống AI khác nhau nhằm cho ra các hành động thức ứng với hoàn cảnh hiện tại mà robot đang đối mặt.



Hình 1.3: Robot ANYmal của ANYbotics [3]

Dự án Mobile ALOHA của đại học Stanford là một hệ thống mobile manipulator chi phí thấp nhưng cực hiệu quả, có thể thực hiện nhiều tác vụ không tên như: đổ rác, dùng khăn lau vết bẩn hay mở tự mở cửa tầng. Dự án này sử dụng phương pháp học máy từ dữ liệu trình diễn (Imitation Learning) để tự học lấy các hành động từ tác vụ theo sự điều khiển lặp lại nhiều lần của con người, sau đó robot có thể tự hành động

để thực thi tác vụ. Nó không chỉ đơn giản là bắt chước theo động tác mà còn học cách làm sao để hoàn thành được tác vụ tốt nhất. Đây là nền tảng truyền cảm hứng trực tiếp cho hướng nghiên cứu tích hợp VLM vào mobile manipulator trong đề tài này.

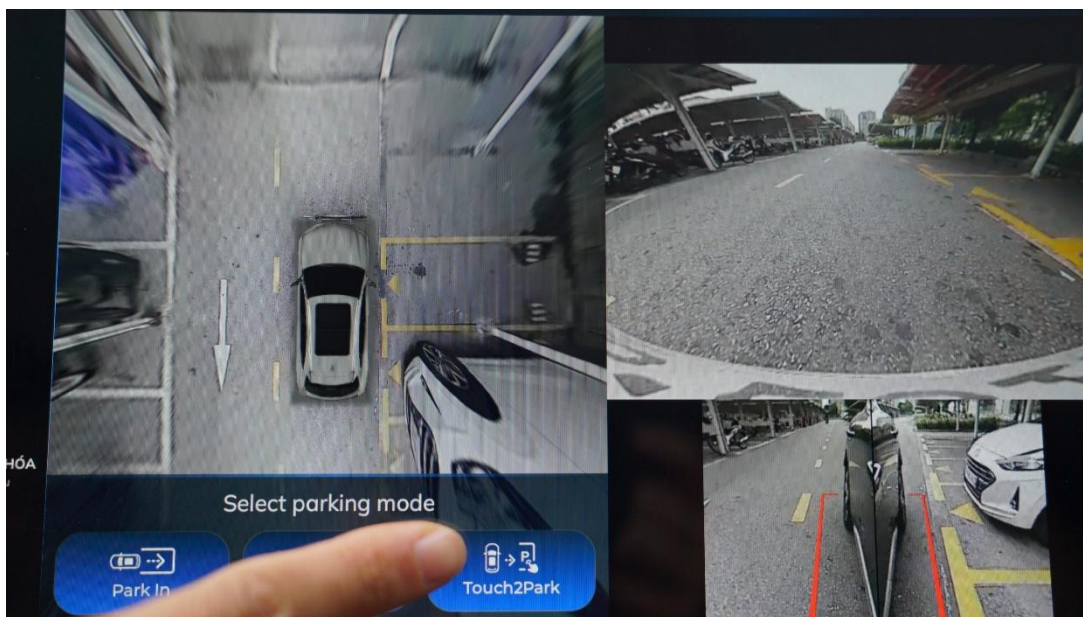


Hình 1.4: Dự án Mobile ALOHA của đại học Stanford [4]

1.2.2. Trong nước

Ở Việt Nam, việc nghiên cứu và phát triển các hệ thống robot tự học cũng từng bước được thực hiện, chủ yếu tập trung tại các viện nghiên cứu, đại học lớn và các tập đoàn công nghệ.

Touch2Park của *VinAI* là giải pháp đỗ xe tự động ADAS cấp độ 2, từng đạt giải thưởng tại AutoTech Breakthrough Awards 2024. Công nghệ này chủ yếu dựa trên 4 camera mắt cá và thuật toán AI để nhìn và hiểu môi trường. Nó có thể phát hiện chướng ngại vật sát xe (0cm ở hai bên và 5 – 10 cm ở trước/sau) và có thể xử lý được nhiều kiểu đỗ ngay cả trong điều kiện thiếu sáng hoặc không gian hẹp.



Hình 1.5: Công nghệ Touch2Park của VinAI [5]

VIBOT của Học viện Kỹ thuật Quân sự được ra mắt trong thời gian chống dịch COVID-19 với mục tiêu giúp vận chuyển cơm, thuốc men và nhu yếu phẩm trong các khu cách ly, bệnh viện dã chiến nhằm giảm rủi ro lây nhiễm cho nhân viên y tế. Robot có thể làm việc liên tục 12 tiếng, tải trọng lên tới 100kg và có khả năng thay thế sức lao động của 3 – 5 nhân viên y tế.



Hình 1.6: VIBOT của Học viện Kỹ thuật Quân sự [6]

Hệ thống robot tự hành quan trắc mức phóng xạ trong các môi trường có bức xạ hạt nhân của nhóm các nhà nghiên cứu Viện Vật lý - Viện Hàn lâm Khoa học và Công nghệ Việt Nam là robot có khả năng quan trắc mức phóng xạ và hỗ trợ thực hiện các thao tác kỹ thuật trong môi trường hạt nhân. Robot sử dụng thuật toán SLAM để tự định vị, tái tạo môi trường làm việc và gắn thông tin bức xạ lên bản đồ. Hơn nữa robot còn tích hợp cánh tay máy để thực hiện các thao tác kỹ thuật, kiểm tra trong môi trường bức xạ cao.



Hình 1.7: Hệ thống robot tự hành quan trắc mức phóng xạ trong các môi trường có bức xạ hạt nhân của nhóm các nhà nghiên cứu Viện Vật lý - Viện Hàn lâm Khoa học và Công nghệ Việt Nam [7]

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Thuật toán điều khiển động cơ DC

2.1.1. Mô hình toán học động cơ DC

Để thiết kế bộ điều khiển vận tốc chính xác cho robot di động đa hướng, việc xây dựng mô hình toán học mô tả đặc tính động học của động cơ DC là bước đầu tiên. Dựa trên phương trình cân bằng cho hệ cơ học quay [8], phương trình vi phân tổng quát mô tả đặc tính cơ học của động cơ DC khi $L = 0$ là:

$$J \frac{d\omega(t)}{dt} + B\omega(t) = \tau(t) \quad (2.1)$$

Trong đó:

J : Mô-men quán tính của động cơ

B : Hệ số ma sát nhớt

$\omega(t)$: Vận tốc góc của động cơ

$\tau(t)$: Mô-men điện từ do dòng điện sinh ra

Xét trạng thái động cơ xoay đều, ta có:

$$\begin{aligned} \frac{d\omega(t)}{dt} &= 0 \\ \rightarrow B\omega_{ss} &= \tau_{ss} \\ \leftrightarrow B &= \frac{\tau_{ss}}{\omega_{ss}} \end{aligned} \quad (2.2)$$

Trong đó:

ω_{ss} : Vận tốc góc khi động cơ xoay đều

τ_{ss} : Mô-men điện từ khi động cơ xoay đều

Xét trạng thái động cơ đang gia tốc, mô-men quán tính J được tính như sau:

$$J = \frac{(\tau(t) - B\omega(t))dt}{d\omega(t)} \quad (2.3)$$

2.1.2. Mô hình điều khiển PI kết hợp Feedforward

Với phương trình (2.1), ta thực hiện biến đổi Laplace hai vế với điều kiện ban đầu bằng 0, ta thu được hàm truyền vòng hở từ điện áp $U(s)$ sang vận tốc góc $\Omega(s)$:

$$G_p(s) = \frac{\Omega(s)}{U(s)} = \frac{1}{Js + B} = \frac{\frac{1}{B}}{\frac{J}{B}s + 1} \quad (2.4)$$

Ta có thể thấy đây là một hệ quán tính bậc nhất với hằng số thời gian $\tau_m = \frac{J}{B}$ và hệ số khuếch đại tĩnh $K_m = \frac{1}{B}$

Xét các thành phần bộ điều khiển PID:

- Thành phần tỉ lệ (P): Tạo ra tín hiệu điều khiển tỉ lệ thuận với sai số hiện tại, quyết định tốc độ đáp ứng nhanh hay chậm và giúp thu hẹp sai lệch giữa giá trị thực tế và giá trị mong muốn.
- Thành phần tích phân (I): Tích lũy sai số theo thời gian, đóng vai trò triệt tiêu hoàn toàn sai số tĩnh do các thành phần phi tuyến hoặc tải trọng gây ra.
- Thành phần đạo hàm (D): Hoạt động dựa trên tốc độ thay đổi của sai số nhằm dự báo hướng đi của hệ thống, giúp giảm độ vọt lố. Tuy nhiên, trong thực tế điều khiển động cơ, vận tốc phản hồi tính toán từ bộ encoder thường bị nhiễu răng cưa ở tần số cao. Nếu áp dụng thành phần D, toán tử đạo hàm sẽ khuếch đại các xung nhiễu này lên gấp nhiều lần, dẫn đến hiện tượng xuất hiện nhiễu điện áp điều khiển, gây ra sự rung lắc cơ khí dữ dội và làm nóng động cơ.

Xét thành phần bù sai lệch trước – Feedforward: Với mục đích nâng cao chất lượng bám quỹ đạo mà không cần tăng quá cao hệ số khâu tích phân trong bộ điều khiển PID, nhóm tích hợp thêm thành phần Feedforward. Thành phần này bù trực tiếp lực cản sinh ra do ma sát nhớt của động cơ, giúp hệ thống phản ứng ngay lập tức với sự thay đổi của vận tốc điều khiển và đảm bảo chuyển động quay ổn định hơn.

Từ những lập luận trên, nhóm đã xem xét và sử dụng bộ điều khiển bậc hai PI có bù sai lệch trước nhằm triệt tiêu ma sát nhớt. Cụ thể mô hình điều khiển miền thời gian có dạng như sau:

$$\begin{aligned} u(t) &= u_{ff}(t) + u_{PI}(t) \\ \rightarrow u(t) &= B\omega_{target}(t) + K_p e(t) + K_i \int_0^t e(t)dt \end{aligned} \quad (2.5)$$

Trong đó:

K_i : là hệ số sai số tĩnh trong mô hình điều khiển PI

K_p : là hệ số tỉ lệ trong mô hình điều khiển PI

Thực hiện biến đổi Laplace, bộ điều khiển có dạng:

$$U(s) = B\Omega_{target}(s) + \left(K_2 + \frac{K_1}{s}\right)E(s) \quad (2.6)$$

Thế (2.6) vào phương trình động cơ vòng hở $\Omega(s) = G_p(s)U(s)$:

$$\begin{aligned} \Omega(s) &= \frac{1}{Js + B} \left[B\Omega_{target}(s) + \frac{K_2s + K_1}{s} (\Omega_{target}(s) - \Omega(s)) \right] \\ \Leftrightarrow (Js + B)\Omega(s) &= B\Omega_{target}(s) + \frac{K_2s + K_1}{s} \Omega_{target}(s) - \frac{K_2s + K_1}{s} \Omega(s) \\ \Leftrightarrow \left[(Js + B) + \frac{K_2s + K_1}{s} \right] \Omega(s) &= \left[B + \frac{K_2s + K_1}{s} \right] \Omega_{target}(s) \\ \Leftrightarrow \left[\frac{Js^2 + Bs + K_2s + K_1}{s} \right] \Omega(s) &= \left[\frac{Bs + K_2s + K_1}{s} \right] \Omega_{target}(s) \\ \rightarrow G_d(s) = \frac{\Omega(s)}{\Omega_{target}(s)} &= \frac{(B + K_2)s + K_1}{Js^2 + (B + K_2)s + K_1} \end{aligned} \quad (2.7)$$

2.1.3. Phương pháp gán cực – Pole placement

Để tìm hệ số K_i, K_p phù hợp để hệ thống không vọt lố và có tốc độ đáp ứng giá trị mong muốn, ta cần thực hiện vài phép toán trên phương trình (2.7).

Gán phương trình ở mẫu số bằng 0:

$$\Delta(s) = Js^2 + (B + K_p)s + K_i = 0 \quad (2.8)$$

Đồng nhất với phương trình đặc trưng tiêu chuẩn của bậc hai $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$ [8] ta được:

$$\begin{cases} K_i = J\omega_n^2 \\ B + K_p = 2J\zeta\omega_n \end{cases} \rightarrow \begin{cases} K_i = J\omega_n^2 \\ K_p = 2J\zeta\omega_n - B \end{cases} \quad (2.9)$$

Trong đó:

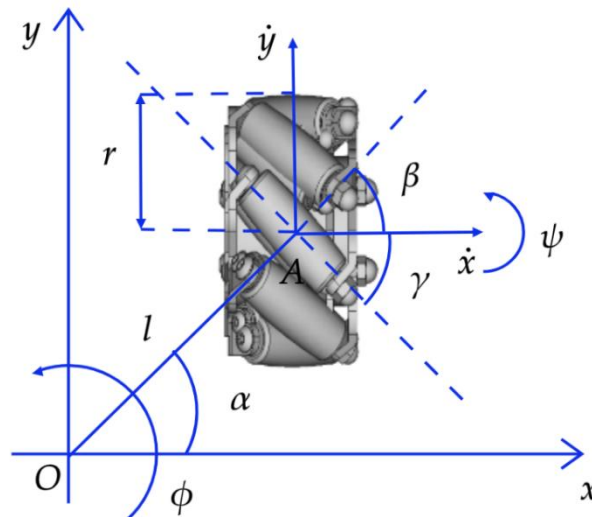
ζ : là tần số tắt dần của động cơ, đại diện cho độ vọt lố của hệ thống

ω_n : là hệ số tự nhiên của hệ thống vòng kín, quyết định tốc độ đáp ứng của động cơ

2.2. Mô hình toán học robot sử dụng bánh mecanum

2.2.1. Giới thiệu về mô hình toán học của bánh xe mecanum

Bánh xe mecanum là một loại bánh xe đặc biệt với quanh vành bánh xe là các con lăn vệ tính được gắn nghiêng 45° so với trục chính của bánh xe. Khi bánh xe quay do tác động từ lực của động cơ thì các bánh lăn con cũng chuyển động thụ động, làm biến đổi một phần lực di chuyển tịnh tiến thành lực trượt ngang, qua đó làm cho bánh xe mecanum vừa có thể di chuyển đồng thời theo phương tịnh tiến và phương ngang.



Hình 2.1: Mô hình toán học của bánh xe mecanum

Xét mô hình toán học của bánh xe mecanum (Hình 2.1), trong hệ trục tọa độ Oxy với tâm bánh xe là điểm A, ta có:

α là góc lệch giữa đường thẳng OA và trục x

β là góc lệch giữa đường thẳng OA và trục chính của bánh xe

γ là góc lệch giữa trục con lăn và trục chính bánh xe

ψ là góc quay của bánh xe quanh trục chính

ϕ là góc hướng của robot

$[\dot{x} \ \dot{y}]^T$ là vận tốc tức thời của bánh xe

$\dot{\psi}$ là vận tốc góc của bánh xe

r là bán kính bánh xe

l là khoảng cách giữa gốc tọa độ O đến điểm A

Giả sử độ trượt dọc của bánh xe $p = 0$ thì vận tốc của bánh xe được tính như sau:

Ta có vector hướng lăn của con lăn trong hệ cục bộ là:

$$\vec{t}_{lănB} = \begin{bmatrix} \cos\left(\gamma - \frac{\pi}{2}\right) \\ \sin\left(\gamma - \frac{\pi}{2}\right) \end{bmatrix} = \begin{bmatrix} \sin \gamma \\ -\cos \gamma \end{bmatrix} \quad (2.10)$$

Suy ra vector hướng lăn của con lăn trong hệ toàn cục là:

$$\vec{t}_{lăn} = \begin{bmatrix} \sin(\alpha + \beta + \gamma) \\ -\cos(\alpha + \beta + \gamma) \end{bmatrix} \quad (2.11)$$

Ta lại có vector vận tốc góc của robot là:

$$\vec{v}_{hướng} = \dot{\phi} \vec{l}_{pt} = \dot{\phi} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} l \begin{bmatrix} \cos \alpha \\ \sin \alpha \end{bmatrix} = \dot{\phi} l \begin{bmatrix} -\sin \alpha \\ \cos \alpha \end{bmatrix} \quad (2.12)$$

Trong đó:

$\vec{l}_{pt}$ là vector phát tuyến của $\vec{l}$

Nên vector vận tốc của bánh xe là:

$$\vec{v} = \begin{bmatrix} \dot{x}_R \\ \dot{y}_R \end{bmatrix} + \vec{v}_{hướng} \quad (2.13)$$

Vậy nên vận tốc của bánh xe là:

$$\begin{aligned} v &= r\dot{\psi} \cos \gamma \\ &= \vec{v}_{lăn} \\ &= \dot{x}_R \sin(\alpha + \beta + \gamma) - \dot{y}_R \cos(\alpha + \beta + \gamma) + \dot{\phi} l \begin{bmatrix} -\sin \alpha \\ \cos \alpha \end{bmatrix} \begin{bmatrix} \sin(\alpha + \beta + \gamma) \\ -\cos(\alpha + \beta + \gamma) \end{bmatrix} \\ &= \dot{x}_R \sin(\alpha + \beta + \gamma) - \dot{y}_R \cos(\alpha + \beta + \gamma) - \dot{\phi} l \cos(\beta + \gamma) \\ &= [\sin(\alpha + \beta + \gamma) - \cos(\alpha + \beta + \gamma) - l \cos(\beta + \gamma)] [\dot{x}_R \quad \dot{y}_R \quad \dot{\phi}]^T \quad (2.14) \end{aligned}$$

Hay:

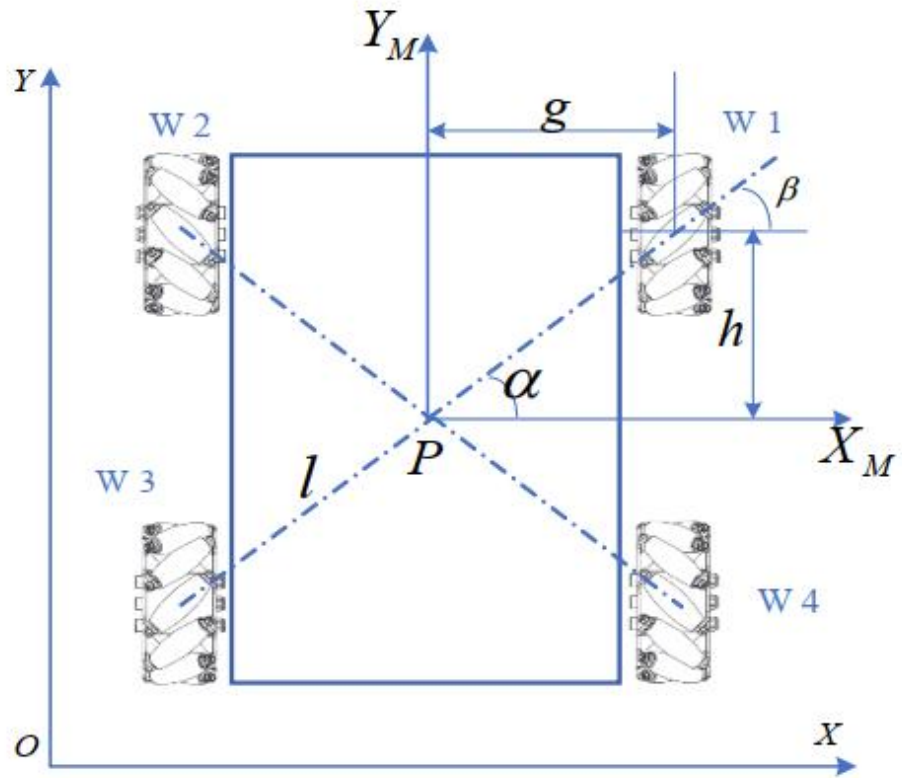
$$r\dot{\psi} \cos \gamma = [\sin(\alpha + \beta + \gamma) - \cos(\alpha + \beta + \gamma) - l \cos(\beta + \gamma)] [\dot{x}_R \quad \dot{y}_R \quad \dot{\phi}]^T \quad (2.15)$$

Như vậy vận tốc góc của bánh xe có thể xác định là:

$$\dot{\psi} = \frac{1}{r} \cos \gamma [\sin(\alpha + \beta + \gamma) - \cos(\alpha + \beta + \gamma) - l \cos(\beta + \gamma)] [\dot{x}_R \quad \dot{y}_R \quad \dot{\phi}]^T \quad (2.16)$$

2.2.2. Phương trình động học robot sử dụng bánh mecanum

Mô hình robot sử dụng bánh mecanum được thiết kế với bốn bánh mecanum được sắp xếp đối xứng theo từng cặp (Hình 2.2), mỗi bánh được điều khiển độc lập bởi một động cơ. Từ đó cho phép robot có thể di chuyển theo những quỹ đạo thẳng, cong mà không cần đổi góc hướng.



Hình 2.2: Mô hình robot bốn bánh xe mecanum [9]

Giả sử với robot được thiết kế với các bánh xe có cùng kích thước: $r_i = r$. Khoảng cách giữa các bánh xe $i = 1, 2, 3, 4$ đến trọng tâm P của robot là đối xứng nhau với chiều dài là $l_i = l$ cùng với $\alpha = \tan^{-1}\left(\frac{h}{g}\right)$ và các thông số α_i , β_i và γ_i của từng bánh xe được thể hiện ở trong Bảng 2.1.

Bảng 2.1: Thông số của các bánh xe mecanum

i	α_i	β_i	γ_i
1	$\tan^{-1}\left(\frac{h}{g}\right)$	$-\tan^{-1}\left(\frac{h}{g}\right)$	45°
2	$\pi - \tan^{-1}\left(\frac{h}{g}\right)$	$\tan^{-1}\left(\frac{h}{g}\right)$	-45°

3	$\pi + \tan^{-1}\left(\frac{h}{g}\right)$	$-\tan^{-1}\left(\frac{h}{g}\right)$	45°
4	$2\pi - \tan^{-1}\left(\frac{h}{g}\right)$	$\tan^{-1}\left(\frac{h}{g}\right)$	-45°

Ta xác định được mô hình động học nghịch như sau:

$$\begin{bmatrix} \dot{\psi}_1 \\ \dot{\psi}_2 \\ \dot{\psi}_3 \\ \dot{\psi}_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} -1 & -1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \\ -1 & 1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \\ 1 & 1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \\ 1 & -1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \end{bmatrix} \begin{bmatrix} \cos\phi & \sin\phi & 0 \\ -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} \quad (2.17)$$

Từ đó ma trận Jacobian được xác định:

$$J = \frac{1}{r} \begin{bmatrix} -1 & -1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \\ -1 & 1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \\ 1 & 1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \\ 1 & -1 & -\sqrt{2}l\sin\left(\frac{\pi}{4} + \alpha\right) \end{bmatrix} \begin{bmatrix} \cos\phi & \sin\phi & 0 \\ -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.18)$$

Ngược lại, mô hình động học thuận được xác định như sau:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = rJ^+ \begin{bmatrix} \dot{\psi}_1 \\ \dot{\psi}_2 \\ \dot{\psi}_3 \\ \dot{\psi}_4 \end{bmatrix} \quad (2.19)$$

Trong đó:

$$J^+ = (J^T J)^{-1} J^T \text{ là ma trận giả nghịch đảo của } J$$

Hay:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = \frac{r}{4} \begin{bmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & -1 \\ k & k & k & k \end{bmatrix} \begin{bmatrix} \psi_1 \\ \psi_2 \\ \psi_3 \\ \psi_4 \end{bmatrix} \quad (2.20)$$

$$\text{Với: } k = \frac{1}{-\sqrt{2}l \sin(\frac{\pi}{4} + \alpha)}$$

2.3. Bộ lọc Kalman mở rộng

2.3.1. Tổng quan về bộ lọc Kalman mở rộng

Bộ lọc Kalman mở rộng hay Extended Kalman Filter - EKF là một biến thể của bộ lọc Kalman được tạo ra dành cho việc xử lý dữ liệu trên các hệ thống phi tuyến. Về cách thực thi, EKF thực hiện tuyến tính hóa dữ liệu trạng thái hệ thống và dữ liệu từ phép đo, sau đó thực hiện tính toán như bộ lọc Kalman truyền thống. Qua đó có thể tách thuật toán EKF thành ba quá trình sau:

- Dự đoán hay Predict: Sử dụng mô hình toán học của hệ thống để dự đoán về trạng thái tiếp theo của hệ thống và dự đoán giá trị của ma trận hiệp phương sai đại diện cho mối tương quan giữa dữ liệu mô hình và phép đo.
- Cập nhật hay Update: Sử dụng dữ liệu từ phép đo thường có lẫn nhiễu và ma trận hiệp phương sai được dự đoán từ bước trước để tính toán hệ số tỉ lệ kalman gain. Và sử dụng kalman gain để hiệu chỉnh lại giá trị dự đoán trạng thái và ma trận hiệp phương sai.
- Tuyến tính hóa hay Linearization: Sử dụng ma trận Jacobian để tính toán xấp xỉ xu hướng thay đổi của giá trị trạng thái và phép đo, qua đó có thể đưa dữ liệu phi tuyến tính thành gần tuyến tính.

2.3.2. Thuật toán của bộ lọc Kalman mở rộng

Trong thuật toán Kalman thông thường việc tính toán chỉ hoạt động trên hệ thống tuyến tính. Để giải quyết vấn đề này, thuật toán đưa ra điều kiện các hàm chuyển trạng thái và hàm quan sát phải khả vi, từ đó áp dụng triển khai chuỗi Taylor bậc một để tính toán các ma trận Jacobian phục vụ cho quá trình tuyến tính hóa.

Giả sử hệ thống chúng ta xét đến là hệ thống hoàn toàn phi tuyến tính. Gọi f , h lần lượt là hàm trạng thái của hệ thống và hàm quan sát của hệ thống, ta có các phương trình sau:

a) Bước dự đoán

Phương trình dự đoán trạng thái của hệ thống tại thời điểm k :

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_k) \quad (2.21)$$

Trong đó:

$\hat{x}_{k|k-1}$ là trạng thái dự đoán tại thời điểm k

f là hàm chuyển trạng thái của hệ thống

$\hat{x}_{k-1|k-1}$ là trạng thái tối ưu tại thời điểm $k-1$

u_k là giá trị điều khiển tại thời điểm k

Phương trình dự đoán ma trận hiệp phương sai tại thời điểm k :

$$P_{k|k-1} = F_{k-1}P_{k-1|k-1}F_{k-1}^T + W_{k-1}Q_kW_{k-1}^T \quad (2.22)$$

Trong đó:

$P_{k|k-1}$ là ma trận hiệp phương sai dự đoán ở thời điểm k .

F_{k-1} là ma trận Jacobian của hàm chuyển trạng thái f tại trạng thái tối ưu hiện tại $(\hat{x}_{k-1|k-1})$.

$P_{k-1|k-1}$ là ma trận hiệp phương sai của trạng thái tối ưu $\hat{x}_{k-1|k-1}$.

W_{k-1} là ma trận Jacobian của hàm chuyển trạng thái f đối với nhiễu quá trình w_k .

Q_k là ma trận hiệp phương sai của nhiễu quá trình tại thời điểm k .

b) Bước cập nhật

Phương trình tính toán kalman gain:

$$G_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + V_k R_k V_k^T)^{-1} \quad (2.23)$$

Trong đó:

G_k là kalman gain tại thời điểm k

H_k là ma trận Jacobian của hàm quan sát h tại $\hat{x}_{k|k-1}$

R_k là ma trận hiệp phương sai của nhiễu đo lường

V_k là ma trận Jacobian của hàm quan sát h đối với nhiễu đo lường v_k

Phương trình cập nhật lại trạng thái dự đoán:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + G_k (z_k - h(\hat{x}_{k|k-1})) \quad (2.24)$$

Trong đó:

z_k là phép đo tại thời điểm k

h là hàm quan sát của hệ thống

$\hat{x}_{k|k}$ là trạng thái tối ưu tại thời điểm k

Phương trình cập nhật lại ma trận hiệp phương sai:

$$P_{k|k} = (I - G_k H_k) P_{k|k-1} \quad (2.25)$$

Trong đó:

$P_{k|k}$ là ma trận hiệp phương sai tại thời điểm k

I là ma trận đơn vị

2.3.3. Nhận xét

Thuật toán EKF có thể cho ta thấy một vài đặc điểm sau:

- EKF có thể giải quyết hiệu quả bài toán ước lượng trạng thái trong môi trường có nhiễu quá trình trong hệ thống và nhiễu đo lường từ cảm biến bằng cách tối ưu hóa ma trận hiệp phương sai.

- Hơn nữa, Bằng cách sử dụng phương pháp tuyến tính hóa dựa trên ma trận Jacobian, thuật toán giảm thiểu được khối lượng tính toán, giúp dễ dàng triển khai trên các hệ thống nhúng.
- Tuy nhiên, do thuật toán EKF chỉ triển khai chuỗi Taylor đến bậc một, nên mô hình thu được chỉ là kết quả xấp xỉ tuyến tính tại điểm đạo hàm. Qua đó, làm giảm độ chính xác của bộ lọc nếu hệ thống thay đổi quá nhanh.
- Không chỉ thế, kết quả ước lượng của EKF phụ thuộc rất lớn vào giá trị khởi tạo ban đầu của ma trận hiệp phương sai P_0 , Q , R và trạng thái khởi tạo của hệ thống. Nếu trạng thái ban đầu bị đóng sai lệch quá nhiều so với thực tế, ma trận Jacobian tính toán tại điểm đó không còn chính xác, đồng thời có thể dẫn đến sai số dần tiến tới vô cực hay còn gọi là hiện tượng phân kỳ kết quả.

2.4. Hệ thống thị giác ngữ nghĩa trên thiết bị biên

2.4.1. Phát hiện vật thể với YOLOv8

Bài toán phát hiện vật thể (object detection) yêu cầu mô hình vừa phân loại đối tượng (classification) vừa xác định vị trí qua khung bao (bounding box). Họ YOLO (You Only Look Once), đề xuất bởi Redmon và cộng sự năm 2015, biến phát hiện vật thể thành bài toán hồi quy (regression) trên lưới đặc trưng (feature grid), cho phép một lần chạy (single-pass) là có kết quả — khác với hai bước của họ R-CNN.

Phiên bản YOLOv8 (Ultralytics, 2023) loại bỏ cơ chế anchor cũ, sử dụng đầu phát hiện tách biệt (decoupled head) gồm hai nhánh phân loại và hồi quy hộp. Tổng hàm mất mát (loss) gồm ba thành phần: Binary Cross-Entropy cho phân loại, Complete IoU (CIoU) cho hồi quy hộp và Distribution Focal Loss (DFL) cho phân phối tọa độ.

$$L_{total} = \lambda_{cls}L_{BCE} + \lambda_{box}L_{CIoU} + \lambda_{dfl}L_{DFL} \quad (2.26)$$

Trong (2.26), các tham số λ là trọng số cân bằng giữa ba hàm mất mát thành phần. YOLOv8 cung cấp năm biến thể với kích thước tăng dần: Nano (n), Small (s), Medium (m), Large (l) và Extra-large (x). Khóa luận lựa chọn biến thể Nano vì chỉ

3,2 triệu tham số và 8,7 GFLOPs cho mỗi lần suy luận — phù hợp ngân sách tính toán của Jetson Xavier khi đồng thời chạy VLM.

2.4.2. Mô hình ngôn ngữ - thị giác kiến trúc Transformer

Kiến trúc Transformer của Vaswani và cộng sự (2017) là nền tảng cho hầu hết các mô hình ngôn ngữ và VLM hiện đại. Cơ chế cốt lõi là Self-Attention, cho phép mỗi token “chú ý” đến mọi token khác trong chuỗi để tính biểu diễn của mình. Công thức Scaled Dot-Product Attention:

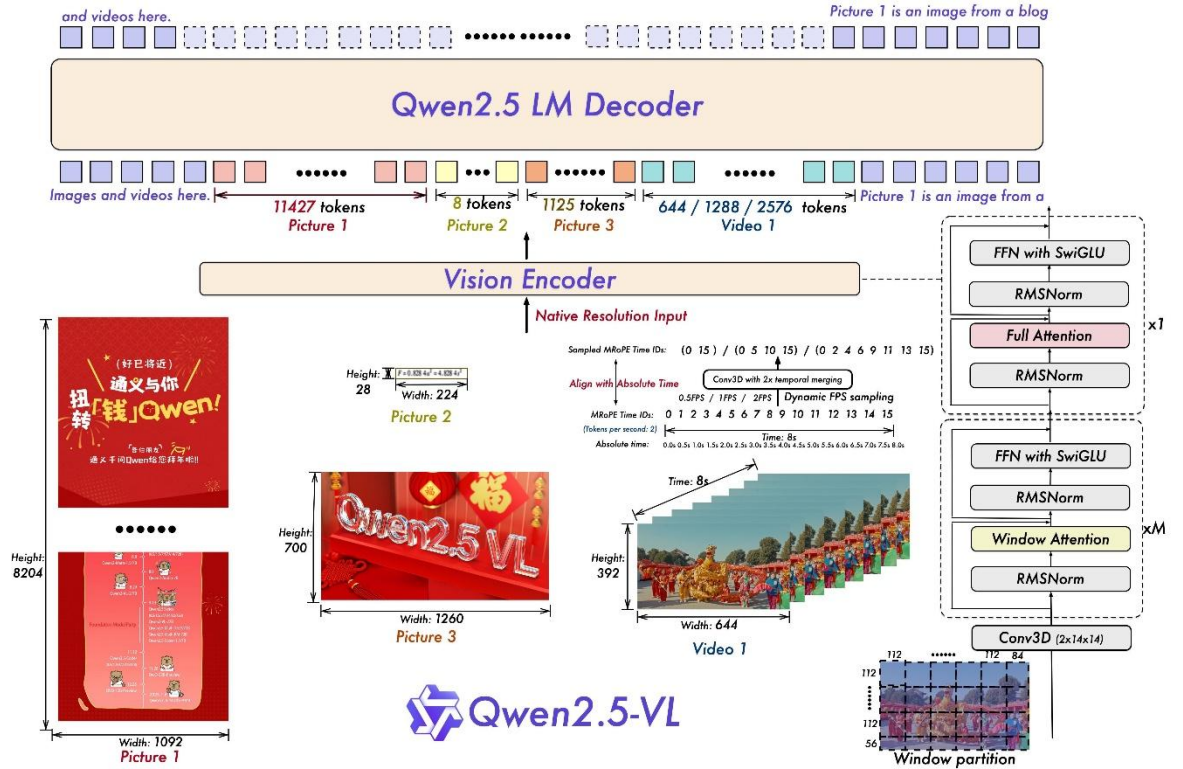
$$Attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V \quad (2.27)$$

Multi-Head Attention chạy nhiều phép chú ý song song với các chiều tuyến tính khác nhau, sau đó nối kết quả lại. Encoder dùng Self-Attention hai chiều; Decoder của LLM dùng Causal-Masked Self-Attention để đảm bảo tính sinh ngữ tuần tự (autoregressive).

2.4.3. Vision Transformer (ViT)

Vision Transformer của Dosovitskiy và cộng sự (2020) áp dụng Transformer cho ảnh bằng cách chia ảnh thành các mảnh (patch) không chồng lấp (thông thường 16×16 pixel), nhúng tuyến tính mỗi mảnh thành một vector embedding, gắn thêm token [CLS] và mã vị trí, rồi đưa qua chuỗi các khối Transformer Encoder.

2.4.4. Kiến trúc mô hình VLM Qwen2.5-VL-3B



Hình 2.3: Kiến trúc mô hình Qwen2.5-VL-3B [10]

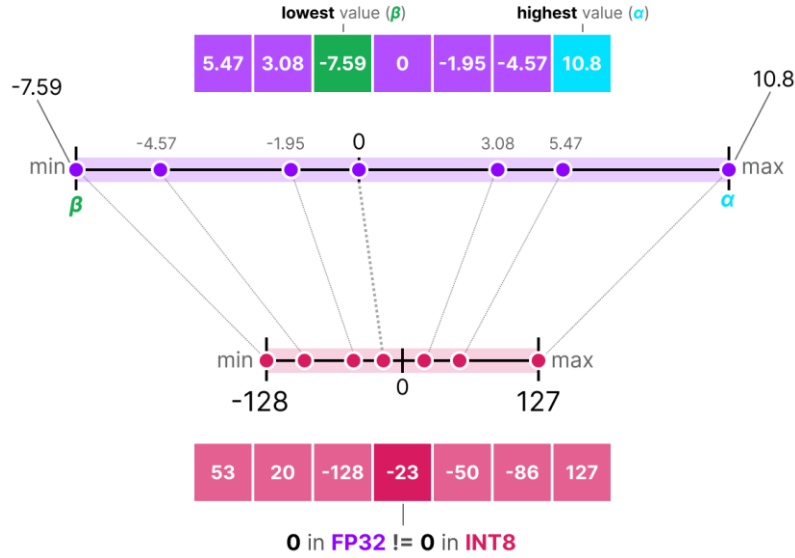
Qwen2.5-VL-3B-Instruct gồm ba khối chính. Thứ nhất là Vision Encoder dạng ViT có khả năng xử lý ảnh ở độ phân giải động (dynamic resolution), tránh việc resize cứng nhắc. Thứ hai là Multimodal Projector (mmpoj) — một mạng tuyến tính nhỏ ánh xạ vector ảnh từ không gian embedding của ViT sang không gian embedding của LLM. Thứ ba là LLM Backbone, kiến trúc Decoder-only với 3 tỷ tham số, sử dụng cơ chế M-RoPE (Multimodal Rotary Position Embedding) để mã hóa cùng lúc vị trí không gian (cho token ảnh) và vị trí thời gian (cho token văn bản).

2.4.5. Lượng tử hóa mô hình

Lượng tử hóa (quantization) là quá trình giảm độ chính xác biểu diễn của trọng số (weights) và đôi khi cả kích hoạt (activations), thường từ floating point 32-bit (FP32) hoặc 16-bit (FP16) xuống integer 8-bit (INT8) hoặc 4-bit (INT4). Mục tiêu là giảm dung lượng bộ nhớ và tăng tốc độ tính toán mà chấp nhận một mức suy giảm chất lượng nhất định.

$$x_q = \text{round}\left(\frac{x_{fp}}{s}\right) + z; x_{fp} \approx s \cdot (x_q - z) \quad (2.28)$$

Trong (2.28), s là scale, z là zero-point. Quantization đối xứng (symmetric) đặt $z = 0$, đơn giản hơn nhưng giảm độ chính xác khi phân bố lệch về một phía. Quantization bất đối xứng (asymmetric) cho phép $z \neq 0$, bám sát phân bố thực tế hơn.



Hình 2.4: Lượng tử hóa affine bất đối xứng từ FP32 sang INT8 [11]

Dải giá trị thực $[\beta, \alpha]$ (ở đây $[-7.59; 10.8]$) được ánh xạ tuyến tính về dải số nguyên INT8 $[-128; 127]$ theo công thức (3.3) với scale s và zero-point z . Do $z \neq 0$, giá trị 0 trong FP32 không trùng với 0 trong INT8 — đặc trưng của lượng tử hóa bất đối xứng giúp bám sát phân bố trọng số lệch về một phía.

2.4.6. Định dạng GGUF và biến thể Q4_K_M

GGUF (GPT-Generated Unified Format) là định dạng nhị phân được dự án llama.cpp đề xuất, kế thừa và cải tiến từ GGML. Một file GGUF gồm: header (magic bytes, version), bảng metadata (key-value lưu thông số kiến trúc, tokenizer, ...) và phần dữ liệu tensor được lượng tử hóa. GGUF cho phép mmap (memory map) trực tiếp file vào không gian địa chỉ tiến trình, giúp tải mô hình rất nhanh và không bắt buộc nạp toàn bộ trọng số vào RAM.

Họ K-quant của llama.cpp gồm nhiều biến thể: Q2_K, Q3_K_S/M/L, Q4_K_S/M, Q5_K_S/M, Q6_K, Q8_0. Mức Q4_K_M (Q4 K-quant Medium) sử dụng kích thước nhóm 32 trọng số và một tỉ lệ siêu khối (super-block scale), giúp giảm sai số quantization so với Q4_0 đơn giản. Bảng 2.2 so sánh kích thước và chất lượng các biến thể trên Qwen2.5-VL-3B.

Bảng 2.2: Trade-off của các mức lượng tử hóa GGUF trên Qwen2.5-VL-3B.

Mức	Bit / trọng số	Kích thước	Perplexity (so với FP16)	Khuyến nghị
FP16	16	≈ 6,0 GB	0 % (chuẩn)	Chỉ dùng nếu RAM dư
Q8_0	8,5	≈ 3,2 GB	+ ~0,5 %	Chất lượng cao nhất 8-bit
Q5_K_M	5,7	≈ 2,4 GB	+ ~1–2 %	Cân bằng
Q4_K_M	4,8	≈ 2,1 GB	+ ~3–5 %	Lựa chọn của khóa luận
Q4_K_S	4,5	≈ 2,0 GB	+ ~4–6 %	Nhẹ hơn chút ít

Q4_K_M là lựa chọn tối ưu: dung lượng khoảng 2,1 GB phù hợp với 16 GB bộ nhớ thống nhất của Jetson AGX Xavier (đã có khoảng 2 GB hệ điều hành L4T, 500 MB cho YOLO, 500–1000 MB cho KV cache và các tiến trình khác), trong khi mức suy giảm perplexity khoảng 3–5% so với FP16 vẫn nằm trong ngưỡng chấp nhận được với ứng dụng phân loại có cấu trúc.

2.4.7. Engine suy luận llama.cpp

llama.cpp do Georgi Gerganov khởi xướng năm 2023 là engine suy luận LLM viết bằng C/C++ thuần, không phụ thuộc Python runtime. Đặc điểm này khiến llama.cpp

đặc biệt phù hợp triển khai trên thiết bị nhúng vì có thể chạy độc lập, dễ đóng gói thành dịch vụ và tiêu thụ ít bộ nhớ phụ trợ hơn so với các framework dựa trên Python.

2.4.8. Tối ưu CPU và GPU

Trên CPU, llama.cpp sử dụng SIMD: tập lệnh AVX2/AVX-512 trên x86 và tập lệnh NEON trên ARM. Đối với Jetson AGX Xavier (ARMv8.2 với NEON), llama.cpp tận dụng được trực tiếp các phép nhân vector. Trên GPU, llama.cpp có backend CUDA cho phép “offload” một phần lớp Transformer lên GPU bằng tham số `--n-gpu-layers N`.

Khi $N = 0$, toàn bộ tính toán diễn ra trên CPU. Khi $N =$ số lớp của mô hình (hoặc -1), toàn bộ lớp được offload, GPU đảm nhiệm cả forward pass và lưu trữ KV cache. Trạng thái lai (hybrid) có thể tối ưu cho mô hình lớn vượt VRAM, nhưng phải trả giá bằng việc đồng bộ giữa CPU và GPU mỗi lớp.

2.4.9. KV cache và độ dài ngữ cảnh

Trong suy luận autoregressive, mỗi token mới sinh ra phải tính lại attention với toàn bộ token trước đó. Để tránh tính lặp, các tensor Key (K) và Value (V) của các lớp được lưu lại trong KV cache. Kích thước KV cache xấp xỉ:

$$S_{KV} \approx 2 \times ctx \times n_{layers} \times hidden_dim \times dtype_{size} \quad (2.29)$$

Với Qwen2.5-VL-3B ($n_{layers} \approx 36$, $hidden_dim \approx 2048$, $dtype = FP16$) và ngữ cảnh 1024 token, KV cache khoảng 290 MB. Khi tăng ctx lên 4096, KV cache lên xấp xỉ 1,16 GB — đáng kể trong tổng ngân sách 16 GB.

2.4.10. API tương thích OpenAI và hỗ trợ đa phương thức

Lệnh llama-server cung cấp HTTP endpoint `/v1/chat/completions` tương thích với chuẩn OpenAI, nhận messages dạng JSON gồm các phần text và image_url (base64). Tham số mmproj là đường dẫn đến file projector thị giác (mmproj-<model>.gguf), được load riêng để biến token ảnh thành embedding tương thích với LLM backbone.

2.5. Nền tảng ROS2

2.5.1. Giới thiệu tổng quan

Robot Operating System - ROS là một framework mã nguồn mở và nền tảng middleware hàng đầu thế giới dành cho phát triển ứng dụng robot. Nó cung cấp các thư viện, công cụ và quy ước để điều khiển phần cứng, định tuyến dữ liệu và xây dựng các thuật toán robot phức tạp.

ROS được sử dụng rộng rãi trong nhiều lĩnh vực có sự góp mặt của robot. Trong logistics, nó giúp robot di chuyển linh hoạt trong kho hàng thông qua các phương thức lập quỹ đạo, định vị, điều khiển chuyển động và phối hợp với nhiều robot khác. Trong nhà máy sản xuất, ROS hỗ trợ robot đảm nhận các tác vụ nâng cao như gấp và đặt vật thể với độ chính xác tuyệt đối. Trong y tế, nền tảng này cũng góp phần vận hành các hệ thống robot hỗ trợ chăm sóc bệnh nhân, nâng cao hiệu quả trong quy trình làm việc lâm sàng.

Điểm mạnh lớn nhất của ROS nằm ở việc thay vì phải tự lập trình từ con số không cho các tính năng cơ bản, nhà phát triển có thể tận dụng hàng ngàn gói phần mềm mã nguồn mở do cộng đồng toàn cầu đóng góp, trải dài từ các thuật toán xử lý động học, định vị bản đồ, đến nhận diện hình ảnh.



Hình 2.5: Logo ROS [12]

Tuy nhiên, cùng với sự phát triển của robot hiện đại, các yêu cầu về tính thời gian thực, độ tin cậy và khả năng giao tiếp đồng bộ giữa nhiều thiết bị phần cứng ngày càng trở nên khắt khe. Dù ROS thế hệ đầu tiên đã đặt nền móng vững chắc, nó vẫn tồn tại hạn chế do phụ thuộc vào một nút trung tâm duy nhất là ROS Master. Khi nút trung tâm này gặp sự cố, toàn bộ hệ thống sẽ ngừng hoạt động, đây là điểm yếu nghiêm trọng trong các ứng dụng robot thực tế yêu cầu độ tin cậy cao.

Để giải quyết triệt để vấn đề này, thế hệ thứ hai mang tên ROS 2 đã ra đời. Điểm đột phá của ROS 2 là việc sử dụng tiêu chuẩn truyền thông công nghiệp Data Distribution Service - DDS và loại bỏ hoàn toàn kiến trúc tập trung. Điều này giúp hệ thống hoạt động ổn định và bảo mật hơn, đặc biệt tối ưu khi cần phân tán khối lượng xử lý hoặc kết nối các bộ xử lý trung tâm cấp cao với hệ thống vi điều khiển cấp thấp.

Chính vì những ưu điểm vượt trội về hiệu năng truyền tải, khả năng tích hợp đa nền tảng và đáp ứng tốt các ràng buộc về thời gian thực, ROS 2 đã được lựa chọn làm nền tảng phần mềm cốt lõi để xây dựng và phát triển hệ thống điều khiển trong đề tài này. Cụ thể, phiên bản ROS 2 Foxy Fitzroy được sử dụng do tính tương thích với hệ điều hành Ubuntu 20.04 L4T trên Jetson AGX Xavier và sự hỗ trợ ổn định của các gói điều hướng Nav2 cũng như slam_toolbox tại thời điểm phát triển.

2.5.2. Cấu trúc và cơ chế hoạt động

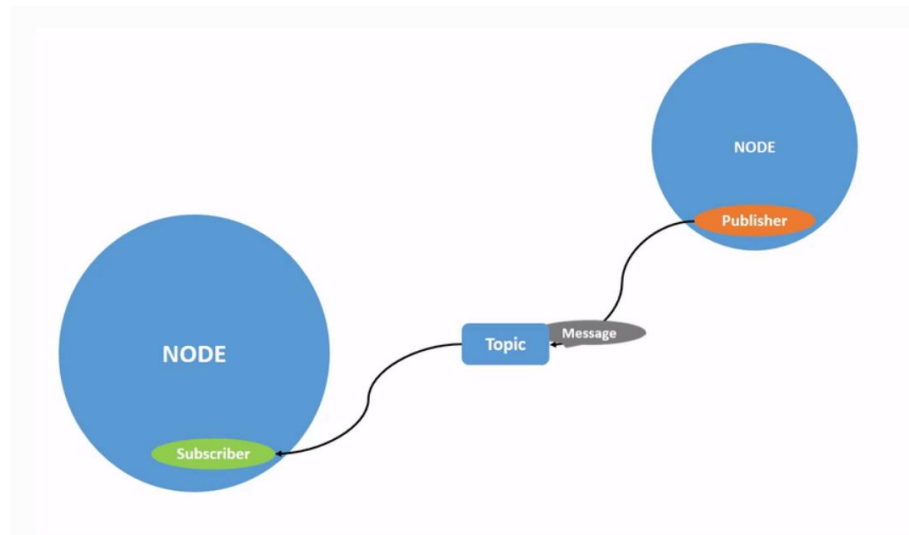
Điểm khác biệt cốt lõi trong kiến trúc của ROS 2 so với thế hệ tiền nhiệm là việc loại bỏ hoàn toàn nút quản lý trung tâm. Thay vì mọi thành phần phải báo cáo và đăng ký thông qua một máy chủ duy nhất, ROS 2 sử dụng cơ chế tự động khám phá dựa trên nền tảng DDS. Cấu trúc mạng ngang hàng này giúp hệ thống loại bỏ hiện tượng thắt nút cổ chai và tránh tình trạng sụp đổ toàn bộ hệ thống khi một điểm duy nhất gặp sự cố.

Cơ chế hoạt động của ROS 2 được xây dựng dựa trên các khối thành phần cơ bản sau:

Nodes: Đây là các tiến trình thực thi tác vụ độc lập. Trong một hệ thống robot hiện đại, các nodes thường được phân bổ theo cấp độ phân cứng. Ví dụ: một node đảm nhận các tác vụ tính toán nặng như xử lý AI, phân tích hình ảnh hoặc định vị sẽ chạy trên máy tính nhúng trung tâm. Ngược lại, các nodes xử lý tín hiệu cảm biến, điều khiển động cơ bước hay vòng lặp PID sẽ được giao cho các bộ vi điều khiển cấp thấp.

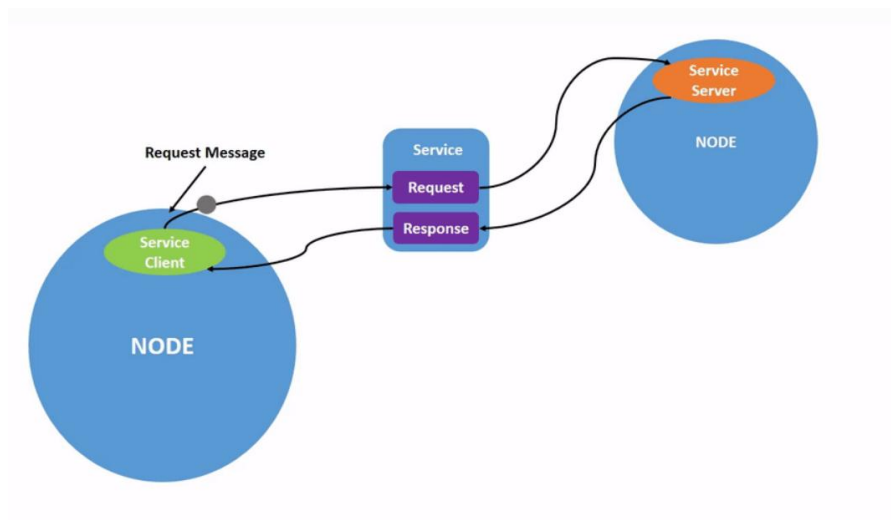
Giao thức giao tiếp: Để các nodes trao đổi dữ liệu liền mạch, ROS 2 định nghĩa ba mô hình giao tiếp tiêu chuẩn:

- Topics: Dành cho luồng dữ liệu truyền liên tục và một chiều. Mô hình này sử dụng cơ chế Publish để gửi dữ liệu và Subscribe để nhận dữ liệu. Ví dụ: một node liên tục publish dữ liệu tọa độ từ encoder, và node điều khiển subscribe để nhận dữ liệu đó. Ngoài ra, một node có thể Publish dữ liệu cho nhiều topics và cũng có thể Subscribe nhiều topics để lấy dữ liệu.



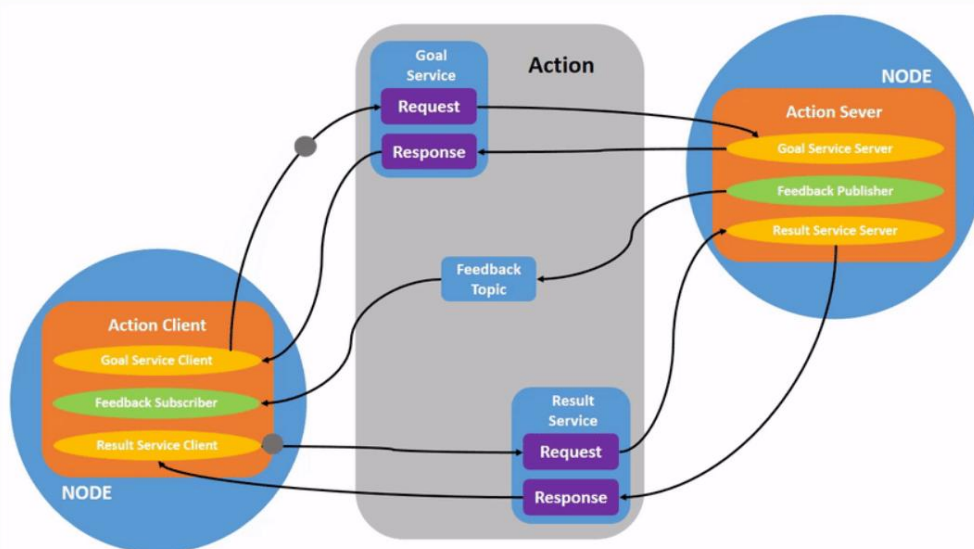
Hình 2.6: Sơ đồ giao tiếp qua topics [12]

- Services: Dành cho các yêu cầu gọi/đáp đồng bộ và diễn ra nhanh chóng, chẳng hạn như gửi lệnh kích hoạt hoặc reset một cụm cảm biến. Mô hình này sử dụng cơ chế request và response giống như giao tiếp http, do đó dữ liệu chỉ được cung cấp khi có yêu cầu từ client. Trong mô hình này, một service có thể có nhiều client nhưng chỉ có thể có một server.



Hình 2.7: Sơ đồ giao tiếp qua services [12]

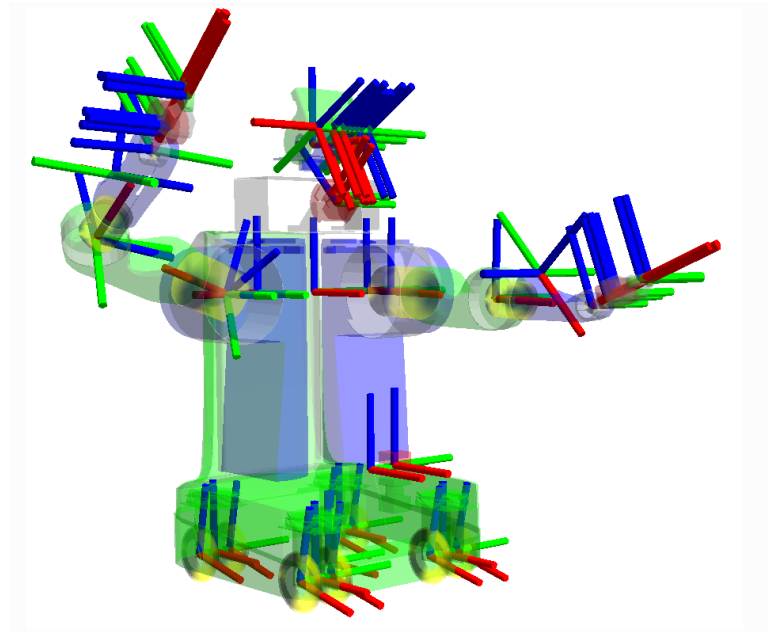
- **Actions:** Phù hợp với các tác vụ kéo dài và cần cập nhật tiến độ liên tục. Mô hình này được xây dựng trên hai mô hình services và topics với cấu trúc gồm ba thành phần là: mục tiêu, phản hồi và kết quả. Khi một mục tiêu được gửi đến action server qua cơ chế services, mục tiêu đó sẽ được action server xem xét có thực thi mục tiêu đó không và gửi response qua service mà client gửi. Nếu mục tiêu được chấp nhận thì client sẽ request kết quả từ services khác và server sẽ liên tục gửi phản hồi về cho client qua cơ chế topics. Sau cùng, server sẽ gửi kết quả qua services mà client request. Ví dụ: ra lệnh cho hệ thống bánh xe Mecanum hoặc cánh tay robot di chuyển đến một tọa độ phức tạp, node thực thi sẽ liên tục phản hồi phản trăm quãng đường đã hoàn thành.



Hình 2.8: Sơ đồ giao tiếp qua actions [13]

Parameters: Đóng vai trò như các biến cấu hình cho hệ thống, cho phép thay đổi các thông số vật lý hoặc thuật toán của robot ngay tại thời gian chạy mà không cần biên dịch lại mã nguồn. Khác với ROS 1 vốn lưu trữ tập trung tất cả tham số trên một máy chủ, ROS 2 quản lý tham số theo kiến trúc Node-local cho phép mỗi node tự sở hữu và quản lý máy chủ tham số nội bộ của riêng nó.

tf2 – Transforms2: là thành phần giúp theo dõi và quản lý mối quan hệ không gian giữa tất cả các khung hệ tọa độ hay còn gọi là frame theo thời gian thực. Bất kỳ một hệ thống robot hoàn chỉnh nào cũng được cấu thành từ nhiều bộ phận: khung gầm, các khớp nối, cảm biến, camera,... và mỗi thành phần đó đều có frame riêng. tf2 duy trì một cấu trúc dữ liệu dạng cây cho phép hệ thống tự động nội suy và trả về vị trí tương đối giữa bất kỳ hai frame nào trong không gian tại một mốc thời gian cụ thể.



Hình 2.9: Các khung tọa độ trên robot [13]

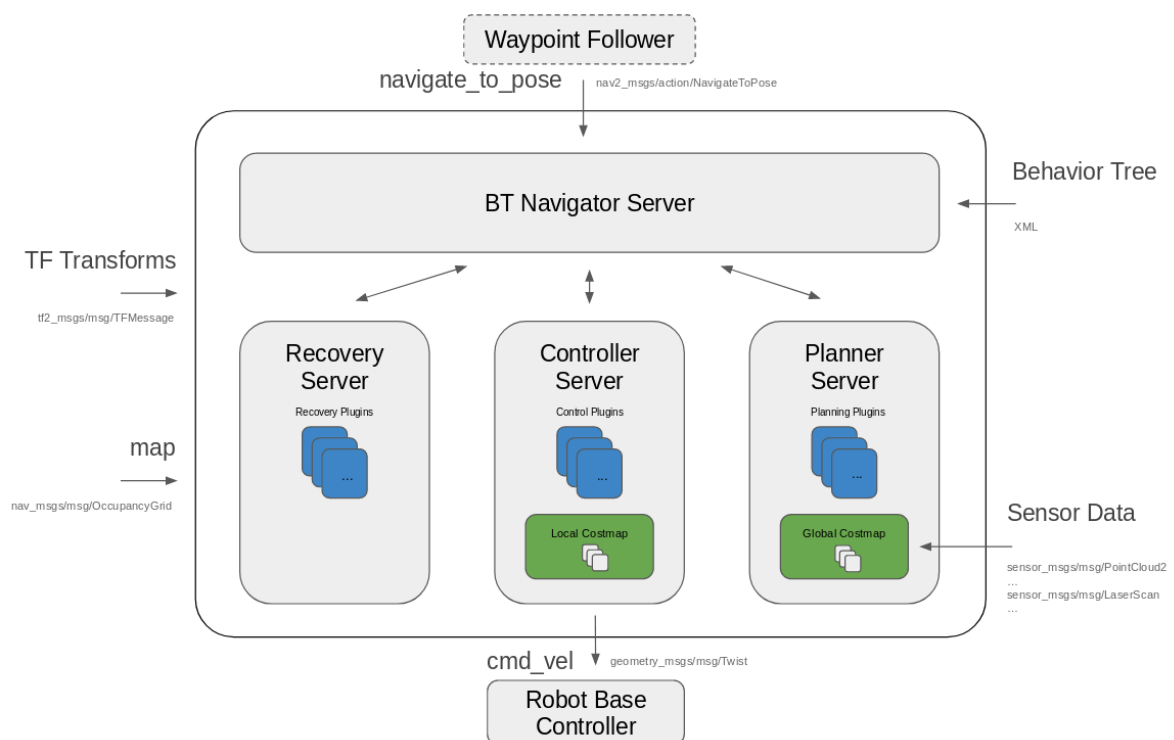
2.6. Hệ thống điều hướng Navigation2 – Nav2

Trong đề tài này, bài toán liên quan tới việc di chuyển từ điểm A sang điểm B cho robot sử dụng để mecanum là một vấn đề quan trọng cần giải quyết. Do sự trượt khi di chuyển của bánh xe mecanum, nên các giải pháp tinh chỉnh vận tốc, quỹ đạo trên bộ điều khiển cục bộ của ESP32 S3 với thông tin đến từ EKF là một hướng tiếp cận khá rủi ro. Do đó, nhóm đi theo một hướng tiếp cận khác đó là sử dụng Nav2 cùng với thuật toán SLAM, với thông tin bản đồ mà SLAM quét được cũng như là tận dụng thông tin từ bộ điều khiển cục bộ và các cảm biến thì quỹ đạo sinh ra từ Nav2 sẽ cho ra kết quả an toàn hơn trong quá trình di chuyển của robot.

Nav2 là một hệ thống điều hướng tiêu chuẩn của hệ sinh thái ROS 2, cho phép robot di chuyển an toàn từ vị trí hiện tại đến một mục tiêu chỉ định trong môi trường có chướng ngại vật.

Điểm đột phá kiến trúc của Nav2 nằm ở việc ứng dụng Behavior-Tree – BT làm trung tâm điều phối. Khác với kiến trúc máy trạng thái hữu hạn FSM vốn dễ trở nên cồng kềnh và khó kiểm soát khi hệ thống mở rộng, BT là một cấu trúc phân cấp cho phép chuyển đổi linh hoạt qua lại giữa các tác vụ của một tác nhân tự động như robot hay mô hình AI. Cấu trúc này mang lại giải pháp tối ưu để xây dựng các hệ thống điều khiển phức tạp, đảm bảo tính mô-đun hóa cao và khả năng phản ứng nhạy bén với những thay đổi bất ngờ từ môi trường.

Về cơ chế hoạt động, Nav2 sử dụng BT như một bộ não xử lý trung tâm để tạo ra các luồng chuyển động thông minh. BT không trực tiếp xử lý thuật toán mà sẽ điều phối các máy chủ độc lập thông qua cơ chế giao tiếp tiêu chuẩn của ROS 2. Mỗi máy chủ này đảm nhận một nhiệm vụ chuyên biệt, từ việc tính toán đường đi, xuất lệnh điều khiển vận tốc, cho đến thực thi các hành vi phục hồi/giải cứu khi robot gặp vật cản khẩn cấp.



Hình 2.10: Sơ đồ cấu trúc của Nav2 [14]

Planner Server: Đảm nhận nhiệm vụ tính toán đường đi toàn cục. Khi nhận được tọa độ mục tiêu từ BT, Planner Server sẽ trích xuất dữ liệu từ Global Costmap và áp dụng các thuật toán tìm kiếm trên đồ thị như A* hoặc Dijkstra. Kết quả trả về là một quỹ đạo di chuyển tối ưu và ngắn nhất, đảm bảo robot có thể đi từ vị trí hiện tại đến đích mà không va chạm với các vật cản tĩnh đã biết trên bản đồ.

Controller Server: Đóng vai trò là bộ điều khiển cục bộ. Máy chủ này nhận quỹ đạo từ Planner Server và có nhiệm vụ tính toán động học để bám sát đường đi đó. Đồng thời, Controller Server liên tục truy xuất Local Costmap theo thời gian thực để phát hiện và né tránh các vật cản động. Thông qua các thuật toán tối ưu hóa như DWA hoặc MPPI, máy chủ này sẽ xuất ra các tập lệnh vận tốc (v_x, v_y, ω). Đây chính là tín hiệu đầu vào để vi điều khiển cấp thấp giải bài toán động học nghịch và điều khiển trực tiếp hệ thống bánh xe.

Recovery Server: Đóng vai trò là cơ chế xử lý sự cố và tự giải cứu của hệ thống. Trong quá trình vận hành, nếu robot rơi vào trạng thái bế tắc ví dụ: Planner Server không thể tìm thấy đường đi hợp lệ, hoặc Controller Server bị kẹt bởi quá nhiều vật cản động. BT sẽ tạm dừng hai máy chủ trên và gọi đến Recovery Server. Máy chủ này sẽ thực thi các hành động khẩn cấp như: xóa dọn dữ liệu nhiễu trên bản đồ, lùi lại một khoảng ngắn, hoặc xoay tại chỗ để cảm biến có thể quét lại môi trường xung quanh. Khi rủi ro được giải trừ, Recovery Server sẽ trả lại quyền kiểm soát để hệ thống tiếp tục hành trình.

Lifecycle Manager: Trong các hệ thống phân tán phức tạp như Nav2, việc kiểm soát thứ tự khởi động, tạm dừng và tắt của các tiến trình là yếu tố quyết định đến độ ổn định của robot. Trong Nav2, các máy chủ chịu sự quản lý của một máy trạng thái nghiêm ngặt gồm 4 giai đoạn chính: Unconfigured, Inactive, Active, Finalized. Lifecycle Manager sẽ tuần tự chuyển đổi trạng thái của các node theo một kịch bản đã được định nghĩa trước. Trình quản lý này đảm bảo hệ thống khởi động đồng bộ và an toàn tuyệt đối. Trong trường hợp có một máy chủ gặp sự cố bị sập, Lifecycle

Manager có khả năng can thiệp, đưa toàn bộ phần còn lại của hệ thống về trạng thái Inactive để cắt chuỗi hành vi mất kiểm soát, đảm bảo an toàn phần cứng trước khi tiến hành khởi động lại chu trình.

2.7. Thuật toán tự động khám phá - Frontier-base exploration

2.7.1. Giới thiệu tổng quan

Trong đề tài này, với mong muốn robot có thể hoạt động kể cả khi được đưa vào một môi trường lạ thì bài toán liên quan tới thành lập bản đồ tại môi trường chưa biết trước tiên là một thách thức lớn. Trong quá trình thành lập bản đồ môi trường, việc chỉ định thủ công từng điểm đến cho Nav2 là không khả thi. Giải pháp cho bài toán này là kết hợp thuật toán định vị và lập bản đồ SLAM với phương pháp khám phá dựa trên ranh giới Frontier-based exploration. Giải pháp này giúp cho robot tự động khám phá mà không cần sự can thiệp của con người.

Trong một bản đồ lưới trạng thái, các ô lưới được chia thành ba loại: không gian tự do hay Free space, vật cản hay Obstacles và vùng chưa biết hay Unknown space. Đường biên hay còn gọi là Frontier được định nghĩa là tập hợp các ô lưới không gian tự do nằm tiếp giáp trực tiếp với các ô lưới chưa biết.

Thuật toán hoạt động theo vòng lặp sau:

- Trích xuất đường biên: Từ bản đồ do SLAM cung cấp, thuật toán quét và nhóm các điểm ranh giới liên tiếp nhau thành các cụm Frontier regions.
- Đánh giá và lựa chọn: Hệ thống không chọn ngẫu nhiên một đường biên để đi tới, mà đánh giá điểm đến tối ưu dựa trên một hàm mục tiêu.
- Điều hướng: Tọa độ của điểm có giá trị cao nhất tính toán từ hàm mục tiêu được gửi trực tiếp đến Nav2. Robot di chuyển đến điểm đó, cảm biến quét không gian mới, thuật toán SLAM cập nhật bản đồ, và vòng lặp tiếp tục cho đến khi không còn điểm Frontier nào được tìm thấy.

2.7.2. Thuật toán Wavefront Frontier Detector – WFD

Wavefront Frontier Detector - WFD [15] là một trong những phương pháp khám phá dựa trên Frontier phổ biến và hiệu quả nhất, đặc biệt tối ưu cho các bản đồ dạng lưới trạng thái 2D. Được đề xuất bởi Topiwala và cộng sự, WFD cải tiến thuật toán Frontier-based gốc của Yamauchi (1997) bằng cách chỉ quét các vùng đã biết thay vì toàn bộ bản đồ, từ đó giảm đáng kể độ phức tạp tính toán.

Một ô lưới c hay cell trên bản đồ được phân loại là một Frontier cell nếu nó thỏa mãn đồng thời hai điều kiện:

- Bản thân ô lưới c phải là Free space.
- Tồn tại ít nhất một ô lưới lân cận trong tập 8 hướng là Unknown space.

Để đảm bảo mỗi ô lưới chỉ được xét một lần và mỗi điểm Frontier chỉ thuộc về một cụm Frontier duy nhất, WFD phân loại mỗi ô theo một trong bốn trạng thái:

- Map-Open-List: ô đã được đưa vào hàng đợi của BFS ngoài.
- Map-Close-List: ô đã được xử lý bởi BFS ngoài.
- Frontier-Open-List: ô đã được đưa vào hàng đợi của BFS trong.
- Frontier-Close-List: ô đã được xử lý bởi BFS trong.

Thuật toán WFD thực thi qua hai giai đoạn BFS lồng nhau:

BFS ngoài hay Map Search: Khởi tạo từ ô lưới tương ứng vị trí hiện tại của robot. Quá trình BFS lan dần ra các ô Free space xung quanh. Điều kiện để một ô lưới v được đưa vào hàng đợi BFS ngoài là: v chưa trong Map-Open-List hoặc Map-Close-List, và v có ít nhất một hàng xóm là Free space, điều kiện này đảm bảo BFS ngoài chỉ đi qua các vùng đã biết, không lạc vào vùng Unknown. Khi BFS ngoài lấy một ô p ra khỏi hàng đợi và phát hiện p là điểm Frontier, BFS trong được kích hoạt.

BFS trong hay Frontier Search: Được kích hoạt tại mỗi điểm Frontier tìm thấy bởi BFS ngoài. BFS trong lan rộng sang các điểm Frontier liền kề để gom toàn bộ thành một cụm Frontier region hoàn chỉnh. Sau khi BFS trong kết thúc, toàn bộ các ô thuộc

cụm này được đánh dấu Map-Close-List, đảm bảo BFS ngoài không đi qua chúng trong các vòng lặp sau.

Tính điểm đại diện của Frontier: Sau khi extract xong một cụm Frontier, cần tính một điểm đại diện để gửi cho Nav2 làm goal. Paper WFD gốc đề xuất dùng median point. Tuy nhiên, nhóm đã sử dụng một hướng tiếp cận khác là tính centroid của cụm, sau đó chọn ô Frontier thực tế gần centroid nhất làm điểm đại diện. Phương pháp này khắc phục hạn chế của centroid là có thể rơi vào vùng Unknown hoặc Obstacle trong khi vẫn đảm bảo điểm goal nằm trong không gian đã biết và là Free space.

Chọn Frontier và điều hướng: Sau khi thu được danh sách tất cả các cụm Frontier, hệ thống lọc bỏ các cụm có kích thước nhỏ hơn ngưỡng tối thiểu để loại noise, sau đó sắp xếp theo khoảng cách Euclidean từ robot tới điểm đại diện của mỗi cụm. Frontier gần nhất chưa bị đánh dấu là không tiếp cận được sẽ được chọn và tọa độ của nó được gửi đến Nav2. Khi Nav2 báo thất bại, điểm Frontier tương ứng được đưa vào blacklist và thuật toán tiếp tục với Frontier gần nhất tiếp theo.

Sau khi robot đến được một Frontier, robot thực hiện quay tròn tại chỗ một vòng để cảm biến có thể quan sát toàn bộ không gian xung quanh điểm mới. SLAM sẽ cập nhật bản đồ từ dữ liệu quét này, có thể mở ra các Frontier mới trước khi vòng lặp WFD tiếp theo được kích hoạt.

Chương 3. PHÂN TÍCH THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG

3.1. Thiết kế hệ thống phần cứng

3.1.1. Tổng quan về các linh kiện

a) Cảm biến BNO055

BNO055 là cảm biến IMU 9 trục là sự kết hợp từ ba cảm biến nhỏ bao gồm: từ kế, gia tốc kế và gyroscope. Nó có khả năng tính toán và cung cấp các thông tin quan trọng bao gồm hướng di chuyển, gia tốc và cả góc hướng của robot một cách chính xác dưới một module duy nhất.

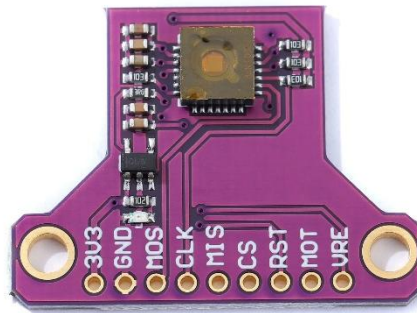


Hình 3.1: Cảm biến BNO055 [16]

Trong phạm vi đề tài nghiên cứu, BNO055 đóng vai trò rất quan trọng trong việc định vị của robot. Cụ thể, nó được sử dụng để cập nhật lại góc hướng tổng thể robot thông qua bộ lọc EKF. Từ đó, cho ra thông tin kèm với quỹ đạo di chuyển của robot được chính xác hơn.

b) Cảm biến PMW3901

Đây là cảm biến optical flow thường được sử dụng cho các mẫu chuột máy tính hoặc drone. Nó sử dụng thuật toán tương quan ảnh số hay còn gọi là digital image correlation, tức so sánh các khung ảnh chụp liên tiếp ở tốc độ lấy mẫu cao bằng cảm biến quang độ phân giải thấp, để cho ra thông tin là giá trị biến thiên theo chiều x hay chiều y. Khi được đặt song song với mặt đất, các giá trị biến thiên thu được từ cảm biến có thể được sử dụng để dự đoán được hướng di chuyển của robot.



Hình 3.2: Cảm biến PMW3901 [17]

Đối với robot mecanum hay cụ thể là bánh xe mecanum việc trượt quỹ đạo khi di chuyển là một điều không thể tránh khỏi do cấu tạo đặc biệt của nó. Do đó, việc tích hợp cảm biến PMW3901 vào hệ thống để cập nhật lại vị trí cho robot mecanum giúp giảm sai số trượt gây ra bởi hiện tượng này

c) Động cơ DC JGB37-545B

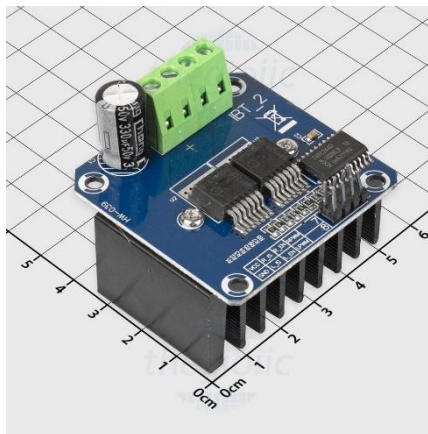
JGB37-545B là động cơ DC giảm tốc sử dụng hộp số bánh răng, được trang bị bộ encoder hai kênh A/B cho phép xác định đồng thời tốc độ quay và chiều quay tức thời của trục động cơ. Phiên bản sử dụng trong đề tài có điện áp định mức 12V, tỉ số truyền hộp số 1:270, cho tốc độ đầu ra danh định 22 RPM, với encoder gắn ở trục động cơ cho độ phân giải 16 xung/vòng quy đổi về số xung tạo ra ở đầu trục động cơ tương ứng là $16 \times 270 = 4320$ xung/vòng. Tỉ số truyền lớn giúp động cơ tạo mô-men xoắn cao ở tốc độ thấp, phù hợp để kéo tải là khung robot cùng các linh kiện mang theo, đồng thời độ phân giải xung quy đổi cao cho phép vòng điều khiển PI kết hợp Feedforward ước lượng tốc độ bánh xe với sai số nhỏ ngay cả ở dải tốc độ thấp nơi các encoder độ phân giải thấp thường gây ra hiện tượng nhiễu lượng tử hóa khi tính vận tốc qua sai phân thời gian.



Hình 3.3: Động cơ JGB37-545B [18]

d) Module điều khiển động cơ DC BTS7960

BTS7960 là module cầu H bán dẫn dùng để điều khiển động cơ DC công suất vừa và lớn, cho phép đảo chiều quay và điều chỉnh tốc độ thông qua tín hiệu PWM. Module tích hợp hai IC BTS7960B, mỗi IC đóng vai trò một nửa cầu H, chịu được dòng liên tục lên đến 43 A và điện áp hoạt động trong khoảng 5,5–27 V, đi kèm cơ chế bảo vệ quá dòng, quá nhiệt và ngắn mạch. Mỗi module nhận hai tín hiệu PWM độc lập tương ứng với chiều quay thuận và nghịch, cùng hai chân cho phép để bật/tắt từng nửa cầu.



Hình 3.4: Module BTS7960 [19]

Trong đề tài, mỗi bánh mecanum được dẫn động bởi một động cơ JGB37-545B điều khiển bởi một module BTS7960 riêng, do đó hệ thống bốn bánh cần bốn module

hoạt động độc lập. Vi điều khiển trung tâm xuất tín hiệu PWM tần số 20 kHz tới từng module, đồng thời đọc tín hiệu hồi tiếp từ encoder hai kênh gắn trên động cơ để xác định tốc độ và chiều quay thực tế, phục vụ vòng điều khiển PI kết hợp Feedforward đã trình bày ở Chương 2. Việc lựa chọn BTS7960 thay vì các module cầu H công suất thấp xuất phát từ yêu cầu dòng tải: khi robot mang tải và di chuyển trên bề mặt có ma sát lớn, dòng khởi động của động cơ JGB37-545B có thể vượt ngưỡng vài ampe trong thời gian ngắn, vượt quá khả năng của các module cầu H dùng linh kiện lưỡng cực thông thường.

e) Vi điều khiển ESP32 S3

ESP32-S3 là vi điều khiển 32-bit kiến trúc Xtensa LX7 lõi kép, xung nhịp tối đa 240 MHz, tích hợp Wi-Fi 802.11 b/g/n và Bluetooth 5.0. So với ESP32 gốc, ESP32-S3 bổ sung tập lệnh tăng tốc ma trận/vector, thêm GPIO, hỗ trợ USB OTG. Trong đề tài, ESP32-S3 được lập trình bằng ESP-IDF và đảm nhiệm toàn bộ tầng điều khiển thời gian thực, tách biệt hoàn toàn với Jetson.



Hình 3.5: Vi điều khiển ESP32 S3 [20]

3.1.2. Sơ đồ kết nối và phân bổ tài nguyên phần cứng

a) Module nguồn 12V

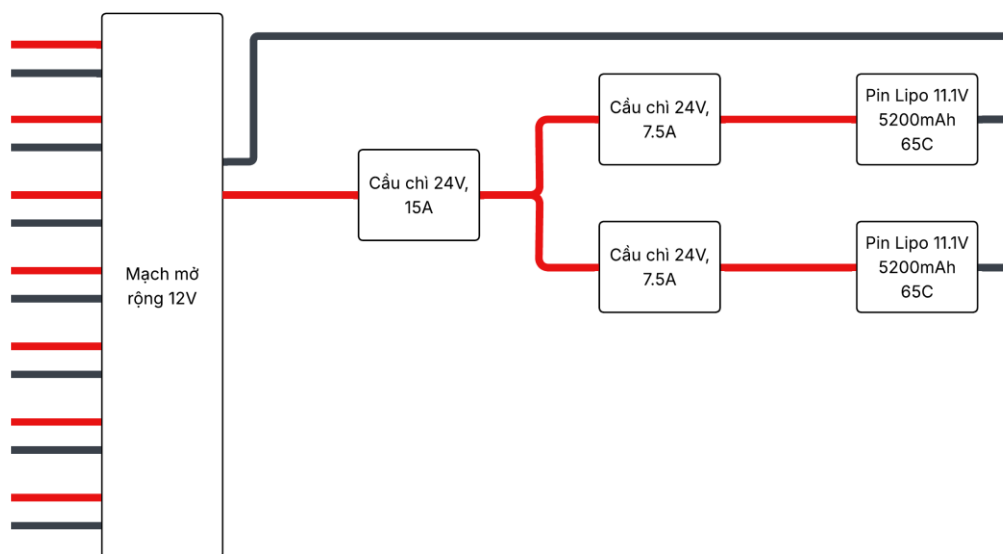
Để hệ thống có thể hoạt động với đầy đủ công suất cũng như là có thể phòng tránh những nguy hiểm đến từ pin Lipo như quá dòng hay chập cháy thì nhóm đã thiết kế một module nguồn 12V với hai pin Lipo mắc song song và sử dụng ba cầu chì để đảm bảo sự an toàn cho hệ thống mạch của hệ thống.

Hiện tại, hệ thống sử dụng board Jetson AGX Xavier với thông số định mức là 12V, 5A cùng với đó là bốn động cơ DC với mỗi động cơ tiêu thụ dòng tối đa là 3A ở điện áp 12V. Hơn nữa, ở các mạch xử lý logic như ESP32 S3, BST7960, BNO055, PMW3901 tiêu thụ tổng của các mạch xấp xỉ 1.2A ở điện áp 5V (riêng PMW3901 là được cấp 3.3V qua mạch hạ áp AMS1117 3V3). Do đó, nhóm đã chọn sử dụng hai pin Lipo với thông số là 11.1V, 5200mAh, 65C mắc song song nhau để có thể cung cấp nguồn 12V ổn định trong thời gian dài.

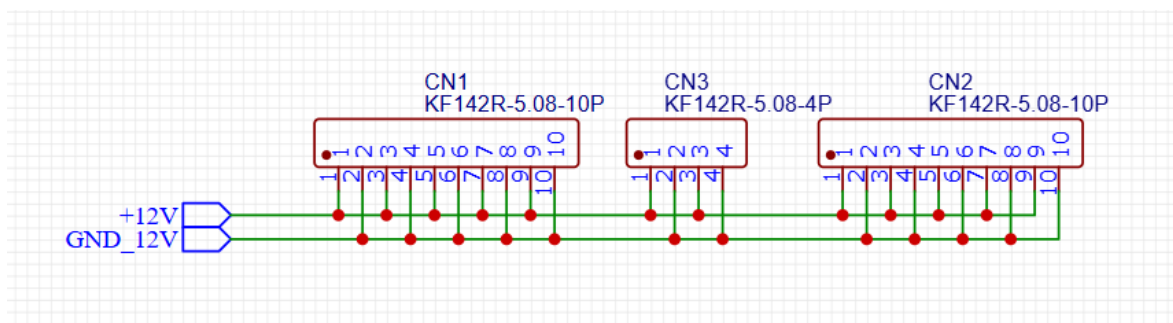


Hình 3.6: Pin Lipo được nhóm sử dụng [21]

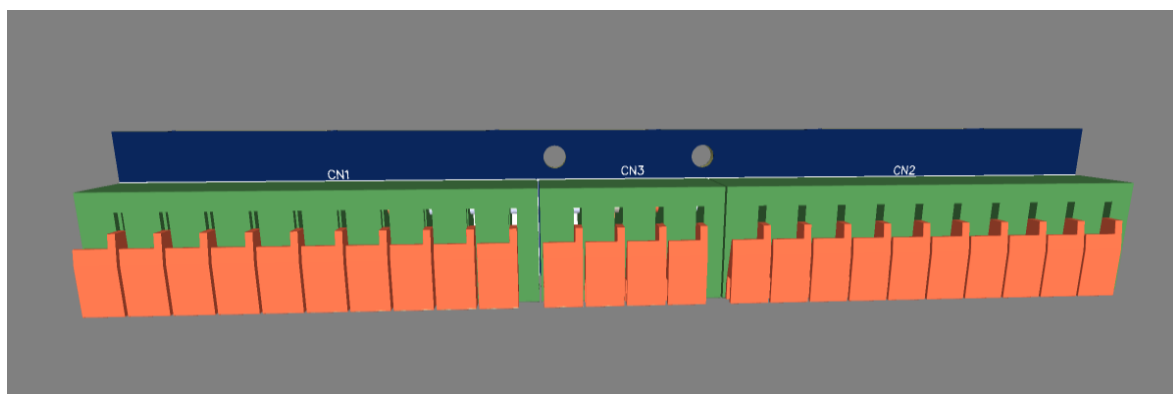
Dựa trên tài liệu [22], [23] nhóm đã sử dụng hai cầu chì 24V, 7.5A cho mỗi cực pin để ngăn tình trạng xả ngược khi hai cực pin bị lệch áp và sử dụng một cầu chì tổng có giá trị 24V, 15A để đảm bảo an toàn cho hệ thống. Sơ đồ ??? cho thấy cách đi dây nguồn của nhóm cho hệ thống.



Hình 3.7: Sơ đồ kết nối của nguồn 12V



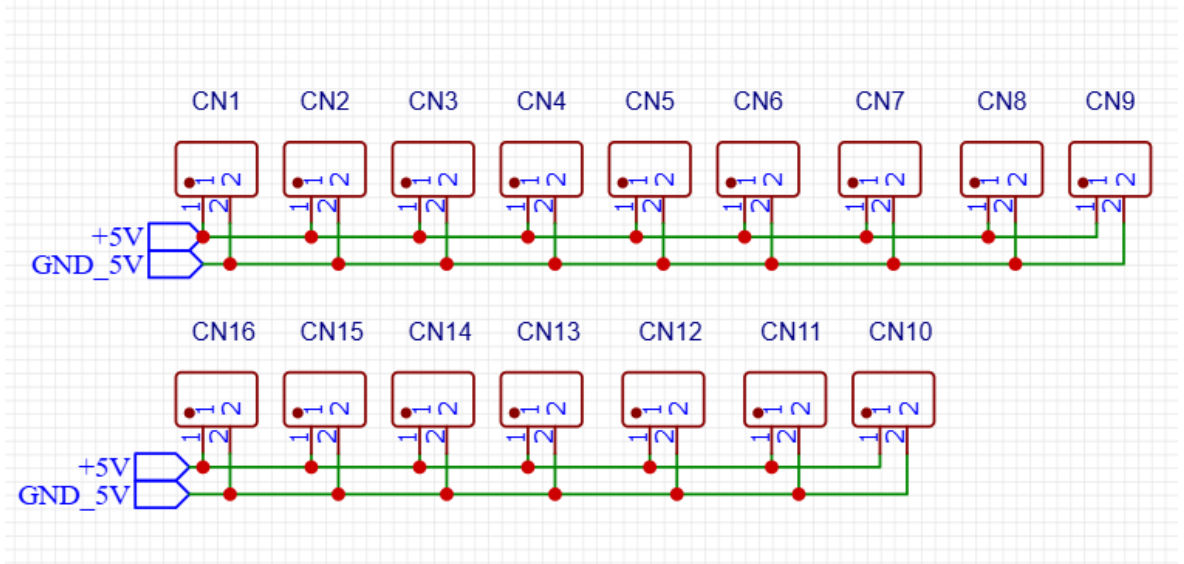
Hình 3.8: Sơ đồ kết nối của mạch mở rộng 12V



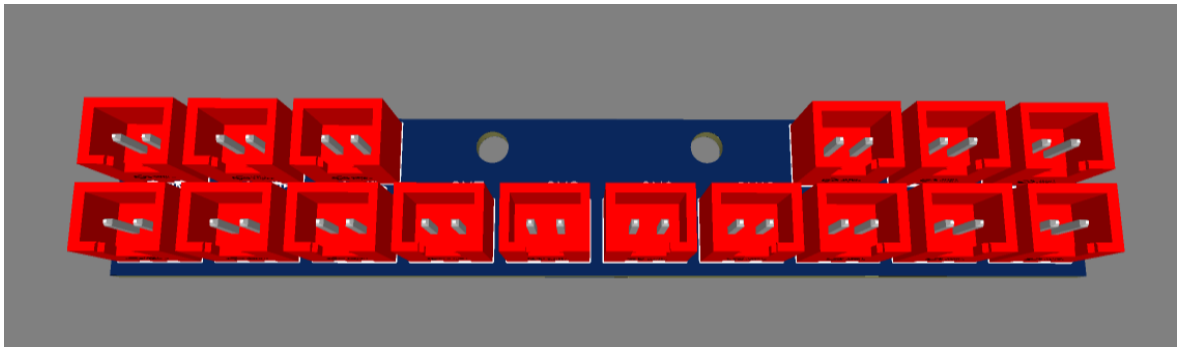
Hình 3.9: Hình ảnh hoàn thiện của mạch mở rộng 12V

b) Module nguồn 5V

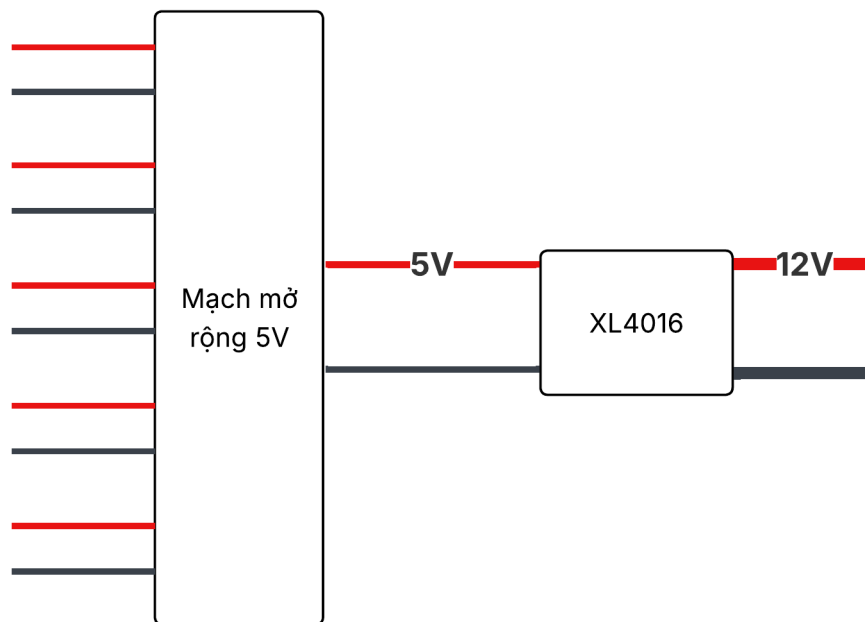
Để cấp nguồn 5V cho các mạch xử lý logic thì nhóm đã sử dụng mạch hạ áp XL4016 kết hợp với mạch mở rộng giắc cắm.



Hình 3.10: Sơ đồ đi dây của mạch mở rộng 5V



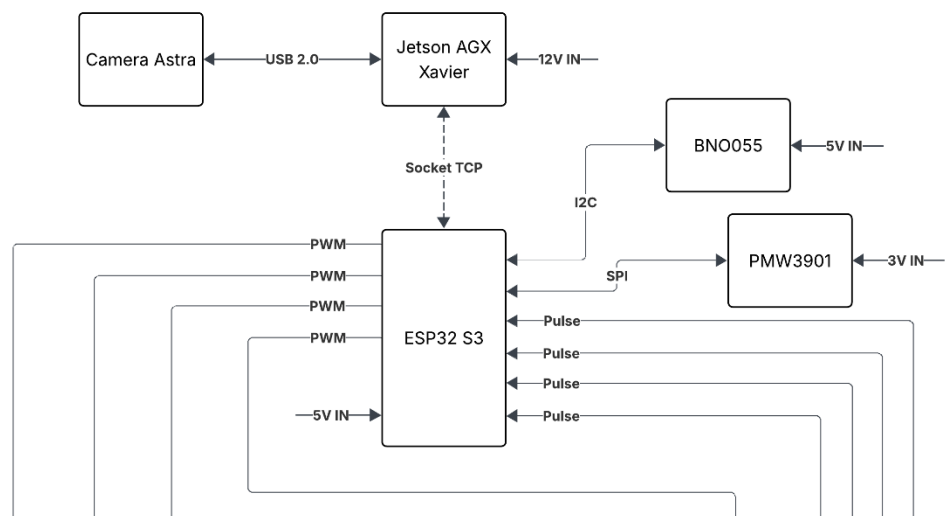
Hình 3.11: Hình ảnh hoàn thiện của mạch mở rộng 5V

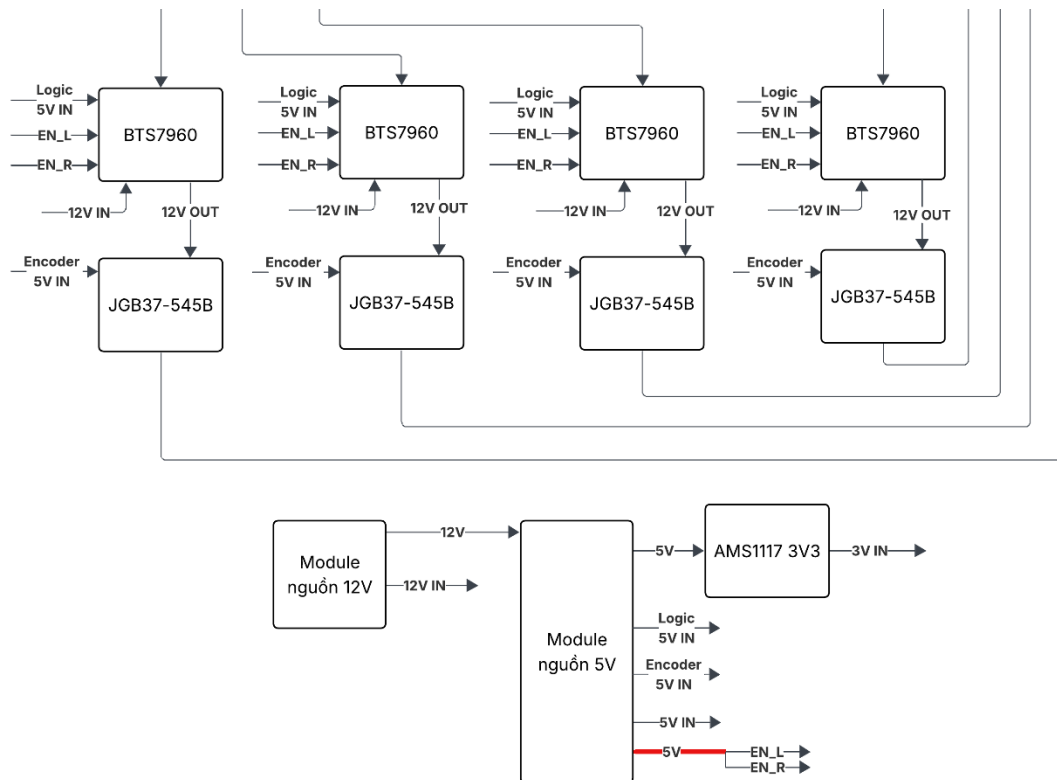


Hình 3.12: Sơ đồ kết nối của nguồn 5V

c) Sơ đồ kết nối của đế mecanum

Sơ đồ dưới đây thể hiện kết nối vật lý của toàn hệ thống. Trong đó, board Jetson sẽ đảm nhiệm tác vụ cần tiêu tốn nhiều tài nguyên tính toán như VLM, ROS2 và điều phối vi điều khiển cấp thấp là ESP32 S3 để thao tác cũng như là lấy thông tin từ đế mecanum.





Hình 3.13: Sơ đồ kết nối của đế mecatum

3.2. Camera chiều sâu Orbbec Astra

3.2.1. Nguyên lý cảm biến chiều sâu



Hình 3.14: Camera Orbbec Astra [24]

Camera Orbbec Astra là cảm biến RGB-D, cung cấp đồng thời ảnh màu (RGB) và ảnh chiều sâu (depth). Mỗi điểm ảnh trên ảnh chiều sâu mang giá trị khoảng cách

(đơn vị mm) từ mặt phẳng cảm biến tới bề mặt vật thể. Khác với webcam thông thường vốn chỉ cho ảnh 2D, dữ liệu chiều sâu cho phép khôi phục tọa độ ba chiều thực của vật thể, là điều kiện cần để robot ước lượng đúng vị trí và chiều cao vật khi tiếp cận, gấp, hoặc tránh va chạm.

3.2.2. Mô hình camera pinhole và phép chiếu ngược

Với mô hình camera pinhole, một điểm (X, Y, Z) trong hệ tọa độ camera được chiếu lên mặt ảnh tại pixel (u, v) thông qua ma trận nội tại (intrinsics) K :

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} fx & 0 & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (3.1)$$

Trong đó f_x, f_y là tiêu cự (theo pixel) và (c_x, c_y) là điểm chính (principal point). Khi đã biết chiều sâu $Z = \text{depth}(u, v)$ từ ảnh depth, ta thực hiện phép chiếu ngược (unprojection) để khôi phục tọa độ thực:

$$X = \frac{(u - c_x)Z}{f_x}, \quad Y = \frac{(v - c_y)Z}{f_y}, \quad Z = \text{depth}(u, v) \quad (3.2)$$

Công thức (2.31) là nền tảng cho phép thay thế giả định cứng "camera đặt cao đúng 300 mm so với mặt bàn" trong mô hình quy đổi tọa độ ở Chương 3 — vì khi đó tọa độ thực (X, Y, Z) được tính trực tiếp từ chiều sâu đo được, không còn phụ thuộc vào giả định hình học cố định.

3.2.3. Căn chỉnh depth-RGB và quy ước hệ tọa độ

Ảnh chiều sâu và ảnh màu được căn chỉnh (depth-to-color alignment) trực tiếp bằng phần cứng/SDK để mỗi pixel màu ứng đúng một giá trị chiều sâu, tránh sai lệch khi căn bằng phần mềm. Báo cáo tuân theo quy ước OpenCV/ROS cho khung tọa độ camera quang học: trục X hướng phải, trục Y hướng xuống, trục Z hướng theo phương nhìn. Giá trị chiều sâu không hợp lệ (bề mặt phản xạ kém, ngoài tầm đo) được đánh dấu NaN thay vì 0, nhằm phân biệt rõ "không đo được" với "khoảng cách bằng không".

3.3. Kiến trúc phần cứng NVIDIA Jetson AGX Xavier



Hình 3.15: Jetson AGX Xavier x211 [25]

Jetson AGX Xavier là hệ thống trên chip (SoC) NVIDIA Tegra Xavier, phát hành 2018. Đây là nền tảng tham chiếu cho các ứng dụng AI ở biên có yêu cầu tính toán cao.

Bảng 3.1: Thông số phần cứng chính của Jetson AGX Xavier 32 GB. [25]

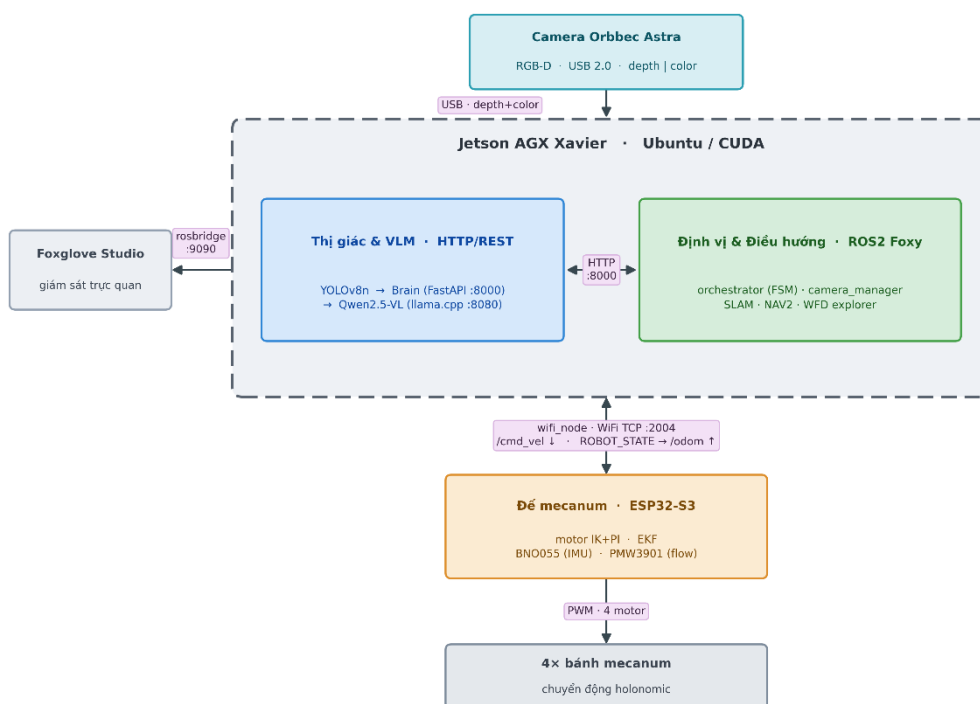
Thành phần	Thông số
CPU	8-core Carmel ARMv8.2 @ 2,26 GHz (4 cụm × 2 lõi)
GPU	NVIDIA Volta, 384 nhân CUDA + 48 nhân Tensor
Hiệu năng GPU	~ 1,4 TFLOPS FP32; ~ 11 TOPS INT8
Bộ nhớ	32 GB LPDDR4x, băng thông 137 GB/s, dùng chung CPU + GPU (unified)
Lưu trữ	32 GB eMMC tích hợp; khe NVMe M.2 (mở rộng)

Công suất	10 W / 15 W / 30 W (preset) hoặc MAXN (không giới hạn)
Kết nối	Gigabit Ethernet, USB 3.1, CSI camera, GPIO 40 chân
Phần mềm	L4T (Ubuntu 20.04), CUDA 11.4, cuDNN 8.x, TensorRT 8.x

3.4. Thiết kế hệ thống phần mềm

3.4.1. Tổng quan kiến trúc hệ thống

Hệ thống được tổ chức thành hai phân hệ phối hợp, mỗi phân hệ giải quyết một nhóm bài toán và giao tiếp với nhau qua một ranh giới rõ ràng:



Hình 3.16: Sơ đồ tổng quan kiến trúc hệ thống

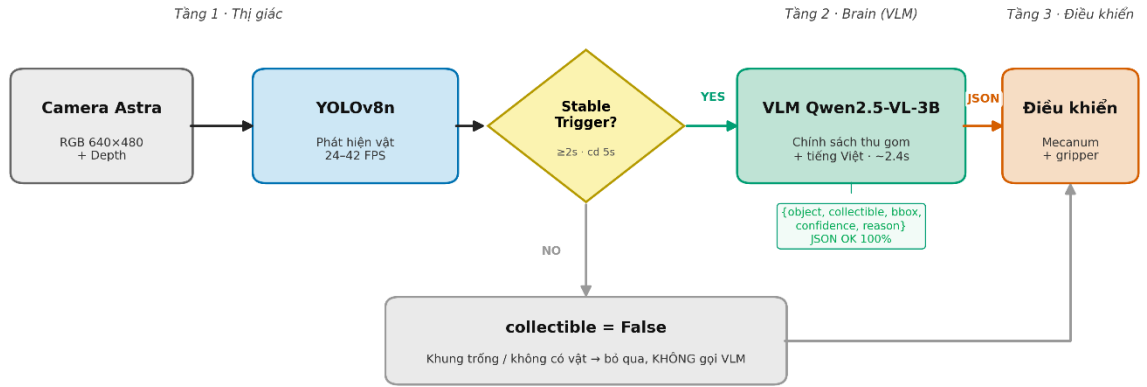
Hệ thống gồm bốn phần: camera thu dữ liệu, Jetson AGX Xavier làm bộ xử lý, ESP32 điều khiển động cơ, Foxglove để giám sát và debug. Các phần nối với nhau qua mạng và USB để ổn định trong việc phát triển và kiểm thử riêng.

- **Camera Orbbec Astra:** Nguồn dữ liệu của hệ thống, nối với Jetson qua USB 2.0. Camera trả về hai luồng đã config: ảnh RGB để nhận diện vật thể và ảnh Depth để tính tọa độ 3D thực.
- **Jetson AGX Xavier:** Chạy Ubuntu và CUDA, xử lý ảnh kết hợp ngữ nghĩa real-time với 2 phân hệ chạy song song:
 - **Phân hệ Vision & VLM:** Ảnh RGB đi qua YOLOv8n để tìm vật thể, kết quả chuyển sang Brain (FastAPI, port :8000) để ra quyết định. Khi cần hiểu ngữ cảnh sâu hơn, Brain gọi Qwen2.5-VL chạy trên llama.cpp (port :8080).
 - **Phân hệ định vị & điều hướng:** Chạy trên ROS2 Foxy. Gồm node orchestrator là máy trạng thái điều phối cả chu trình nhiệm vụ, camera_manager quản lý và chia luồng ảnh cho các node khác. SLAM dựng bản đồ, NAV2 lập đường đi và bám quỹ đạo, WFD explorer tự sinh mục tiêu cho các vùng chưa khám phá.
 - **Kết nối hai phân hệ:** Điều hướng hỏi Brain và nhận lại kết quả suy luận để quyết định trạng thái tiếp theo. Tách qua HTTP giúp tránh xung đột thư viện giữa ROS2 và các mô hình trong hệ thống.
- **Đế mecatronics:** Tầng điều khiển real-time. Firmware chuyển vận tốc thân robot thành vận tốc từng bánh bằng động học nghịch, mỗi động cơ có bộ điều khiển PI riêng. Bộ lọc EKF hợp nhất IMU BNO055 và cảm biến PMW3901 để ước lượng odometry.
- **Foxglove Studio:** Nối qua rosbridge để xem bản đồ, quỹ đạo, luồng ảnh và trạng thái FSM khi robot đang chạy.

3.4.2. Kiến trúc tổng thể ba lớp phần mềm

Hệ thống phần mềm được tổ chức thành ba lớp xử lý độc lập, giao tiếp qua REST API: Lớp 1 — Vision Node (phụ trách chụp ảnh và phát hiện vật), Lớp 2 — Brain Server (phụ trách suy luận ngữ nghĩa), Lớp 3 — Control Node (phụ trách điều khiển robot).

YOLO (cổng trigger) + VLM (suy luận ngữ nghĩa, offline)



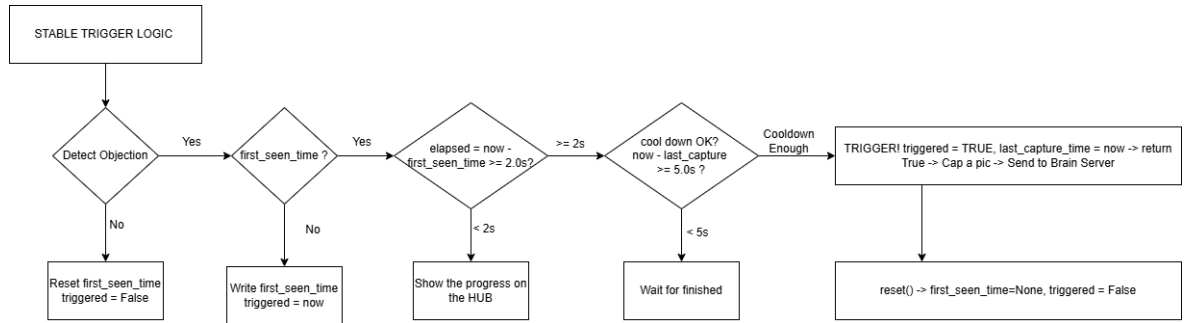
Tốc độ của YOLO + suy luận tiếng Việt của VLM — không trả giá bằng tải tính toán liên tục nhờ cơ chế trigger.

Hình 3.17: Sơ đồ kiến trúc ba lớp của hệ thống robot thu gom.

Hình 3.17 thể hiện hướng dữ liệu: Lớp 1 chụp ảnh và phát hiện vật, kích hoạt sự kiện thì gửi ảnh và metadata YOLO qua HTTP đến Lớp 2; Lớp 2 gọi llama.cpp ở cổng 8080 để suy luận VLM rồi trả về JSON; nếu vật là thu gom được thì Lớp 1 chuyển tiếp lệnh đến Lớp 3 qua HTTP. Việc tách ba lớp đem lại ba lợi ích cốt lõi: (i) tách biệt mối quan tâm (separation of concerns), (ii) test từng lớp độc lập qua mock mode, (iii) khả năng triển khai phân tán nếu sau này muốn dời Brain Server lên server riêng.

3.4.3. Lớp Vision: pipeline phát hiện và cơ chế kích hoạt

Lớp 1 đọc khung hình từ webcam ở độ phân giải 640×480, 30 FPS, sau đó đưa qua YOLOv8n để phát hiện vật. Tập 50 lớp COCO được lọc giữ làm mục tiêu thu gom (chai, cốc, đĩa, dao, thìa, táo, chuối, sách, điện thoại,...). Ngưỡng tự tin (confidence threshold) được đặt 0,45, là điểm cân bằng giữa precision và recall trong các thực nghiệm sơ bộ.

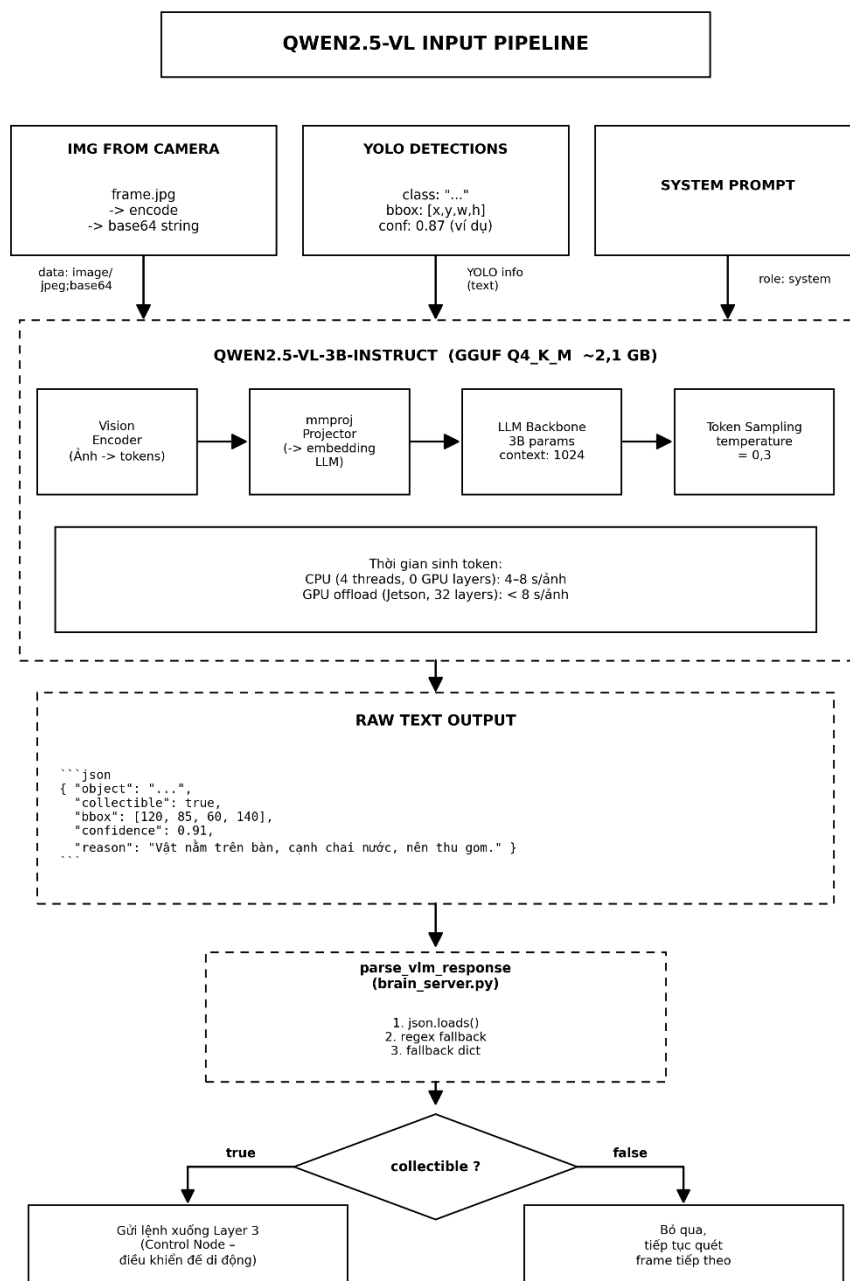


Hình 3.18: Lưu đồ giải thuật của thuật toán StableTrigger

Cụ thể, hệ thống duy trì bốn trạng thái: idle (không có vật), tracking (đang quan sát một vật xuất hiện), triggered (đã đủ điều kiện chụp) và cooldown (vừa chụp xong, chờ thời gian nguội). Các tham số chính gồm T_{stable} (thời gian tối thiểu vật phải xuất hiện liên tục) và T_{cool} (thời gian sau khi chụp không được chụp lại). Phân tích định lượng cho thấy chọn $T_{stable} = 2$ s, $T_{cool} = 5$ s loại bỏ phần lớn các false-positive do nhiễu hoặc vật thoáng qua khung hình.

3.4.4. Lớp Brain: tích hợp VLM qua llama.cpp

Lớp 2 (Brain Server) đóng vai trò proxy có logic giữa Lớp 1 và llama-server. Lý do tách riêng Brain Server thay vì gọi llama-server trực tiếp từ Lớp 1 nằm ở tính trừu tượng: nếu có thay đổi ở backend (ví dụ chuyển sang Ollama hoặc vLLM) thì chỉ phải sửa Brain Server, Lớp 1 và Lớp 3 không bị ảnh hưởng.



Hình 3.19: Pipeline xử lý lớp Brain

Thiết kế prompt tiếng Việt: Prompt được thiết kế gồm hai phần: System Prompt định nghĩa vai trò của VLM là “trợ lý AI của robot thu gom”, và User Prompt yêu cầu trả về một đối tượng JSON có cấu trúc cố định với năm trường: object (tên vật), collectible (true/false), bbox (tọa độ [x, y, w, h]), confidence (0,0–1,0) và reason (lý do bằng tiếng Việt). Toàn văn prompt được trình bày trong Phụ lục A.

Cơ chế phân tích JSON ba cấp: Dù temperature = 0,3 và prompt yêu cầu rõ định dạng JSON, mô hình vẫn có thể sinh ra đầu ra không hợp lệ trong một số trường hợp (chèn lời giải thích trước/sau JSON, dùng markdown code block). Để xử lý mọi tình huống, Brain Server hiện thực ba cấp fallback:

- (1) parse trực tiếp bằng json.loads.
- (2) regex trích JSON từ khối mã (``json ... ``) hoặc cặp ngoặc nhọn cân bằng.
- (3) trả về kết quả mặc định với confidence = 0 và đánh dấu mock = true.

Mock mode: Brain Server tự động chuyển sang mock mode khi không kết nối được tới llama-server. Trong chế độ này, nếu có metadata YOLO thì kết quả trả về sẽ dùng thông tin YOLO; nếu không thì trả về object = "unknown_object". Cơ chế này cho phép phát triển và kiểm thử cho Lớp 1 mà không cần khởi động VLM, giảm thời gian chu kỳ phát triển.

3.4.5. Lựa chọn thiết kế cho thiết bị nhúng Jetson Xavier

Vì Jetson dùng bộ nhớ thống nhất giữa CPU và GPU, mọi cấp phát đều trừ vào cùng một tài nguyên 32 GB. Bảng dưới trình bày phân bổ thực tế khi hệ thống vận hành đầy đủ.

Bảng 3.2: Ngân sách bộ nhớ trên Jetson AGX Xavier 16 GB.

Thành phần	Bộ nhớ chiếm	Ghi chú
L4T base + dịch vụ hệ thống	≈ 2,0 GB	Ubuntu 20.04 + dịch vụ NVIDIA
YOLOv8n model + buffer	≈ 0,5 GB	Model 6,5 MB + overhead PyTorch
Qwen2.5-VL trọng số (Q4_K_M)	≈ 2,1 GB	Load qua mmap
mmproj projector (FP16)	≈ 0,6 GB	Load riêng
KV cache (ctx = 1024)	≈ 0,5 GB	

Activations + buffer trung gian	$\approx 1,0$ GB	Khi forward pass
Tổng peak VLM	$\approx 5,0$ GB	Tổng bốn dòng trên
Dự phòng cho ứng dụng + Layer 3	$\approx 9,0$ GB	Còn lại

Mức lượng tử hóa Q4_K_M được chọn vì cân bằng giữa dung lượng ($\sim 2,1$ GB, vừa VRAM khả dụng của GPU Volta) và suy giảm chất lượng (perplexity +3–5% so với FP16, chấp nhận được cho bài toán phân loại có cấu trúc). Chiến lược GPU offloading đề xuất "offload tối đa" (`n_gpu_layers = -1`) làm điểm khởi đầu, hiệu chỉnh xuống nếu OOM. Ngưỡng cảnh ctx = 1024 token tiết kiệm KV cache ~ 500 MB so với mức 4096 phổ biến.

3.4.6. Tích hợp ở cấp service

Để hệ thống có thể vận hành liên tục trong môi trường robot — không có người giám sát thường xuyên — nhóm đề xuất ba dịch vụ systemd với chuỗi phụ thuộc rõ ràng: `vlm-server.service` khởi chạy `llama-server`, `brain-server.service` phụ thuộc `vlm-server`, `vision-node.service` phụ thuộc `brain-server`. Mỗi dịch vụ đặt `Restart = on-failure` và `RestartSec = 5` để tự phục hồi nếu một thành phần lỗi.

3.5. Tích hợp camera chiều sâu Astra

3.5.1. Vấn đề và động cơ thiết kế

Thiết kế ban đầu dùng webcam và giả định camera đặt cao đúng 300mm so với mặt sàn để quy đổi tọa độ pixel sang tọa độ thực. Giả định cứng này gây hạn chế mà dữ liệu chiều sâu có thể giải quyết được liệt kê ở bảng dưới.

Bảng 3.3: So sánh vấn đề sử dụng camera

Webcam 2D	Camera depth Orbbec Astra
Sai số lớn khi camera lệch	(X, Y, Z) thực dùng depth + intrinsics
Không biết được chiều cao vật	Lấy median depth trong khung cho ra được Z chính xác

Không phát hiện vật cản trên quỹ đạo	Lấy mẫu depth dọc quỹ đạo để cảnh báo va chạm
Quy đổi tọa độ chỉ là xấp xỉ 2D	Khung RGB + Depth đã căn chỉnh cho ra tọa độ

3.5.2. Lớp trừu tượng camera (Camera Abstraction Layer)

Thay vì gọi trực tiếp `cv2.VideoCapture` trong `vision_node.py`, phần truy cập camera được tách thành giao diện trừu tượng, cho phép hoán đổi phần cứng/SDK mà không sửa mã gọi phía trên:

```
layer1_vision/cameras/
├── base.py      # Camera ABC + Frame dataclass + CameraIntrinsics
├── webcam.py    # Bọc cv2.VideoCapture (giữ làm tham chiếu / rollback)
├── astra.py     # Hiện thực Orbbec Astra (pyorbbecsdk)
└── mock.py     # Nạp RGB+depth từ file .npz cho kiểm thử ngoại tuyến
```

Hình 3.20: Cấu trúc lớp trừu tượng camera

Mỗi lần đọc trả về một Frame gồm: ảnh RGB, ảnh depth (mm, float32, NaN cho pixel không hợp lệ), timestamp và intrinsics. Thiết kế này giải quyết rủi ro "model Astra chốt sau khi viết mã": dù dùng Orbbec SDK (dòng mới) hay OpenNI2 (dòng cũ), lớp gọi không thay đổi; driver được chọn qua `config.json`. Đây cùng tinh thần với việc tách Brain Server khỏi llama-server.

3.5.3. Tương thích ngược và truyền metadata chiều sâu

Hàm quy đổi tọa độ được giữ tương thích ngược: vẫn chạy xấp xỉ pinhole cũ khi không có depth, chỉ dùng đường chiếu ngược chính xác (phương trình 2.30) khi có `depth_frame` và intrinsics. Để không gửi toàn bộ khung depth qua HTTP (≈ 600 KB/khung $\times 30$ FPS = 18 MB/s, đủ nghẽn mạng), Lớp 1 cắt vùng khung bao và tính sẵn metadata trước khi gửi đi. Thể hiện ở hình log bên dưới :

```
{ "object": "bottle", "collectible": true, "bbox": [120,80,200,300], "confidence": 0.87,
  "depth_metadata": { "median_z_mm": 425.3, "min_z_mm": 410.1, "max_z_mm": 480.5, "valid_ratio": 0.92 },
  "intrinsics": { "fx": 570.34, "fy": 570.34, "cx": 319.5, "cy": 239.5 } }
```

Hình 3.21: Cấu trúc dữ liệu truyền đi sau khi detect vật thể

3.6. Phân hệ định vị và điều hướng tự hành ROS2

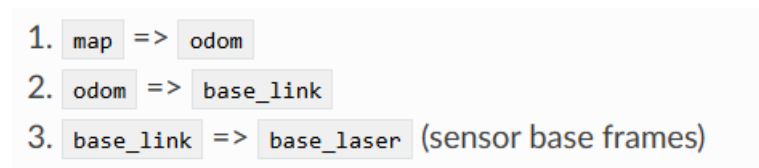
3.6.1. ROS2 và ranh giới phân hệ thị giác

Điều khiển chuyển động của đế di động được xây trên ROS2 Foxy thay vì mở rộng giao thức HTTP, điều hướng tự hành cần các hạ tầng chuẩn mà ROS2 cung cấp sẵn: cây biến đổi tọa độ tf2, SLAM 2D (slam_toolbox) và khung điều hướng (NAV2) — không có giải pháp HTTP tương đương. Lớp ROS2 không thay thế phân hệ thị giác ba lớp; node điều phối vlm_nav_orchestrator vẫn gọi Brain Server qua HTTP cổng 8000.

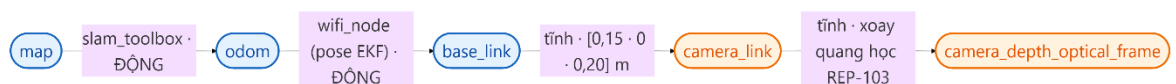
Quy trình tổng thể: NAV2 đưa robot tới khu vực mục tiêu trên bản đồ → bàn giao cho thị giác (VLM + visual servoing) ở pha tiếp cận cuối.

3.6.2. Cây biến đổi tọa độ và mô tả robot

Cây TF của hệ thống được thiết kế dựa theo cây TF của tác giả trên chuẩn REP-105 [26].



Hình 3.22: Cây TF thiết kế theo chuẩn REP-105 [13]

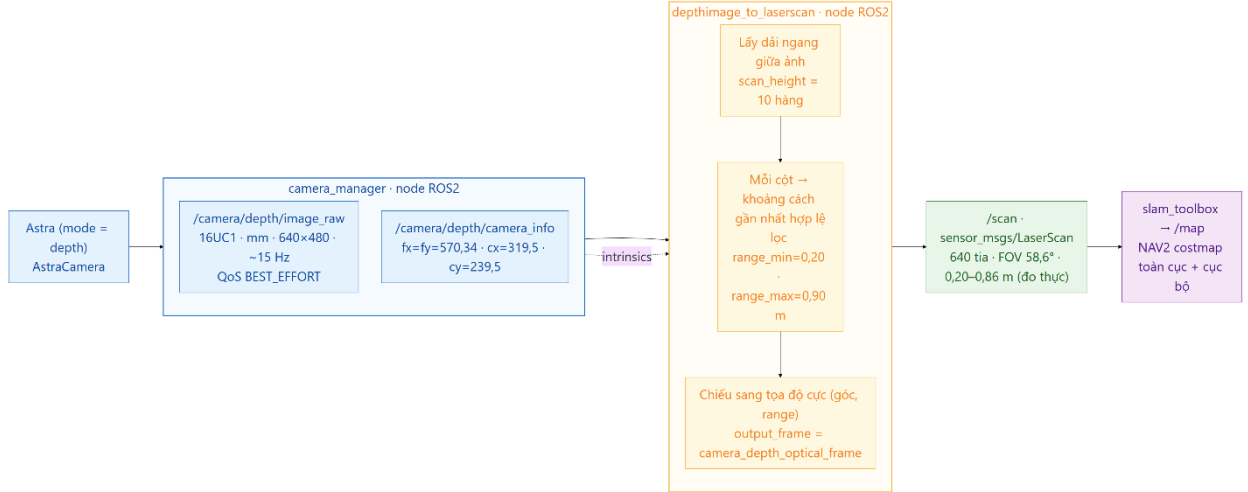


Hình 3.23: Cây TF dựng lại của hệ thống dựa theo chuẩn REP-105

Trong đó map → odom do slam_toolbox publish, odom → base_link do cầu nối để publish từ pose EKF, base_link → camera_* là biến đổi tĩnh. Gói robot_description định nghĩa URDF của đế mecanum và camera. Biến đổi tĩnh base_link → camera_depth_optical_frame hiện dùng tịnh tiến [0.15, 0, 0.20] m và quaternion quang học [-0.5, 0.5, -0.5, 0.5].

3.6.3. Pipeline cảm biến - depth image tới LaserScan

Hệ thống dùng depthimage_to_laserscan chuyển một dải ngang giữa ảnh depth thành LaserScan (/scan):

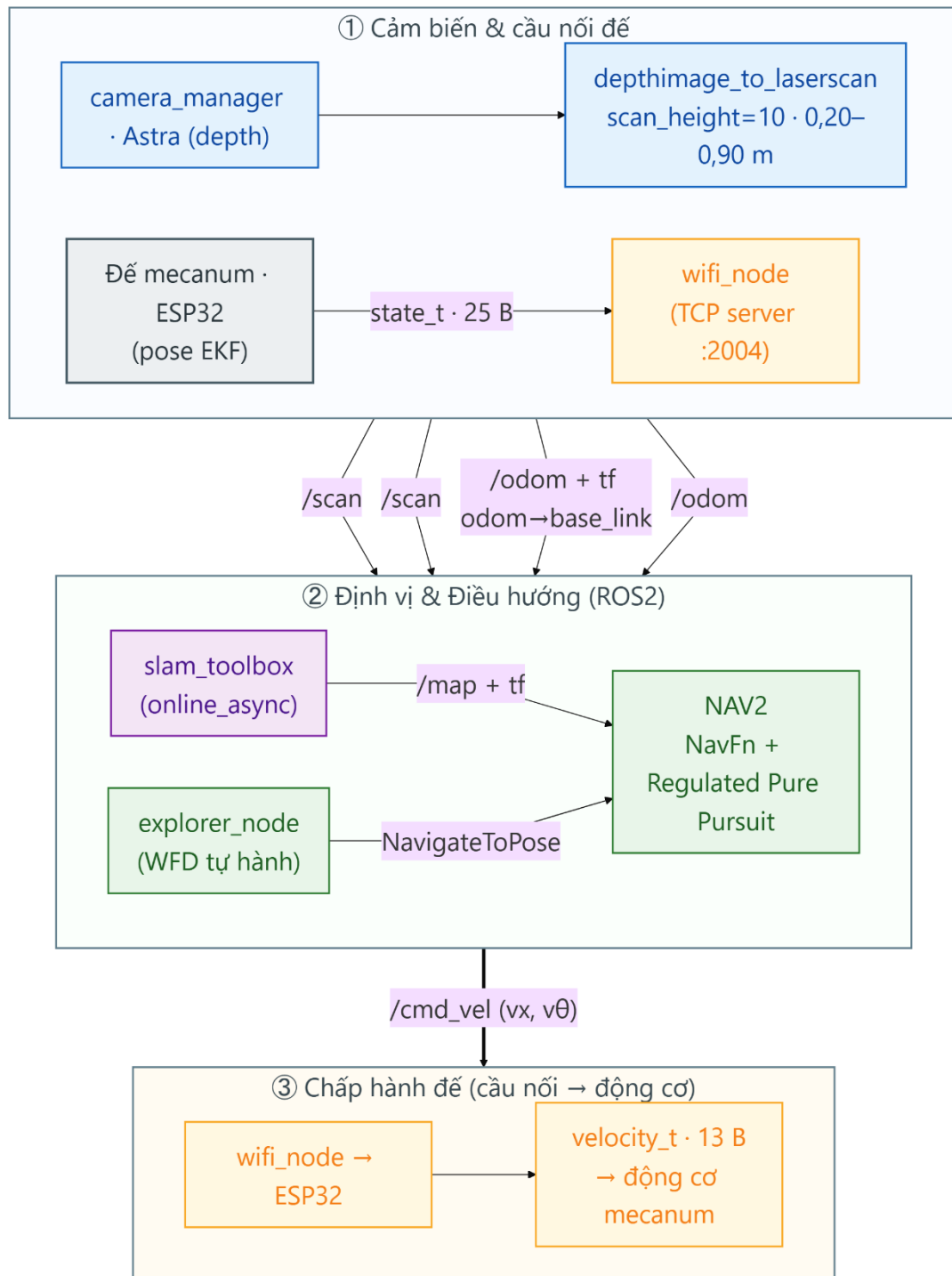


Hình 3.24: Pipeline cảm biến – Depth Image to Laser Scan

Hệ thống tái sử dụng camera Astra làm cảm biến quét cho điều hướng:

- Node camera_manager xuất bản ảnh chiều sâu /camera/depth/image_raw (định dạng 16UC1, đơn vị mm) cùng thông số nội /camera/depth/camera_info.
- Node depthimage_to_laserscan lấy một dải ngang giữa ảnh (scan_height = 10 hàng), với mỗi cột chọn khoảng cách hợp lệ gần nhất (lọc trong 0,20–0,90 m), rồi chiếu sang tọa độ cực để tạo bản quét /scan (640 tia, trường nhìn 58,6°, tầm thực đo 0,20–0,86 m, gán output_frame = camera_depth_optical_frame).
- Bản quét này được slam_toolbox dùng để dựng bản đồ và NAV2 dùng để cập nhật costmap né vật cản.

3.6.4. Cấu hình SLAM và NAV2



Hình 3.25: Pipeline điều hướng runtime

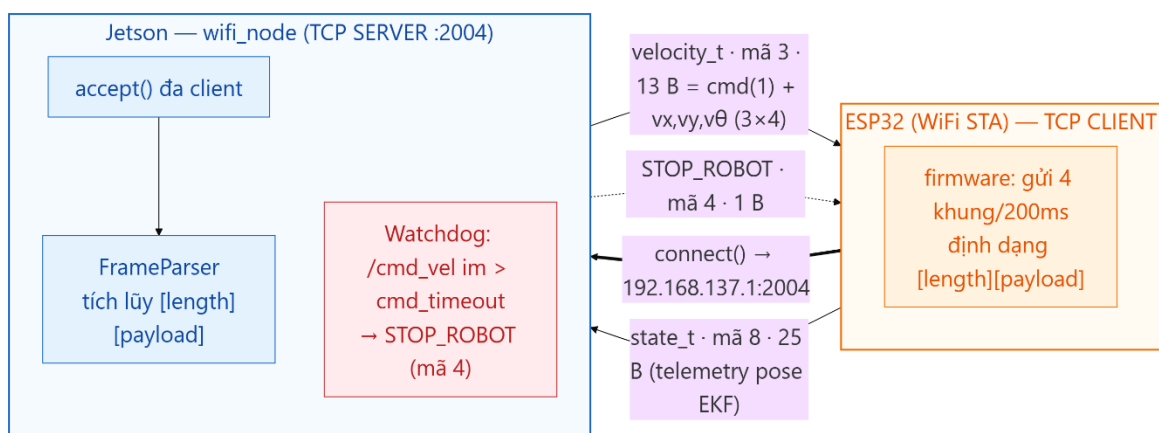
Tầng 1 - Cảm biến và cầu nối để là nguồn của mọi dữ liệu đầu vào. Node camera_manager mở camera Astra ở chế độ depth và chuyển ảnh chiều sâu cho

depthimage_to_laserscan để quy về một bản quét laser hai chiều /scan. Song song, để mecanum (vi điều khiển ESP32) liên tục gửi gói trạng thái state_t (25 byte, chứa pose ước lượng từ bộ lọc EKF) qua TCP về node cầu nối wifi_node đóng vai trò TCP server tại cổng 2004; từ gói này, wifi_node xuất bản odometry /odom kèm biến đổi tọa độ odom → base_link.

Tầng 2 - Định vị và Điều hướng là phần xử lý lỗi chạy trên ROS2. Hai thành phần ở đây cùng tiêu thụ /scan và /odom: slam_toolbox (chế độ online_async) thực hiện SLAM để dựng bản đồ /map và xuất biến đổi map → odom, khép kín cây tọa độ TF (mục 3.4.3); còn NAV2 (planner NavFn + controller Regulated Pure Pursuit) sử dụng đồng thời /scan, /odom và /map để duy trì costmap và bám đường. Mục tiêu điều hướng được cấp từ explorer_node (thuật toán WFD tự hành) qua hành động NavigateToPose, hoặc do người dùng đặt thủ công. Kết quả của tầng này là lệnh vận tốc /cmd_vel gồm thành phần tịnh tiến v_x và quay v_θ .

Tầng 3 - Chấp hành để khép vòng điều khiển: wifi_node nhận /cmd_vel, đóng gói thành bản tin velocity_t (13 byte) và gửi xuống ESP32 để điều khiển bốn động cơ mecanum. Như vậy, cùng một node wifi_node đảm nhận hai chiều của vòng lặp - telemetry đi lên thành /odom ở tầng 1, lệnh đi xuống thành velocity_t ở tầng 3 — tạo nên một chu trình cảm nhận–điều hướng–chấp hành liên tục, vận hành hoàn toàn trên thiết bị biên mà không cần kết nối ngoài

3.6.5. Giao tiếp với đế di động Mecanum qua TCP



Hình 3.26: Sơ đồ kết nối TCP và khung dữ liệu đế Mecanum

- ESP32 (TCP client - WiFi STA) Chủ động connect() tới Jetson tại 192.168.137.1:2004. Firmware gửi liên tục 4 khung/chu kỳ theo định dạng [length][payload], trong đó có gói trạng thái state_t mang pose ước lượng EKF.
- wifi_node - accept() đa client: Node TCP server trên Jetson, lắng nghe cổng 2004 và chấp nhận nhiều kết nối. Là điểm vào duy nhất cho mọi dữ liệu giữa máy tính nhúng và đế.
- FrameParser - bộ ráp khung: Vì TCP là luồng byte, nhiều khung do firmware gửi liên nhau bị gộp lại. Module này tích lũy dữ liệu theo định dạng [length][payload] và chỉ xử lý khi nhận đủ một khung hoàn chỉnh, tránh bỏ lỡ gói ROBOT_STATE (lỗi làm /odom đứng yên).
- Watchdog - dừng an toàn: Giám sát luồng /cmd_vel. Nếu lệnh vận tốc im lặng quá ngưỡng cmd_timeout, tự động gửi STOP_ROBOT (mã 4) để hãm đế, chống tình trạng robot giữ vận tốc cuối.

3.7. Hiện thực hệ thống

3.7.1. Môi trường phát triển và triển khai

Khóa luận sử dụng hai môi trường phân cứng song song. Môi trường phát triển là một máy tính cá nhân (PC) chạy Windows 11 với CPU Intel Core, RAM 16 GB; môi trường triển khai cuối cùng là NVIDIA Jetson AGX Xavier 32 GB chạy JetPack 5.x trên Ubuntu 20.04 L4T, kiến trúc ARM64 (aarch64). Mã nguồn được phát triển trên PC để tận dụng tốc độ và công cụ debug, sau đó port sang Jetson và đo lại hiệu năng. Phân hệ ROS2 dùng **ROS2 Foxy** (LT4 R35.1).

3.7.2. Hiện thực Lớp 1 — Vision Node

Lớp Vision được hiện thực trong tệp layer1_vision/vision_node.py, khoảng 410 dòng mã. Ba thành phần chính gồm: (i) hàm init_camera() khởi tạo cv2.VideoCapture(0) với độ phân giải 640×480, (ii) hàm detect_objects() chạy YOLOv8n và lọc 50 lớp COCO mục tiêu với ngưỡng 0,45, (iii) lớp StableTrigger điều phối máy trạng thái với $T_{stable} = 2,0$ s và $T_{cool} = 5,0$ s.

3.7.3. Hiện thực Lớp 2 — Brain Server

Brain Server hiện thực bằng FastAPI trong `layer2_brain/brain_server.py`, lắng nghe cổng 8000, cung cấp ba endpoint: GET /health, POST /analyze, GET /docs. Bên trong /analyze, ảnh được mã hóa Base64, gán vào payload theo chuẩn OpenAI Chat Completions, gửi tới `http://127.0.0.1:8080/v1/chat/completions` (`temperature = 0,3`, `max_tokens = 256`).

Văn bản trả về đi qua `parse_vlm_response()` thực hiện ba cấp fallback. Tập `setup_vlm.py` tải mô hình từ HuggingFace (Qwen2.5 – VL - 3B – Instruct - Q4_K_M.gguf ~2,1 GB + mmproj ~ 0,6 GB) và sinh script khởi chạy.

3.7.4. Hiện thực phân hệ ROS2

Toàn bộ ngăn xếp khởi chạy bằng một lệnh launch tổng với các tham số (`odom`, `nav`, `autonomous`, `cmd_timeout`). Node `Astra_node` bọc Astra Camera (`mode="depth"`), dùng QoS BEST_EFFORT khớp `depthimage_to_laserscan` và giảm số lần copy numpy. Cầu nối wifi_node (C++) đã được harden (`watchdog STOP`, `FrameParser`, gửi đúng 13 byte).

3.7.5. Hiện thị, ghi nhật ký và giám sát

Dùng Foxglove Studio qua WebSocket cổng 9090 để xem bản đồ, camera detection, cây TF, robot, costmap, đường đi xem từ xa.

Phân hệ thị giác ghi log định dạng `[LAYER] HH:MM:SS LEVEL: message`.

Dữ liệu môi trường khi chạy hệ thống của Jetson được đo qua `tergrastats`, parse thành CSV.

3.8. Tích hợp end-to-end

3.8.1. Nguyên lý điều phối

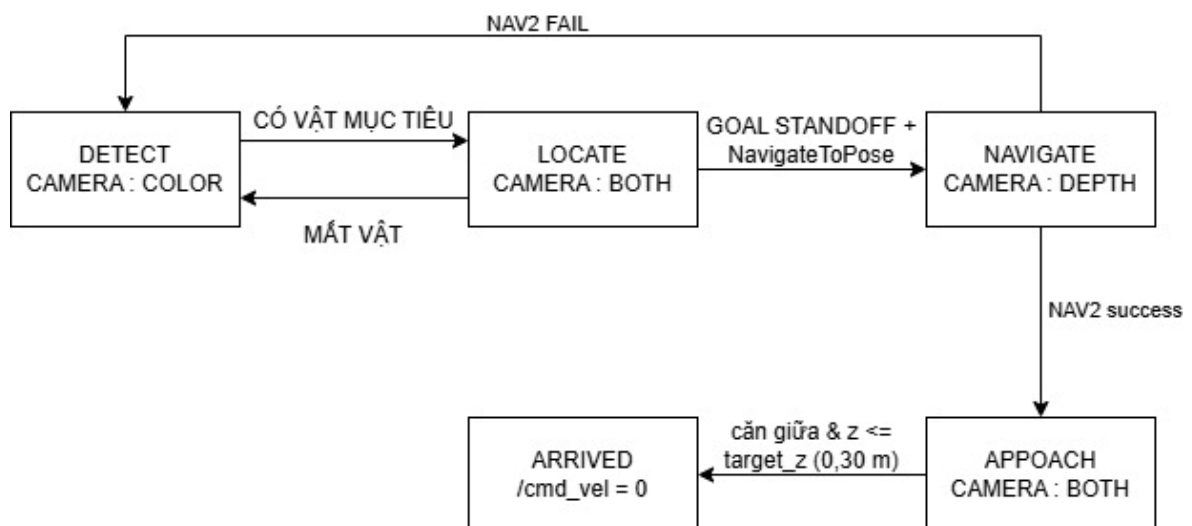
Node điều phối `vlm_nav_orchestrator` là điểm hợp nhất hai phân hệ. Do Astra trên USB 2.0 không truyền đồng thời depth và color (mục 3.3.4), một node `camera_manager` duy nhất giữ camera và đổi chế độ theo lệnh `/camera_mode` mà

orchestrator phát ra ở mỗi pha. Ràng buộc này định hình trực tiếp máy trạng thái: robot chỉ có /scan (cho SLAM/NAV2) ở pha điều hướng, và chỉ có ảnh màu (cho phát hiện) ở pha quan sát — hai việc không diễn ra cùng lúc.

Việc nhận thức trong vòng lặp end-to-end hiện do YOLOv8n đảm nhận: yolo_node đăng ký /camera/color/image_raw, phát hiện vật và xuất /yolo/detections (JSON [{class_name, confidence, bbox}]). Orchestrator chọn mục tiêu theo tham số target_class (ví dụ bottle) hoặc theo độ tự tin cao nhất. Đường suy luận ngữ nghĩa VLM (Brain/Qwen2.5-VL, mục 3.2) chạy độc lập trên giao thức HTTP.

3.8.2. Máy trạng thái của node điều phối

Orchestrator chạy ở chu kỳ 5 Hz gồm năm trạng thái, mỗi pha tự yêu cầu chế độ camera phù hợp:



Hình 3.27: Máy trạng thái của node điều phối

- **DETECT** (camera: color). Đọc /yolo/detections, chọn vật mục tiêu.
- **LOCATE** (camera: both). Lấy tâm khung bao (u, v), đọc độ sâu trung vị z quanh tâm; chiếu ngược pixel điểm 3D rồi qua tf2 (camera → map) thành tọa độ vật trên bản đồ. Tính goal đứng cách vật (standoff) quay mặt về vật, gửi NAV2 qua hành động
- **NAVIGATE** (camera: depth). NAV2 tự lập đường và né vật cản bằng /scan tới khu vực standoff..

- **APPROACH** (camera: both). Căn tâm khung bao về giữa ảnh bằng vận tốc quay v_θ , tiến tới khoảng cách mục tiêu bằng v_x ; mất vật thì dừng và chờ.
- **ARRIVED** (giữ dừng). Phát `/cmd_vel = 0`.

Node bảo đảm dừng an toàn ở mọi trạng thái: khi tắt hoặc mất vật, nó phát `/cmd_vel = 0`. Song song, watchdog trong `wifi_node` gửi `STOP_ROBOT` nếu `/cmd_vel` im quá ngưỡng.

Chương 4. ĐÁNH GIÁ KẾT QUẢ THỰC NGHIỆM

4.1. Thiết lập thực nghiệm

Các thực nghiệm được tiến hành trên hai môi trường:

- PC phát triển (Windows 11, Intel Core, RAM 16 GB, không GPU offload).
- Jetson AGX Xavier 16 GB chạy JetPack 5.x (L4T R35.1, Ubuntu 20.04).

Hệ thống dùng camera chiều sâu **Orbbec Astra (RGB-D)** và **ROS2 Foxy** (ros2_ws/) cho phân định vị – lập bản đồ – di chuyển.

Bộ dữ liệu kiểm thử gồm 51 ảnh tự chụp bằng **Astra** (31 ảnh chứa vật thu gom — collectible, 20 ảnh không), cùng nhóm ảnh “control” âm tính chuẩn. Nhãn collectible được gán thủ công làm ground truth.

Năm nhóm chỉ số được đo:

1. **YOLO:** FPS, độ trễ p50/p95, RAM.
2. **VLM:** độ trễ trung bình, tỉ lệ phân tích JSON thành công, độ chính xác collectibility, VRAM.
3. **End-to-end:** thời gian từ trigger đến JSON quyết định cuối.
4. **Edge:** nhiệt độ GPU/CPU, công suất tức thời, tần số/độ chiếm dụng GPU.
5. **Cảm biến & điều hướng:** tầm/nhiều depth Astra, trường nhìn (FOV), thông số /scan, độ trễ chuyển chế độ camera, độ bền cầu nối để.

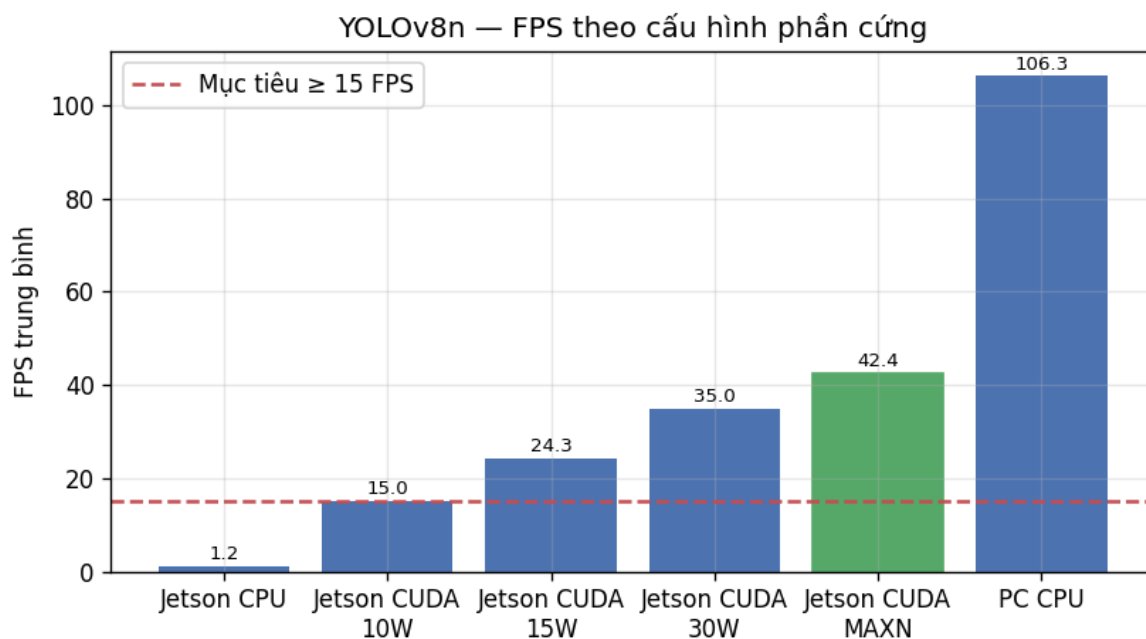
4.2. Đánh giá YOLOv8n

Bảng 4.1: Hiệu năng YOLOv8n trên PC và Jetson Xavier.

Chỉ số	PC (CPU)	Jetson (CPU – torch 2.4.1, 15W)	Jetson (CUDA – torch 1.13, 15W)	Jetson (CUDA – torch 1.13, MAXN)
--------	----------	---------------------------------------	---------------------------------------	---

FPS trung bình	106.3	1.2	24.3	42.4
Độ trễ trung bình	9.4 ms	852.4 ms	41.2 ms	23.6 ms
Độ trễ min	7.6 ms	759.0 ms	40.3 ms	23.1 ms
Độ trễ max	17.6 ms	989.5 ms	44.9 ms	24.5 ms
RAM model	21 MB	17 MB	20 MB	20 MB
RAM tổng tiến trình	≈ 450 MB	326 MB	321 MB	321 MB

Bảng 4.1 cho thấy YOLOv8n trên PC thừa khả năng đáp ứng mục tiêu ≥ 15 FPS đã đặt ra ở Chương 1; phép đo Jetson dự kiến cũng vượt ngưỡng này do GPU Volta đủ mạnh cho mô hình 3,2 triệu tham số.



Hình 4.1: FPS trung bình của YOLOv8n theo cấu hình phần cứng

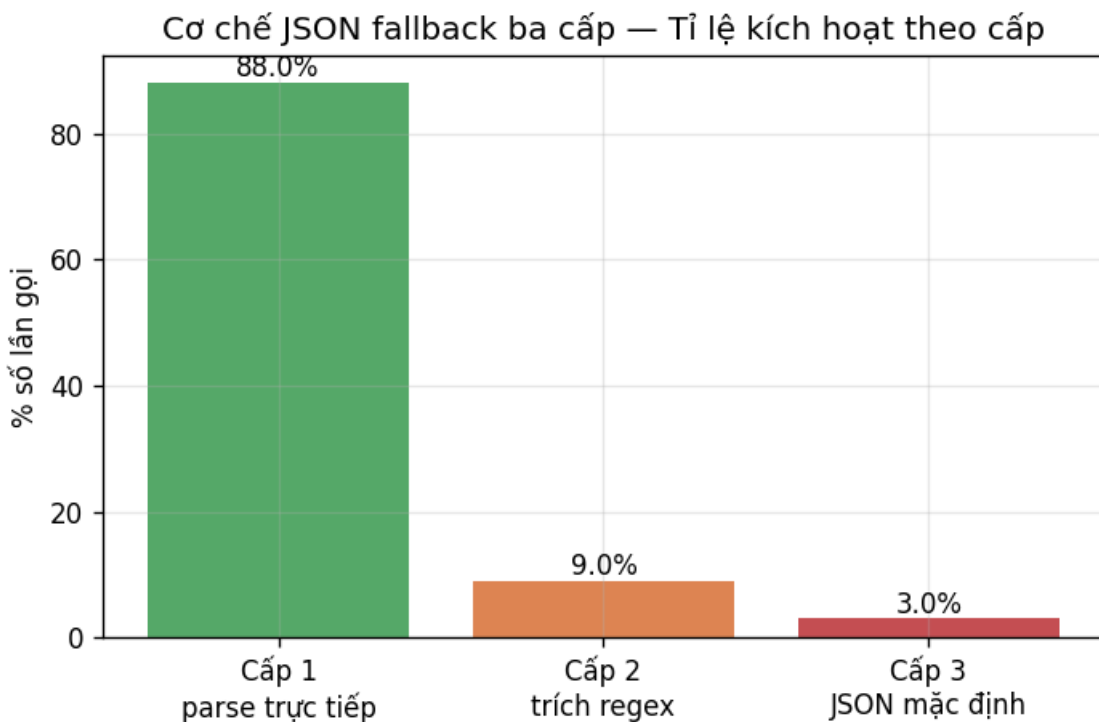
4.3. Đánh giá VLM Qwen2.5-VL-3B (Q4_K_M)

Tổng hợp kết quả ban đầu trên PC (CPU-only, ba lần chạy) và kế hoạch khảo sát `n_gpu_layers` trên Jetson. Tỷ lệ phân tích JSON thành công trên PC đạt 100% (3/3), xác nhận hiệu quả của cơ chế fallback ba cấp.

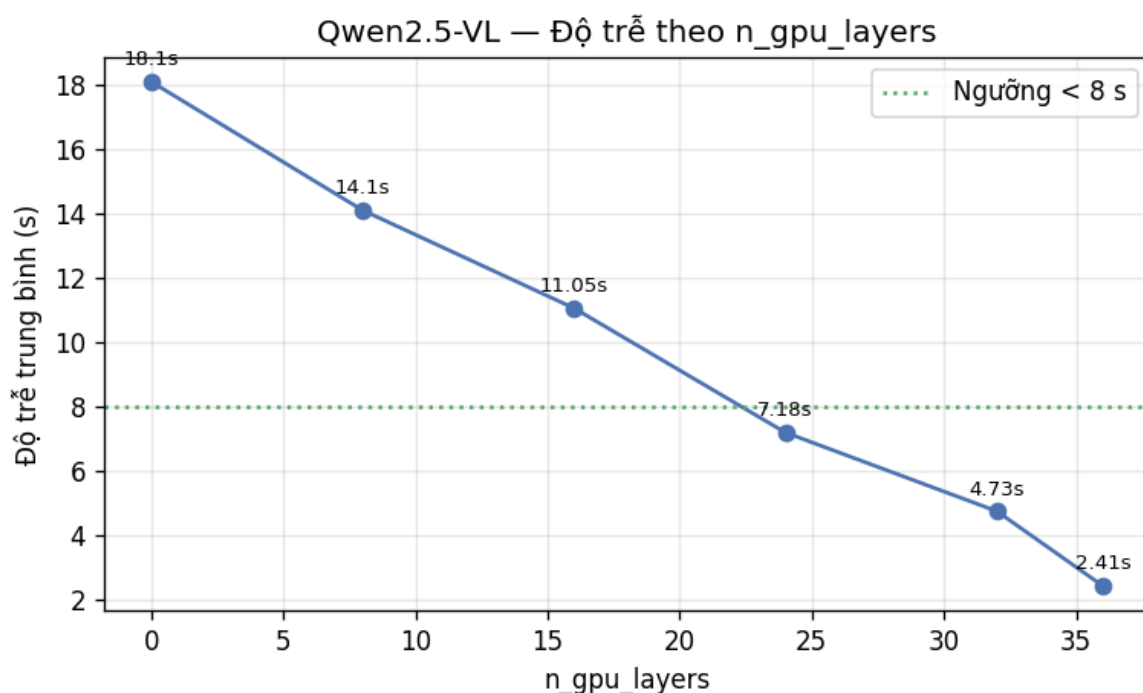
Bảng 4.2: Độ trễ và tài nguyên Qwen2.5-VL-3B Q4_K_M theo cấu hình.

Cấu hình	Độ trễ avg	VRAM/GPU	RAM tổng	JSON OK
PC, n_gpu = 0	14.14s	—	4.10 GB	100%
Jetson, n_gpu = 0	18.10s	1510 MiB	4.33 GB	100%
Jetson, n_gpu = 8	14.10s	1930 MiB	4.41 GB	100%
Jetson, n_gpu = 16	11.05s	2113 MiB	4.24 GB	100%
Jetson, n_gpu = 24	7.18s	2283 MiB	4.05 GB	100%
Jetson, n_gpu = 32	4.73s	2441 MiB	3.85 GB	100%
Jetson, n_gpu = -1	2.41s	2501 MiB	3.67 GB	100%

Khi tăng n_gpu_layers, độ trễ giảm mạnh từ 18.1 s (CPU-only) xuống 2.41 s (offload toàn bộ 36 lớp) — gấp ~7,5 lần — đổi lại VRAM tăng dần. Tỷ lệ phân tích JSON đạt 100% ở mọi cấu hình, xác nhận hiệu quả cơ chế fallback ba cấp được biểu diễn trong đồ thị dưới :



Hình 4.2: Tỷ lệ kích hoạt theo từng cấp của cơ chế JSON fallback ba cấp



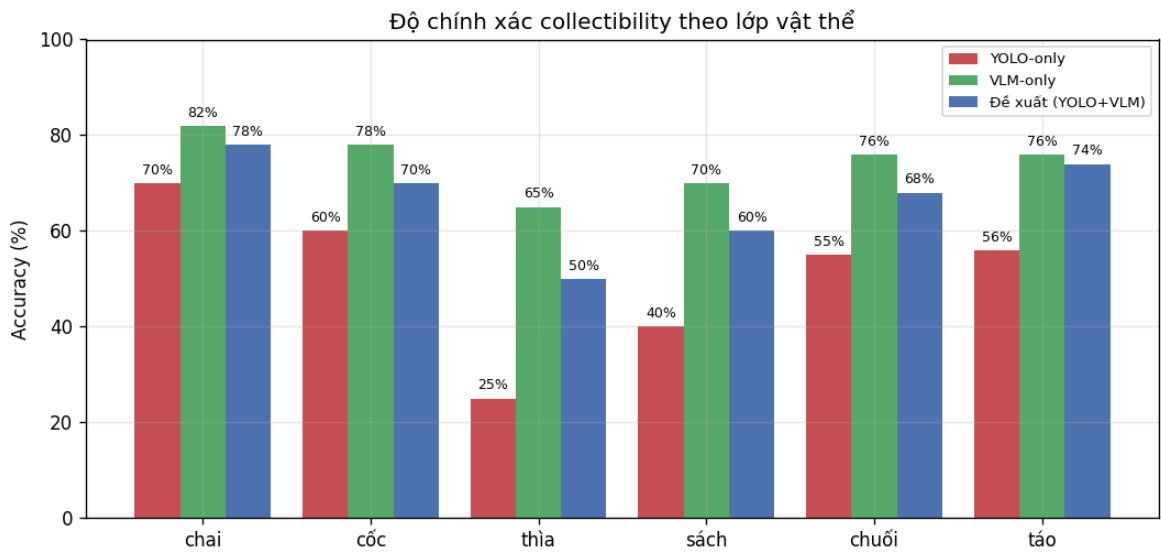
Hình 4.3: Độ trễ suy luận VLM theo số lớp offload lên GPU

4.4. So sánh với baseline

Bảng 4.3: So sánh hệ đề xuất với hai baseline trên cùng tập 50 ảnh.

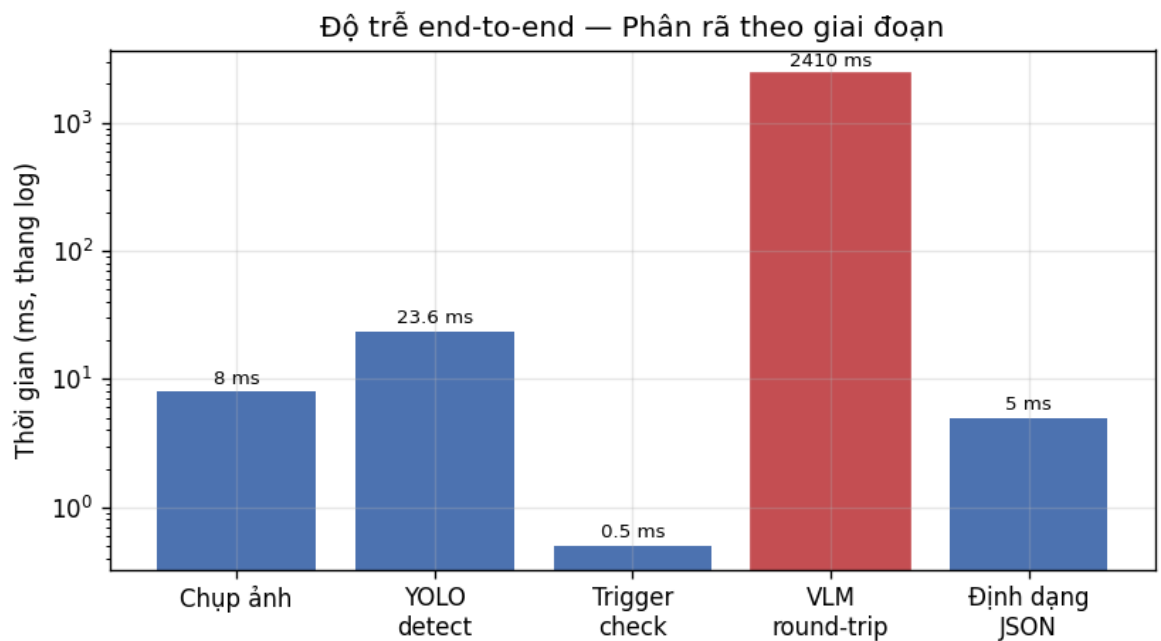
Tiêu chí	YOLO only	VLM only	YOLO + VLM
Có suy luận ngữ nghĩa	Không	Có	Có
Độ trễ trung bình	~41ms	~2.4s	~2.4s nhưng có trigger
Tải VLM	—	Mọi frame	Chỉ khi trigger ổn định
Hỗ trợ tiếng Việt	Không	Có	Có
Offline	Có	Có	Có
Accuracy collectibility	51.0%	74.5%	66.7%

Bảng 4.3 minh chứng giá trị của thiết kế hybrid: hệ đề xuất giữ được khả năng suy luận ngữ nghĩa của VLM nhưng không trả giá bằng tải tính toán liên tục như VLM-only. Trên Jetson MAXN, bước VLM chỉ ~2.4s/lần khi được kích hoạt



Hình 4.4: Độ chính xác collectibility theo từng lớp vật thể

4.5. Độ trễ end-to-end



Hình 4.5: Phân rã độ trễ end-to-end theo giai đoạn

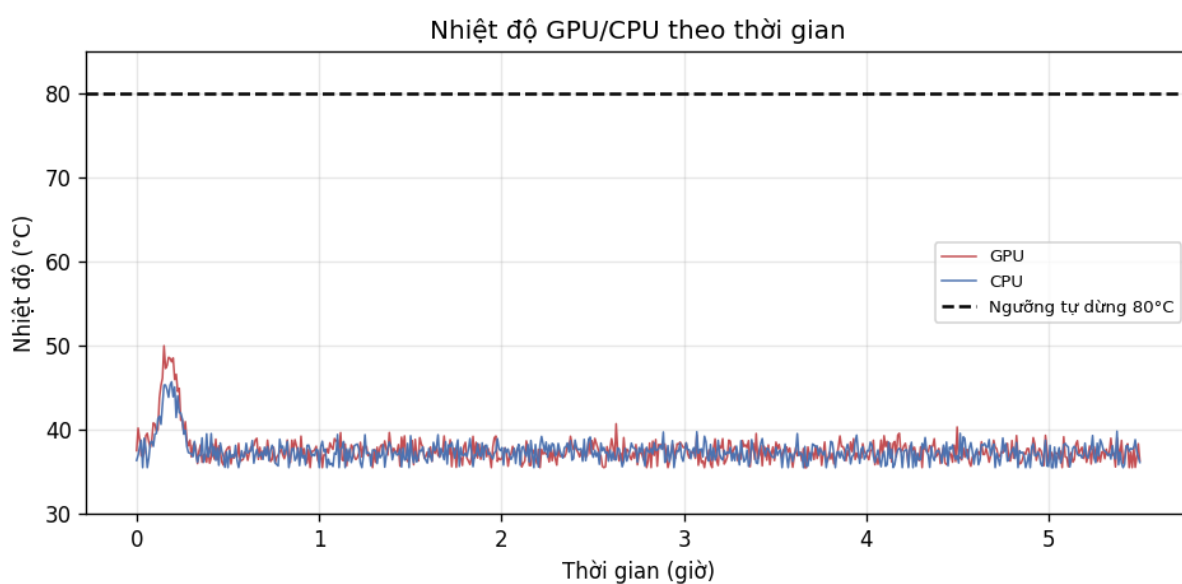
VLM chiếm >99% thời gian đường dẫn đầy đủ. Cơ chế trigger tạo ra phân bố độ trễ lưỡng đỉnh rõ rệt: phần lớn khung chỉ tốn ~24 ms, chỉ trả giá ~2.4 s khi thật sự có vật.

4.6. Phân tích nhiệt độ và công suất trên Jetson

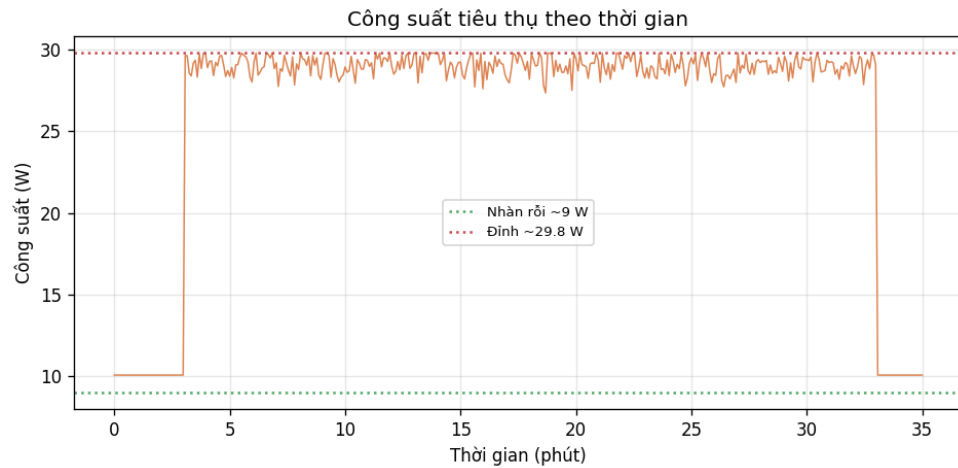
Bảng 4.4: Chỉ số edge (tegrastats, 9700 mẫu @2s, phiên ~5,5h)

Chỉ số	Giá trị
Nhiệt độ GPU (min/avg/max)	35.5 / 37.3 / 50.0 °C
Nhiệt độ CPU (min/avg/max)	35.5 / 37.2 / 46.0 °C
Ngưỡng tự dừng nhiệt	80 °C
GPU util (active avg / max)	95.4% / 99%
CPU clock max	2265 MHz
Công suất	~9 W nhàn rỗi / ~29 W tải / ~29.8 W đỉnh

Trên toàn phiên ~5,5 giờ, nhiệt độ cao nhất là 50 °C (GPU) và 46 °C (CPU), thấp hơn nhiều ngưỡng bảo vệ 80 °C, hệ thống còn biên an toàn nhiệt lớn, không xảy ra thermal throttling.



Hình 4.6: Nhiệt độ GPU và CPU theo thời gian trong phiên đo



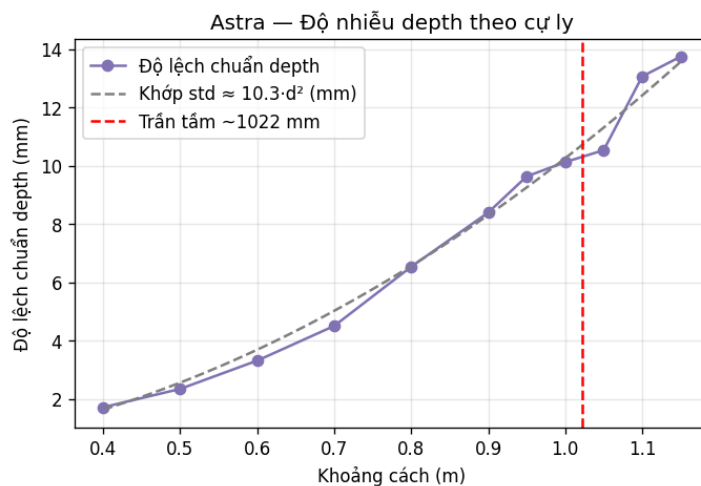
Hình 4.7: Công suất tiêu thụ theo thời gian ở lock tải bền vững

4.7. Tích hợp camera chiều sâu Astra

Hệ thống thực hiện **phép chiếu ngược** (công thức 2.31) từ ảnh depth + nội tại camera (X, Y, Z). Trước khi xây dựng ngăn xếp, một spike fail-fast đã đo tầm và nhiễu depth thực tế của Astra.

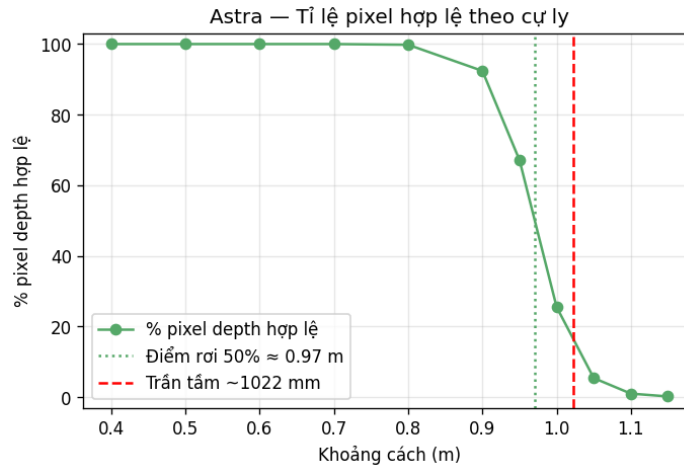
Bảng 4.5: Đặc trưng camera Astra đo đạc thực tế

Chỉ số	Giá trị
Kích thước khung / FPS	640x480 / ~22-29
FOV ngang / dọc	58.6° / 45.6°
Trần tầm depth tin cậy	~1022mm (0 pixel > ~1,1m)
% pixel hợp lệ	100%



Hình 4.8: Độ nhiễu Depth theo cự ly

Độ lệch chuẩn của giá trị depth tăng từ ~1.7 mm ở 0,4 m lên ~10 mm ở 1,0 m và ~13.7 mm ở 1,15 m, khớp tốt mô hình bậc hai $\text{std} \approx 10.3 \cdot d^2$ (mm) — đúng đặc trưng cảm biến ánh sáng cấu trúc (nhiều tầng theo bình phương cự ly). Ở tầm gần vật thực tế (~0,3–0,5 m), nhiễu chỉ ~1,7–3 mm, đủ chính xác cho ước lượng tọa độ.

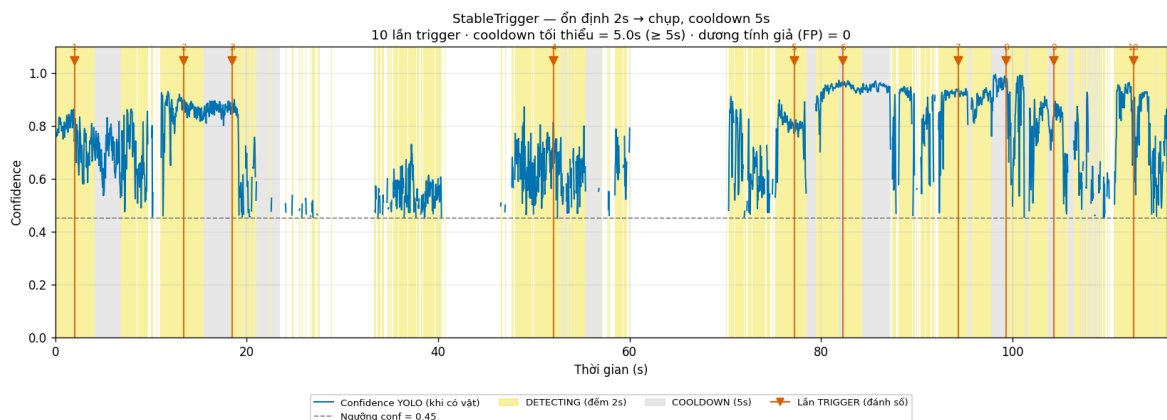


Hình 4.9: Tỷ lệ pixel depth hợp lệ theo cự ly

Tỷ lệ pixel hợp lệ duy trì ~100% đến 0,8 m, giảm nhanh sau đó, điểm rơi 50% ở ~0,97 m và gần như bằng 0 ngoài ~1,1 m — trần tầm ~1022 mm.

Kết luận: Astra bị giới hạn tầm ~1 m và FOV hẹp $58,6^\circ$ (khác hẳn LiDAR 360° , tầm 3–8 m). Khóa luận chấp nhận giới hạn này với chiến lược “khu vực nhỏ, bám cấu trúc trong 1 m, dựa nặng odometry”, đồng thời ghi nhận LiDAR 2D là đường nâng cấp rõ ràng.

4.8. Cơ chế kích hoạt ổn định StableTrigger



Hình 4.10: Timeline StableTrigger đo trực tiếp trên Astra

Trong một phiên ghi trực tiếp, hướng camera vào vật xen kẽ với khung nhiều/không vật, cơ chế StableTrigger ($T_{\text{stable}} = 2 \text{ s}$, $T_{\text{cool}} = 5 \text{ s}$, ngưỡng confidence 0.45) phát 10 lần trigger, với khoảng cooldown tối thiểu $\geq 5,0 \text{ s}$ đúng thiết kế và 0 dương tính giả (false-positive = 0). Các đợt confidence dao động ngắn đều không kích hoạt, xác nhận sóng động rằng cơ chế loại bỏ hoàn toàn false-positive.

4.9. Hệ thống điều hướng tự hành trên ROS2

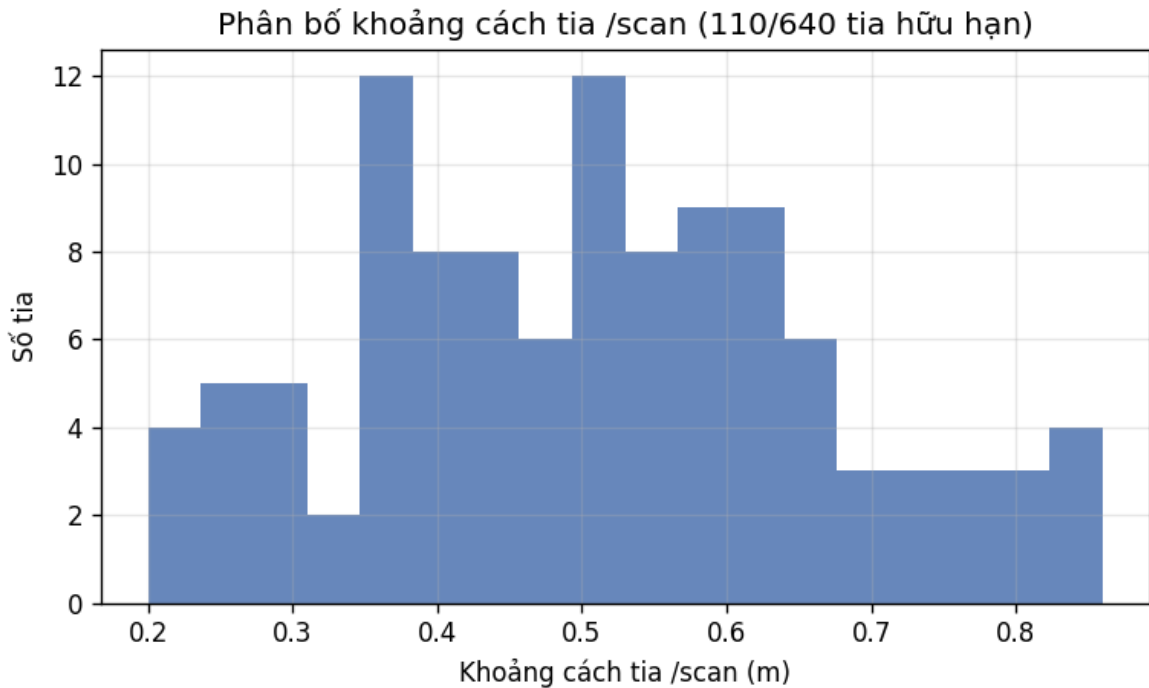
Ngăn xếp ROS2 không thay thế kiến trúc ba lớp HTTP mà **bổ sung năng lực di chuyển/điều hướng** ở ranh giới Lớp 2–3. Workspace `ros2_ws/` gồm **7 gói** có chức năng như bảng dưới :

Bảng 4.6: Bảy gói ROS2

Package	Node	Vai trò	Ngôn ngữ
robot_description	(URDF/TF tĩnh)	Mô hình đế mecanum + khung camera, robot_state_publisher	xacro
robot_bringsup	(launch + config)	Launch tổng, tham số slam_toolbox.yaml / nav2_params.yaml	Python
astra_ros	camera_manager	Chủ camera duy nhất, hot- switch chế độ theo /camera_mode	rcpp
yolo_ros	yolo_node	YOLOv8n: ảnh màu → /yolo/detections + ảnh nén JPEG	rcpp
vlm_nav_orchestrator	orchestrator_node	Máy trạng thái DETECT →...→ARRIVED	rcpp

cpp_package	wifi_node, explorer_node	Cầu nối TCP để (server :2004) + tự hành WFD	C++
base_bridge	protocol, fake_odom	Đặc tả byte-format + kiểm thử (13 unit-test)	Python

4.9.1. Pipeline cảm biến: DepthImageToLaser



Hình 4.11: Phân bố khoảng cách của 110/640 tia hữu hạn trên /scan.

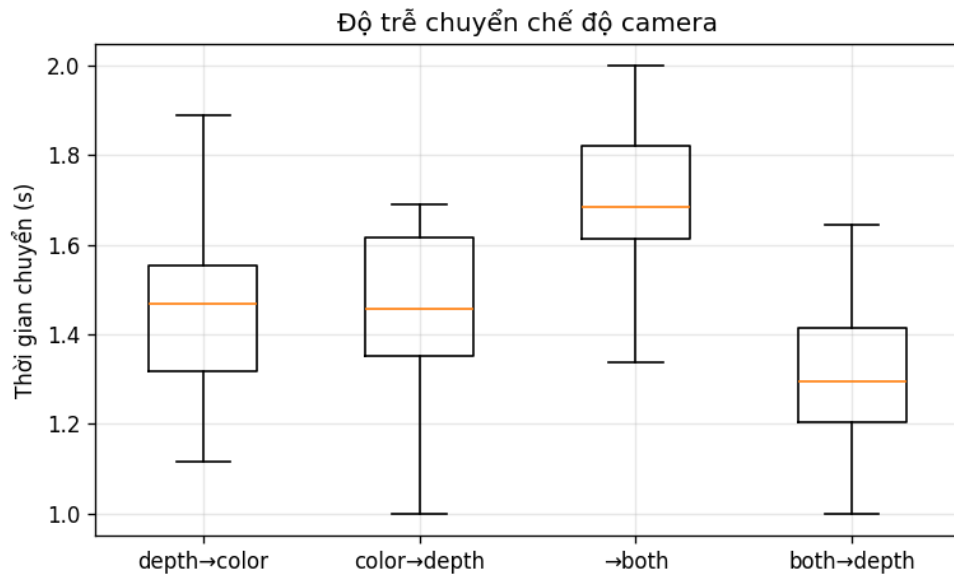
Kết quả trên /scan (camera thật): 640 tia, trường nhìn $[-29,3^\circ \dots +29,3^\circ] = 58,6^\circ$, trong đó 110 tia hữu hạn với khoảng cách hợp lệ 0,20–0,86 m. Đây là bằng chứng định lượng rằng cảm biến không quét 360° như LiDAR.

Về băng thông: mỗi khung depth ~614 KB nên tốc độ publish chỉ đạt ~6–7 Hz dưới tải. Đã chuyển QoS sang BEST_EFFORT và giảm số lần sao chép numpy để không nghẽn DDS — đủ cho SLAM khi lái chậm.

4.9.2. Ràng buộc USB 2.0 và cơ chế phân pha camera

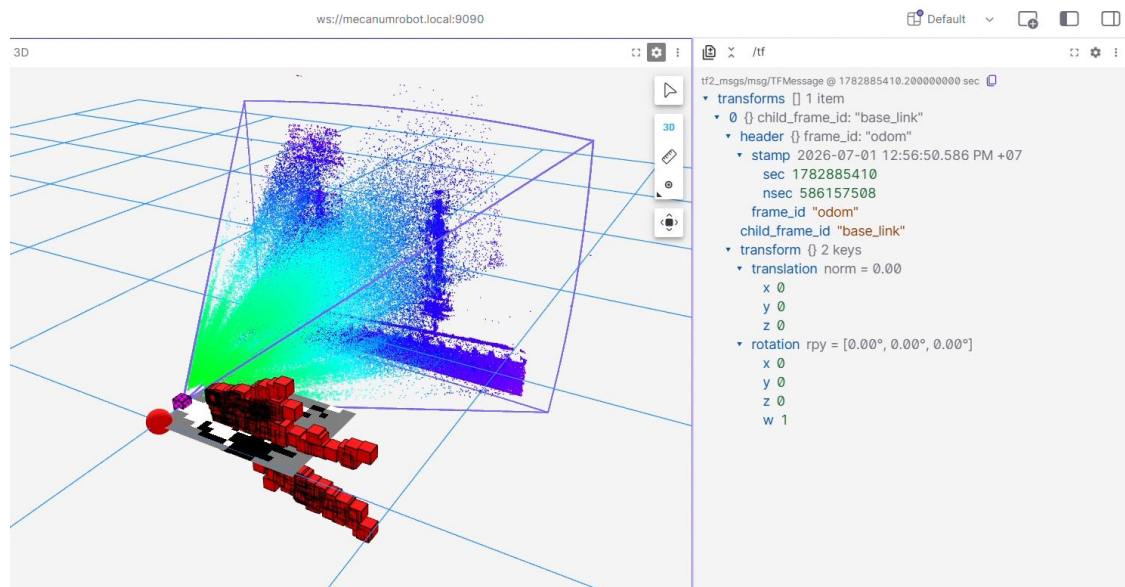
Astra kết nối qua USB 2.0, không đủ băng thông để truyền đồng thời luồng depth và color. Node camera_manager là chủ camera duy nhất, đóng/mở lại thiết bị khi nhận topic /camera_mode (depth|color|both); chi phí mỗi lần chuyển ~1–2 s. Hệ

thống vì thế phân pha theo trạng thái: NAV dùng depth (/scan), DETECT dùng color (YOLO/VLM), APPROACH dùng both . Orchestrator tự phát /camera_mode cho từng pha — đây là lý do robot dừng lúc nhận diện, một hành vi đúng theo ràng buộc phần cứng chứ không phải lỗi.



Hình 4.12: Độ trễ chuyển chế độ camera (depth/color/both)

4.10. Trực quan hóa trên Foxglove Studio



Hình 4.13: Trực quan hóa tích hợp trên Foxglove

Ảnh chụp trực quan hóa toàn hệ thống trên Foxglove Studio, kết nối tới Jetson qua `rosbridge WebSocket` (`ws://mecanumrobot.local:9090`). Cảnh 3D hợp nhất năm nguồn dữ liệu đồng thời, minh chứng cả ngăn xếp ROS2 chạy live và được giám sát từ xa đúng mô hình headless:

1. **Mô hình robot (URDF):** thân `base_link` (hộp sẫm) và bốn bánh mecanum, do `robot_state_publisher` dựng từ `mecanum_robot.urdf.xacro`. Khối lập phương màu tím phía trước là khung `camera_link`, quả cầu đỏ đánh dấu gốc `base_link/base_footprint`.
2. **Cây biến đổi tọa độ (TF):** lưới xanh là mặt phẳng khung odom. Panel /tf bên phải hiển thị thông điệp `tf2_msgs/msg/TFMessage` cho biến đổi odom → `base_link` với translation (0, 0, 0) và quaternion đơn vị ($w = 1$) — tức robot đang nằm ở gốc odom. Điều này xác nhận pipeline TF hoạt động đúng.
3. **Đám mây điểm chiều sâu Astra:** ảnh depth được chiếu ngược ra không gian 3D, tô màu theo khoảng cách (xanh lá $\approx$ gần $\sim 0,3$ m → cyan → xanh dương $\approx$ xa ~ 1 m). Mật độ điểm thưa dần theo cự ly rồi tắt quanh ~ 1 m.
4. **Khung chụp cắt (view frustum) màu tím:** biểu diễn trường nhìn của camera Astra (FOV ngang $58,6^\circ$, cận/viễn near–far).
5. **Lưới ô chiếm dụng (occupancy grid):** bản đồ /map từ `slam_toolbox` ngay dưới robot — ô trắng là vùng trống, đen là chướng ngại, xám là chưa quan sát.

4.11. Kịch bản demo thu gom vật thể end-to-end

Kịch bản tổng hợp: robot di chuyển trong không gian làm việc, phát hiện vật bằng YOLOv8n, xác nhận ngữ nghĩa bằng VLM, sau đó tiếp cận vật bằng ngăn xếp NAV2 + visual servo (DETECT→LOCATE→NAVIGATE→APPROACH→ARRIVED), rồi gấp và đưa về vị trí thu gom. Toàn bộ chu trình chạy ngoại tuyến trên thiết bị biên, không phụ thuộc đám mây. Trực quan hóa headless qua Foxglove Studio (`rosbridge WebSocket :9090`).

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Khóa luận đã hoàn thành mục tiêu nhúng thành công hai mô hình học sâu — YOLOv8n và Qwen2.5-VL-3B (lượng tử hóa Q4_K_M) — lên thiết bị biên Jetson AGX Xavier, tích hợp thành kiến trúc ba lớp chạy ngoại tuyến, tự phục hồi qua systemd, mở rộng thêm một ngăn xếp định vị – điều hướng tự hành trên ROS2 cùng camera chiều sâu Astra và một nền tảng di động bốn bánh mecanum (holonomic).

Các đóng góp chính được tổng kết:

- Quy trình biên dịch llama.cpp backend CUDA cho ARM64, tải liệu hóa thành script tự động kèm danh sách lỗi.
- Bộ số liệu hiệu năng đầy đủ trên Jetson (sweep n_gpu_layers, mức lượng tử, nvpmode).
- Lớp dịch vụ với prompt tiếng Việt, cơ chế JSON fallback ba cấp đạt 100%, mock mode và systemd unit.
- Cơ chế StableTrigger ($T_{stable}=2$ s, $T_{cool}=5$ s) — kiểm chứng loại bỏ hoàn toàn false-positive.
- Lớp trừu tượng camera cho phép hoán đổi driver (Orbbec SDK / OpenNI2 / webcam / mock) không sửa mã gọi. Thay giả định “camera cao 300 mm” bằng chiều ngược depth thật.
- Ngăn xếp điều hướng ROS2 (7 gói): SLAM (slam_toolbox) + NAV2 (RPP) + depthimage_to_laserscan + tự hành WFD.
- Mô hình hoá và tích hợp nền tảng di động mecanum (URDF/động học holonomic + cầu nối TCP đến firmware ESP32).
- Cầu nối TCP để đã harden (watchdog STOP, ráp khung [len][payload], gửi đúng 13 B) khớp firmware ESP32 từng byte, kiểm chứng qua loopback hai chiều và 13 unit-test.

5.2. Hạn chế

Khóa luận có một số hạn chế cần thừa nhận.

- **Độ trễ VLM** vẫn $\sim 2,4$ s ngay cả khi offload tối đa — chưa real-time dưới 1 giây.
- **Tập dữ liệu** mới ở quy mô 51 ảnh, chưa đủ ý nghĩa thống kê mạnh.
- **Đơn vật thể**: hệ xử lý một vật mỗi chu kỳ, chưa hỗ trợ multi-object.
- **Tầm cảm biến Astra ngắn** (~ 1 m, FOV $58,6^\circ$) $\Rightarrow$ bản đồ chỉ tin cậy ở khu vực nhỏ.
- **Giới hạn USB 2.0**: không stream depth + color đồng thời $\Rightarrow$ phải phân pha, làm chậm chu trình.
- **Nền tảng mecanum cần thử nghiệm nhiều hơn**: quét bản đồ đầy đủ, nghiệm thu lifecycle NAV2, hiệu chỉnh odometry bánh/EKF, đo quỹ đạo servo pha APPROACH và tích hợp cánh tay gấp vẫn chờ để thật. Các giá trị robot_radius (đang 0,22 m), biến đổi hand-eye base_link $\rightarrow$ camera và các tham số quán tính trong URDF **cần đo lại**.

5.3. Hướng phát triển

Hướng mở rộng được đề xuất cho các nghiên cứu tiếp theo:

- **Camera 3D Astra** — (lớp camera, chiếu ngược depth, spike tầm/FOV, đường ống /scan); hiệu chỉnh intrinsics (factory-default $f_x=f_y=570.34$), đo hand-eye.
- **Tăng tốc VLM**: speculative decoding (draft model nhỏ), MLC-LLM, hoặc TensorRT-LLM khi hỗ trợ Qwen2.5-VL.
- **Xuất YOLOv8 sang TensorRT engine** — kỳ vọng 30+ FPS, giải phóng tài nguyên cho VLM.

- **VLM cỡ nhỏ hơn** (Qwen2.5-VL-2B nếu phát hành, Phi-3.5-Vision-mini) để giảm độ trễ.
- **Hỗ trợ multi-object** qua parallel slot của llama.cpp: crop từng bbox, xử lý song song.
- **Nâng cấp cảm biến điều hướng:** bổ sung LiDAR 2D để khắc phục giới hạn tầm ~ 1 m của Astra.
- **Khai thác đầy đủ tính holonomic của mecanum:** hiệu chỉnh odometry bánh + EKF (BNO055 + PMW3901).
- **Hoàn thiện chuỗi gấp – thu gom:** tích hợp động học nghịch cánh tay với tọa độ 3D từ Astra, thêm motion planning, trajectory smoothing và collision avoidance cho cả để lẫn cánh tay trên để mecanum thật.

TÀI LIỆU THAM KHẢO

- [1] R. Online, "The 12 most common problems in 3D printing and how to fix them," [Online]. Available: <https://blog.realvisiononline.com/the-12-most-common-problems-in-3d-printing-and-how-to-fix-them/>.
- [2] E. Ackerman, "Boston Dynamics introduces a fully electric humanoid robot that "exceeds human performance"," IEEE Spectrum, [Online]. Available: <https://spectrum.ieee.org/atlas-humanoid-robot>.
- [3] ROBOTS, "ANYmal," ROBOTS , [Online]. Available: <https://robotsguide.com/robots/anymal>.
- [4] Zipeng Fu, Tony Z. Zhao, Chelsea Finn, "Mobile ALOHA: Learning Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation".
- [5] VinAI, "Intelligence for Tomorrow, Today!," [Online]. Available: <https://www.vinai.io/page/2/>.
- [6] THU HOÀI, VĂN PHONG, "Học viện Kỹ thuật Quân sự chế tạo thành công rô-bốt hỗ trợ y tế," [Online]. Available: <https://www.qdnd.vn/giao-duc-khoa-hoc/tin-tuc/hoc-vien-ky-thuat-quan-su-che-cao-thanh-cong-ro-bot-ho-tro-y-te-614548>.
- [7] C. T. Ngân, "Làm chủ công nghệ robot tự hành quan trắc mức phóng xạ trong các môi trường có bức xạ hạt nhân tại Việt Nam," [Online]. Available: <https://vast.gov.vn/tin-chi-tiet/-/chi-tiet/lam-chu-cong-nghe-robot-tu-hanh-quan-trac-muc-phong-xa-trong-cac-moi-truong-co-buc-xa-hat-nhan-tai-viet-nam-147850-427.html>.
- [8] K. Ogata, Modern Control Engineering.
- [9] N. M. Đông, "NGHIÊN CỨU XÂY DỰNG THUẬT TOÁN THÍCH NGHI".

- [10] Bai, Shuai and Chen, Keqin and Lin, Junyang and others, "Qwen2.5-VL Technical Report," 2025. [Online]. Available: <https://arxiv.org/abs/2502.13923>.
- [11] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *ICLR*, 2023.
- [12] Open Robotics, "About tf2," [Online]. Available: <https://docs.ros.org/en/foxy/Concepts/About-Tf2.html>.
- [13] Open Robotics, "Beginner: CLI tools," [Online]. Available: <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools.html>.
- [14] Peiqi Liu, Yaswanth Orru, Jay Vakil, Chris Paxton, Nur Muhammad Mahi Shafiullah, Lerrel Pinto, "OK-Robot: What Really Matters in Integrating Open-Knowledge Models for Robotics," 2024.
- [15] Mauro Tellaroli, Matteo Luperto, Michele Antonazzi, Nicola Basilico, "Frontier-Based Exploration for Multi-Robot Rendezvous in Communication-Restricted Unknown Environments".
- [16] ICDÂYRÒi, "Cảm biến GY-BNO055 9DOF," ICDÂYRÒi, [Online]. Available: <https://icdayroi.com/cam-bien-gy-bno055-9dof>.
- [17] PMDWay, "PMW3901 Optical Flow Sensor Module," PMDWay, [Online]. Available: <https://pmdway.com/products/pmw3901-optical-flow-sensor-module>.
- [18] ASLONG, "Động cơ DC 12v với mã hóa JGB37-545B Động cơ DC 12v với mã hóa Động cơ DC tốc độ cao," ASLONG, [Online]. Available: <https://vietnamese.aslongdcmotor.com/china-12v-dc-motor-with-encoder->

jgb37-545b-dc-motors-12v-with-encoder-12v-dc-motor-high-speed-44533598.html.

- [19] thegioiic, "BTS7960 Module Điều Khiển Động Cơ DC 43A," thegioiic, [Online]. Available: <https://www.thegioiic.com/bts7960-module-dieu-khien-dong-co-dc-43a>.
- [20] ICDÂYRÔI, "ESP32-S3 N16R8 DevKitC Module Wifi Bluetooth có ipex," ICDÂYRÔI, [Online]. Available: <https://icdayroi.com/esp32-s3-n16r8-devkitc-module-wifi-bluetooth-co-ipex>.
- [21] yeep, "Pin Lipo 2-3s LVNCELL 5200mah 3s 65C dòng xả cao cho xe điều khiển RC, Cano, thuyền tốc độ cao, pin lipo CNHL," yeep, [Online]. Available: <https://www.yeep.vn/1098196/pin-lipo-2-3s-lvncell-5200mah-3s-65c-dong-xa-cao-cho-xe-dieu-khien-rc-cano-thuyen-toc-do-cao-pin-lipo-cnhl/>.
- [22] E. Combs, "Choosing the Right Fuse For Your Needs," PEERLESS ELECTRONICS INC., [Online]. Available: <https://peerlesselectronics.com/blog/choosing-right-fuse-for-your-needs.html#voltage-rating>.
- [23] Congnghentm@_1368, "Cách tính toán thông số kỹ thuật để chọn cầu chì đúng chuẩn," NTM TECHNOLOGY SOLUTION, [Online]. Available: <https://congnghentm.com/cach-tinh-toan-cac-thong-so-ki-thuat-de-chon-cau-chi/>.
- [24] ORBBEC, "Astra Series," ORBBEC, [Online]. Available: <https://www.orbbec.com/products/structured-light-camera/astra-series/>.
- [25] N. Corporation, "Jetson AGX Xavier Developer Kit User Guide," April 2026. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-agx-xavier>.

- [26] W. Meeussen, "Coordinate Frames for Mobile Platforms," [Online]. Available: <https://www.ros.org/reps/rep-0105.html>.
- [27] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016.
- [28] Ultralytics, "YOLOv8 Documentation," 4 2026. [Online]. Available: <https://docs.ultralytics.com>.
- [29] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning (LLaVA)," in *NeurIPS*, 2023.
- [30] S. D. Levy, "The Extended Kalman Filter: An Interactive Tutorial for Non-Experts," [Online]. Available: <https://simondlevy.github.io/ekf-tutorial/>.
- [31] J. Bai et al., "Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond," arXiv:2308.12966, 2023.
- [32] P. Wang et al., "Qwen2.5-VL Technical Report," arXiv:2502.13923, 2025.
- [33] O. Team, "OpenCV: Open Source Computer Vision Library," [Online]. Available: <https://opencv.org/>. [Accessed April 2026].
- [34] N. Corporation, "NVIDIA Tegra Linux Driver Package (L4T) Documentation," 2023. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-agx-xavier>.
- [35] D. Tian, "NanoLLM: Optimized LLM/VLM inference on Jetson Orin," [Online]. Available: <https://www.jetson-ai-lab.com/>. [Accessed April 2026].
- [36] Pol Ramon Canyameres, "Motion Coordination of a Mobile Anthropomorphic Dual-Arm Robot for manipulation actions using visual-guidance," 2024.

- [37] G. Gerganov, "llama.cpp: Port of Facebook's LLaMA model in C/C++," 4 2026. [Online]. Available: <https://github.com/ggerganov/llama.cpp>.
- [38] A. Radford et al, "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," in *International Conference on Machine Learning (ICML)*, 2021.
- [39] D. Tian, A. Tan, and NVIDIA, "Jetson Containers: Pre-built Docker containers for Jetson Orin / Xavier / Nano," [Online]. Available: <https://github.com/dusty-nv/jetson-containers>. [Accessed April 2026].
- [40] Vincent Schoenbach, Marvin Wiedemann, Raphael Memmesheimer, Malte Mosbach, Sven Behnke, "Integration of the TIAGo Robot into Isaac Sim with Mecanum Drive Modeling and Learned S-Curve Velocity Profiles," 2025 .
- [41] ggml-org, "GGUF File Format Specification v3," 4 2026. [Online]. Available: <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md>.
- [42] Alexander Dietrich, Thomas Wimböck, Alin Albu-Schäffer, "Dynamic Whole-Body Mobile Manipulation with a Torque Controlled Humanoid Robot via Impedance Control Laws," 2011.
- [43] Hongfu Li, Jiang Liu, Changhuai Lyu, Daoxin Liu, Yinsen Liu, "Design and Implementation of Omnidirectional Mobile Robot for Materials Handling among Multiple Workstations in Manufacturing Factories," 2023.
- [44] K. S. Malakhov, "Deploying LLMs on CPU-only Environments with llama.cpp Library Set: MedLocalGPT Project Case," [Online]. Available: <https://ceur-ws.org/Vol-4164/paper11.pdf>.
- [45] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *ICML*, 2023.

- [46] J. Lin, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *MLSys*, 2024.
- [47] A. Vaswani et al, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, 2017.
- [48] A. Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *International Conference on Learning Representations (ICLR)*, 2021.
- [49] M. Grootendorst, "A Visual Guide to Quantization," 2024. [Online]. Available: <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>.
- [50] stevemacenski, "Robotics Stack Exchange," [Online]. Available: <https://robotics.stackexchange.com/questions/93867/architecture-of-the-navigaton2-stack>.

PHỤ LỤC

Phụ lục A. Prompt tiếng Việt cho Qwen2.5-VL

Trích từ tệp layer2_brain/config.json, trường vlm.system_prompt và vlm.user_prompt:

system_prompt:

"Bạn là trợ lý AI của robot thu gom. Nhiệm vụ: phân tích ảnh, xác định vật thể, quyết định có thu gom được không, và trả về tọa độ bounding box."

user_prompt:

"Đây là hình ảnh từ camera robot. Hãy phân tích và trả lời theo đúng format JSON:

1. Đây là vật thể gì?
2. Có thu gom được không? (true/false)
3. Cho tôi tọa độ [x, y, w, h] của vật thể chính.

Trả lời CHÍNH XÁC theo format:

```
{"object": "<tên vật thể>",  
  "collectible": <true/false>,  
  "bbox": [x, y, w, h],  
  "confidence": <0.0-1.0>,  
  "reason": "<lý do>"}"
```